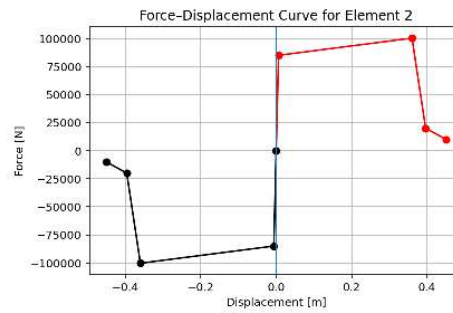
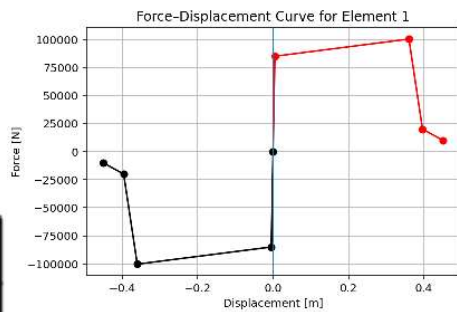


>> IN THE NAME OF ALLAH, THE MOST GRACIOUS, THE MOST MERCIFUL <<

COMPREHENSIVE NONLINEAR SEISMIC ASSESSMENT OF A MULTI-DEGREE- FREEDOM ELEVATOR AND STRUCTURE SYSTEM: AN OPENSEES FRAMEWORK FOR STATIC PUSHOVER, CYCLIC DEGRADATION, STATIC TIME-HISTORY AND DYNAMIC TIME-HISTORY ANALYSIS

THIS PROGRAM IS WRITTEN BY SALAR DELAVAR GHASHGHAEE (QASHQAI)



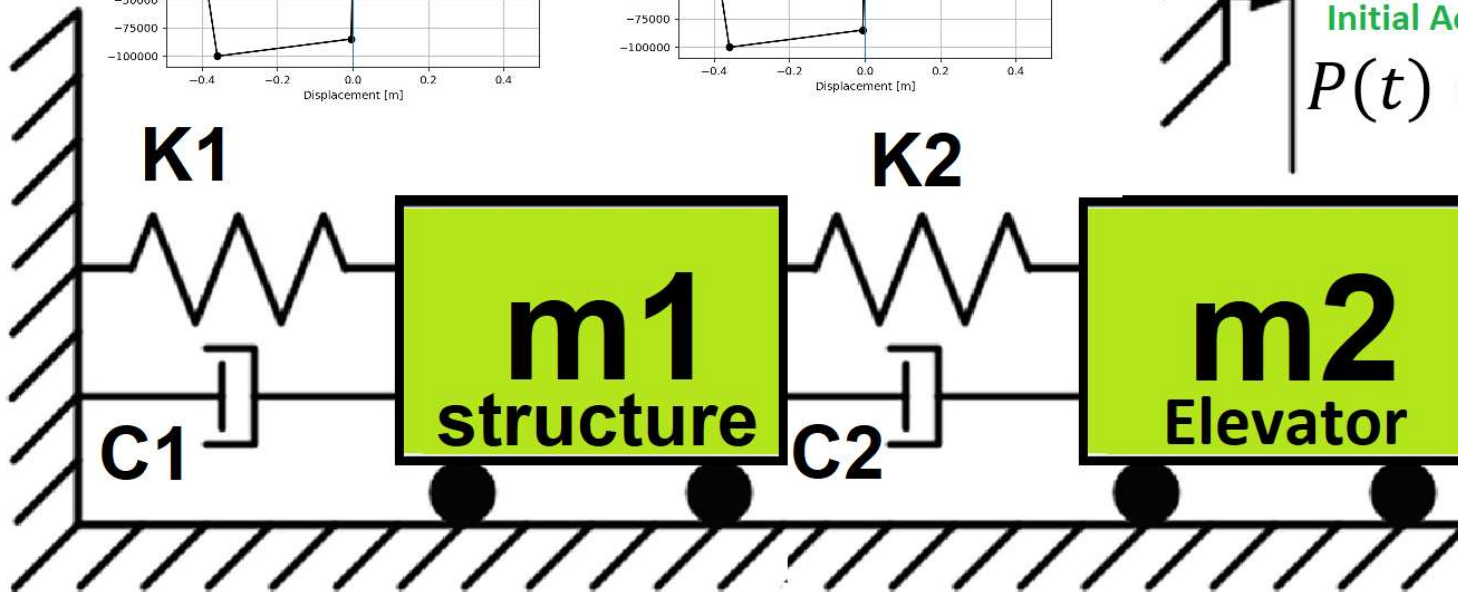
➡ INCREMENTAL
DISPLACEMENT



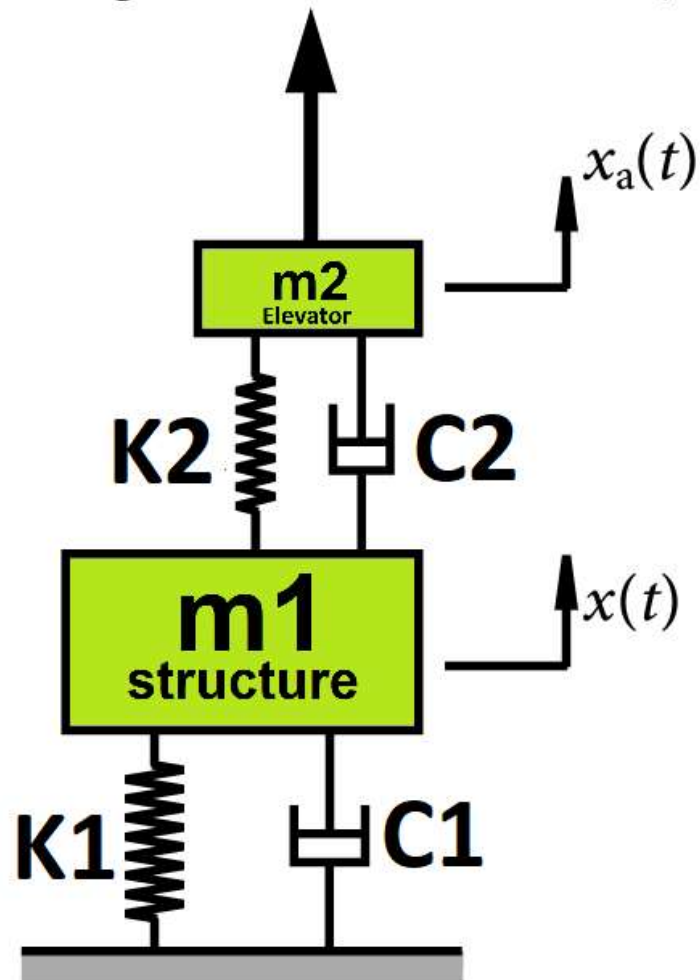
Initial Displacement
Initial Velocity
Initial Acceleration



$$P(t) = P_0 e^{-0.05\bar{\omega}t} \sin(\bar{\omega}t)$$



$$P(t) = P_0 e^{-0.05\bar{\omega}t} \sin(\bar{\omega}t)$$



$$\text{Structural Ductility Damage Index} = \frac{\Delta_d - \Delta_y}{\Delta_u - \Delta_y}$$

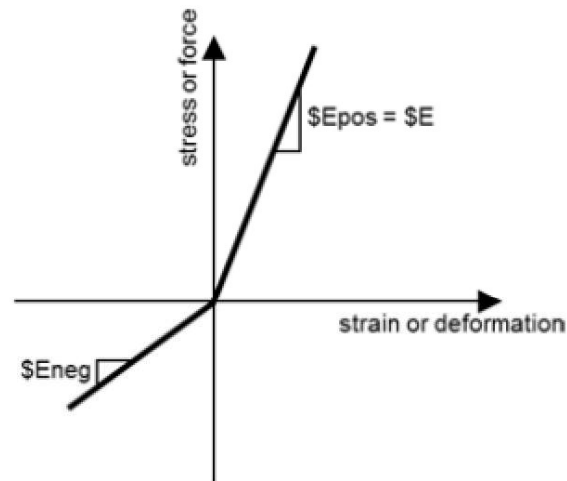
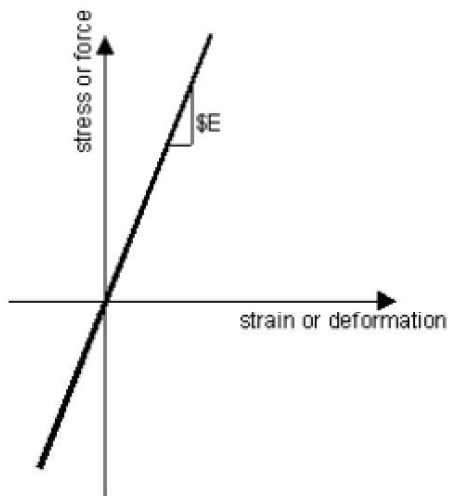
Δ_d = Lateral Displaement from Dynamic Analysis

Δ_y = Lateral Yield Displaement from Pushover Analysis

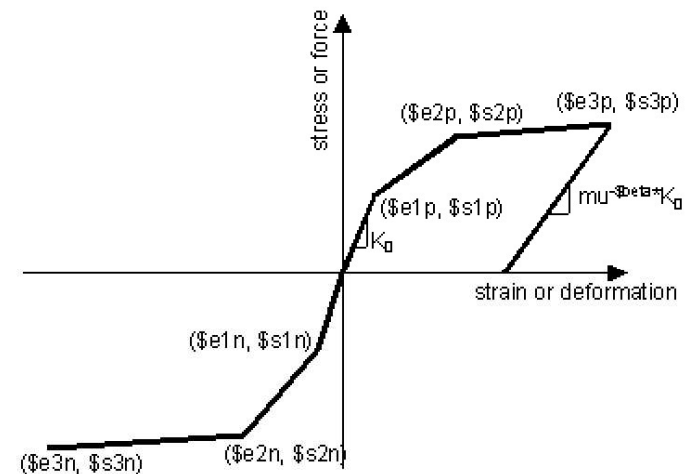
Δ_u = Lateral Ultimate Displaement from Pushover Analysis

$$P(t) = P_0 e^{-0.05\bar{\omega}t} \sin(\bar{\omega}t)$$

$$m\ddot{u} + c\dot{u} + ku = P(t)$$

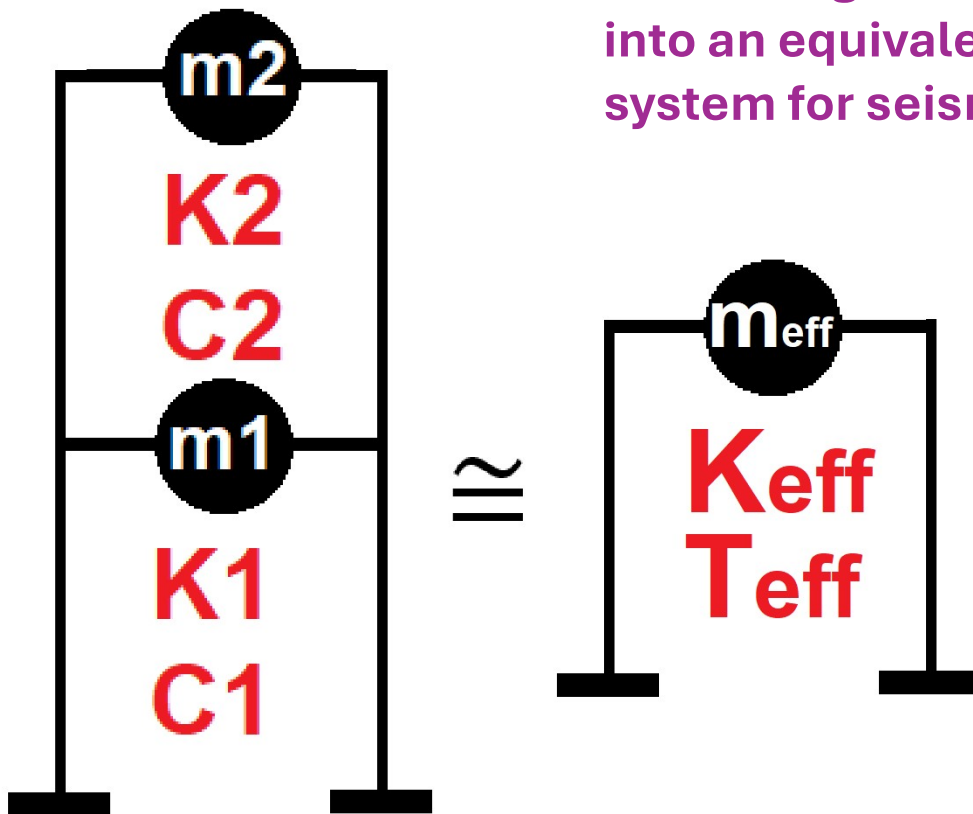


ELASTIC MATERIAL



INELASTIC MATERIAL

Displacement-based seismic analysis transformation, converting a multi-degree-of-freedom (MDOF) system into an equivalent single-degree-of-freedom (SDOF) system for seismic assessment



$$\Delta_d = \frac{\sum_{i=1}^n m_i \Delta_i^2}{\sum_{i=1}^n m_i \Delta_i} \quad m_{eff} = \frac{\sum_{i=1}^n m_i \Delta_i}{\Delta_d}$$

$$k_{eff} = \frac{\sum_{i=1}^n k_i \Delta_i}{\Delta_d} \quad T_{eff} = \frac{2\pi}{\sqrt{\frac{k_{eff}}{m_{eff}}}}$$

FileEditSearchSourceRunDebugConsolesProjectsToolsViewHelp

C:\Users\ DELL\Desktop\OPENSEES_FILES\+EIGHT_ANALY..._MDOF_8_ANA\p(t)_ELEVATOR_STRUCTURE_MDOF_8_ANA.py

P(t)_ELEVATOR_STRUCTURE_MDOF_8_ANA.py X

1#####

2#

3#>> IN THE NAME OF ALLAH, THE MOST GRACIOUS, THE MOST ME

4#COMPREHENSIVE NONLINEAR SEISMIC ASSESSMENT OF A MULTI-DEGREE-FREEDOM EL

5#AN OPENSEES FRAMEWORK FOR STATIC PUSHOVER, CYCLIC DEGRADAT

6#STATIC TIME-HISTORY AND DYNAMIC TIME-HISTORY ANAL

7#-----

8#EQUIVALENT SDOF SYSTEM DERIVATION VIA DISPLACEMENT-BASED SEISMIC DESIGN PR

9#-----

10#P(t) = P0 sin(wt)

11#P(t) = P0 exp(-0.05wt) sin(wt)

12#-----

13#ASSESSMENT OF DUCTILITY DAMAGE INDICES FOR STRUCTURAL ELEMENTS AN

14#OF ENERGY DISSIPATION CAPACITY IND

15#-----

16#THIS PYTHON SCRIPT WRITTEN BY SALAR DELAVAR GHASHGHAEI

17#EMAIL: salar.d.ghashghaei@gmail.com

18#-----

19Nonlinear Seismic Performance Assessment of a MDOF Elevator and Structure S

20An OpenSeesPy Framework for Material and Geometric Nonlinearity Under Stati

21

22This OpenSeesPy script performs rigorous nonlinear static and dynamic analy

23for performance-based earthquake engineering. The 2D model incorporates bo

24(elastic-perfectly plastic or hysteretic steel with strain hardening)

25and geometric nonlinearity (corotational truss formulation) to capture P-d

26and large displacements-essential for collapse assessment.

27

28Eight analysis protocols are implemented:

29(1) [PERIOD] : Structural Period

30(2) [STATIC] : Gravity load analysis establishing dead load state

31(3) [PUSHOVER] : Displacement-controlled pushover generating full capacity

32and plastic mechanism identification

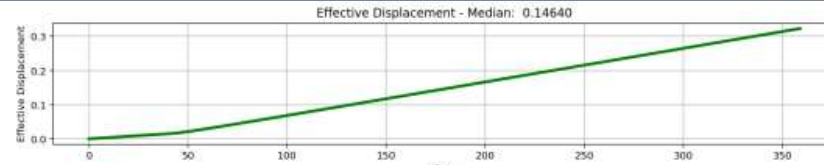
33(4) [CYCLIC_DISPLACEMENT] : Symmetric cyclic displacement protocol capturin

34pinching behavior, and energy dissipation degradation

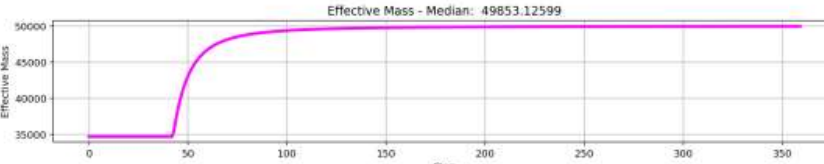
ISSIPATION_CAPACITY_INDEX\54_ELEVATOR_STRUCTURE_MDOF_8_ANA

32 %

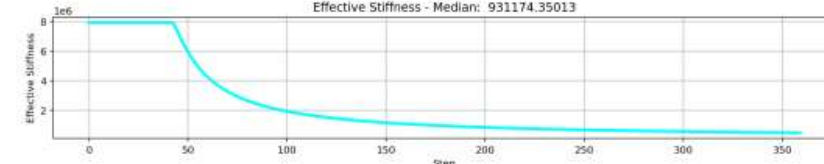
Effective Displacement - Median: 0.14640



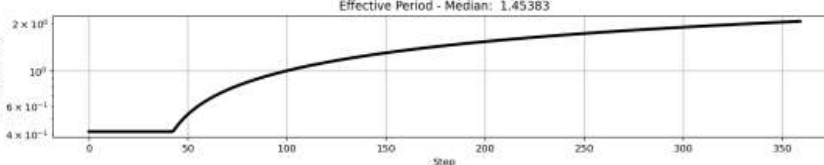
Effective Mass - Median: 49853.12599



Effective Stiffness - Median: 931174.35013



Effective Period - Median: 1.45383



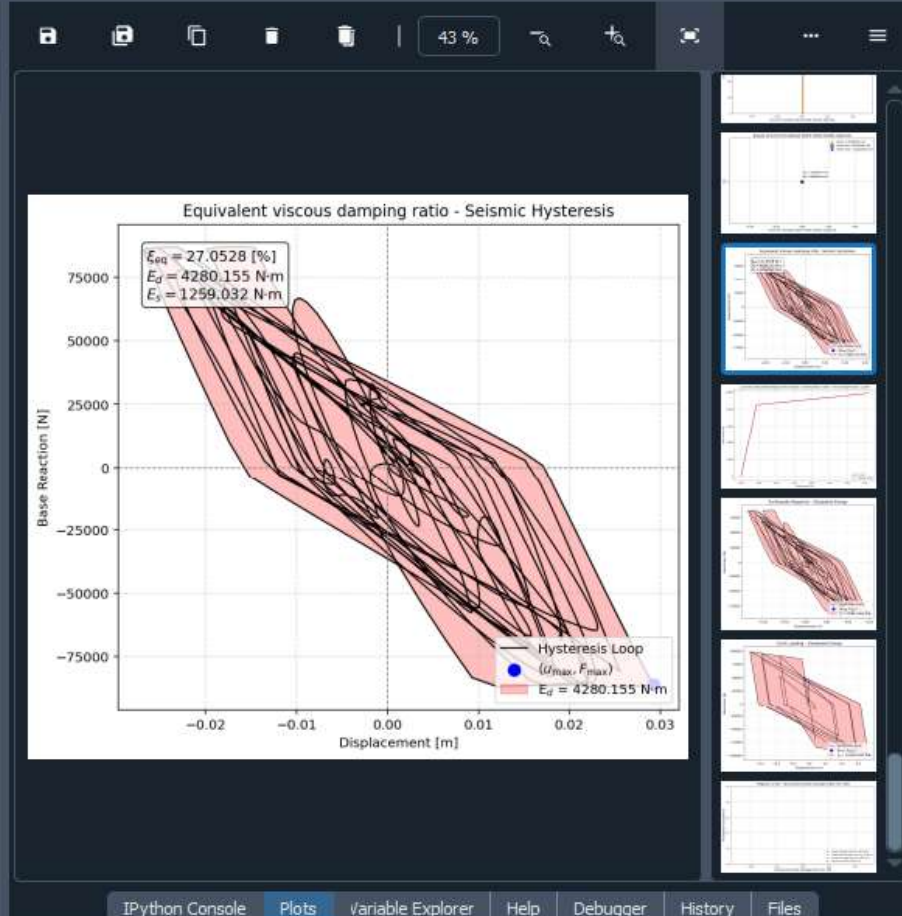
IPython ConsoleFilesHelpVariable ExplorerDebuggerPlotsHistory

TringaCody: anaconda3 (Python 3.12.7) ✓ LSP: Python Line 170 Col 4 LITE-S CRLE R/W Mem 48%

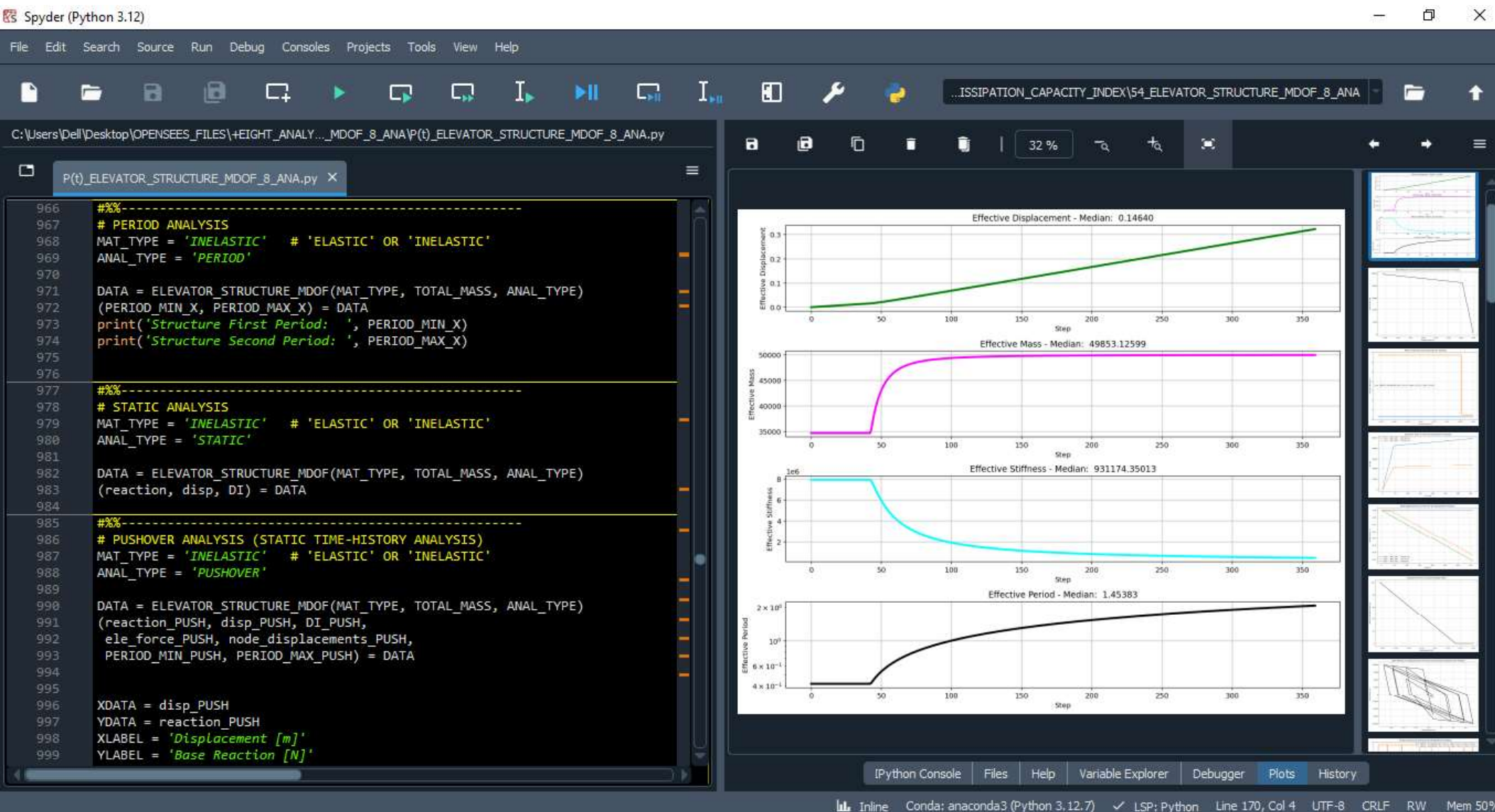

```

1  """ EQUIVALENT VISCOUS DAMPING RATIO
2  """
3  [1] The function EQUIVALENT_VISCOUS_DAMPING_RATIO_FUN computes the dissipated energy (E_d) us
4
5  [2] Both methods produce the same styled plot: the hysteresis loop in black, the enclosed area
6
7  [3] The bottom section generates a synthetic elliptical hysteresis loop (sinusoidal displaceme
8  THIS PYTHON SCRIPT WRITTEN BY SALAR DELAVAR GHASHGHAEI (QASHQAI)
9  """
10 def EQUIVALENT_VISCOUS_DAMPING_RATIO_FUN(displacement, base_shear, method, XLABEL, YLABEL, TI
11     import numpy as np
12     from scipy.spatial import ConvexHull
13     import matplotlib.pyplot as plt
14     if method == 1:
15
16         """
17         Compute the equivalent viscous damping ratio from a hysteresis loop.
18
19         The dissipated energy per cycle (E_d) is the area enclosed by the loop,
20         computed using the convex hull (Shoelace formula on the hull). The maximum
21         strain energy (E_s) is estimated as 0.5 * F_max * u_max, where u_max is the
22         maximum absolute displacement and F_max is the absolute base shear at that
23         same displacement.
24
25         Parameters
26         -----
27         displacement : array-like
28             Displacement values (assumed to form a closed hysteresis loop).
29         base_shear : array-like
30             Corresponding base shear values.
31         XLABEL : str
32             Label for the x-axis.
33         YLABEL : str
34             Label for the y-axis.
35         TI

```



STATIC ANALYSIS

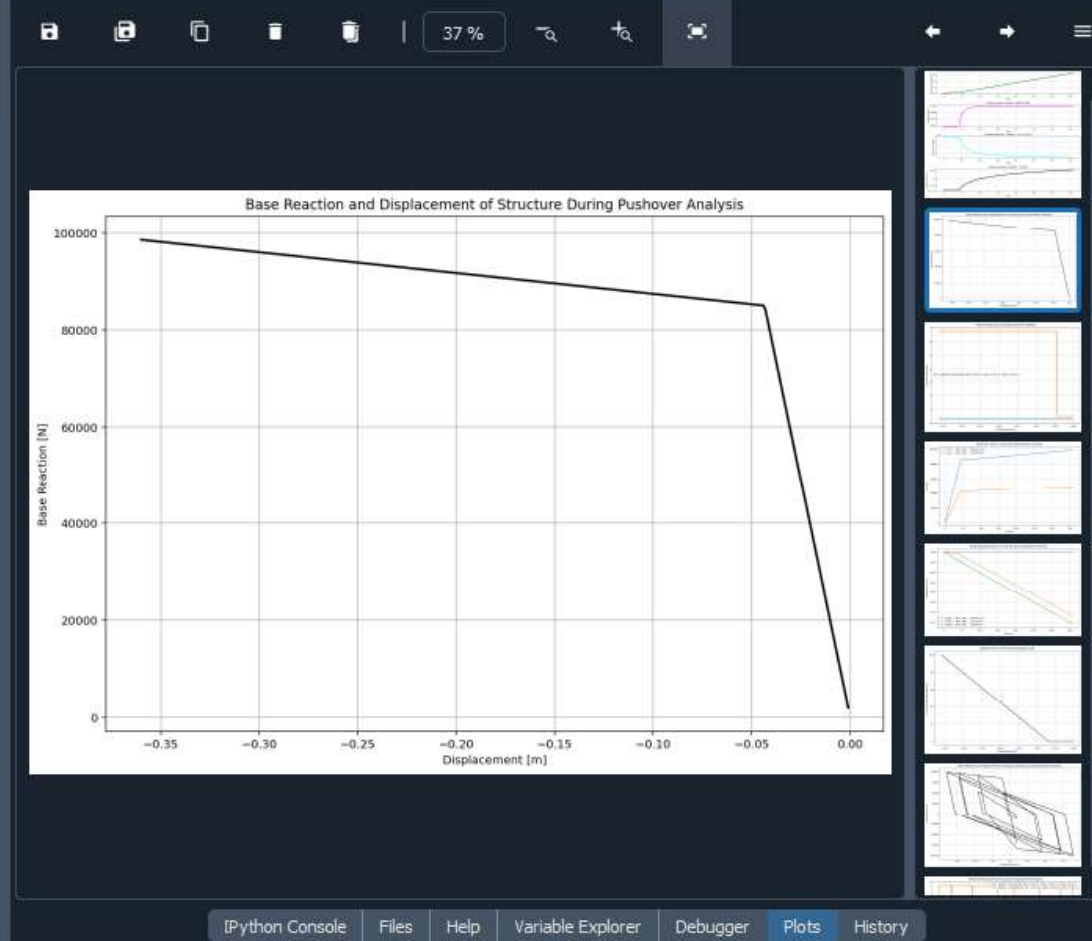


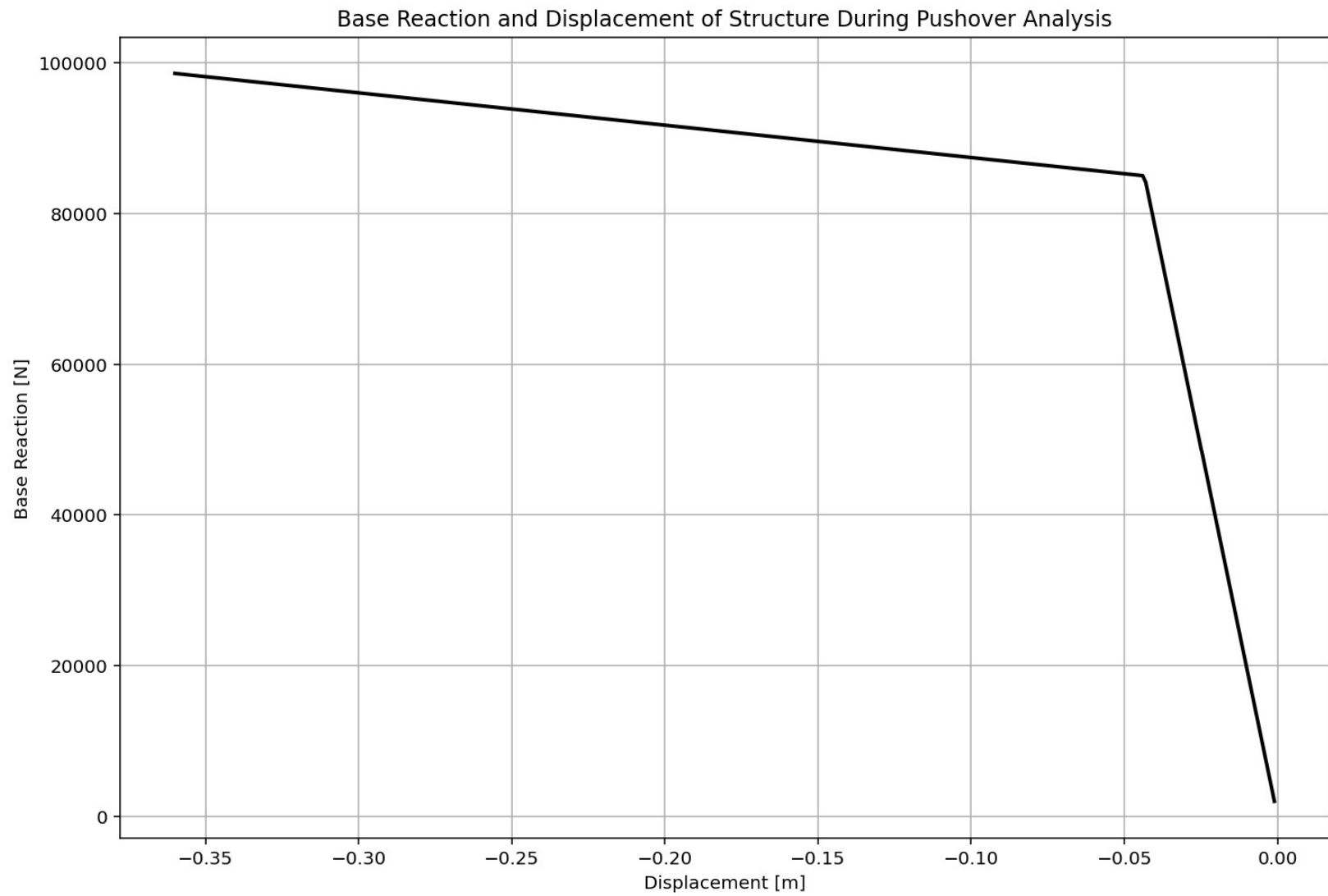
PUSHOVER ANALYSIS

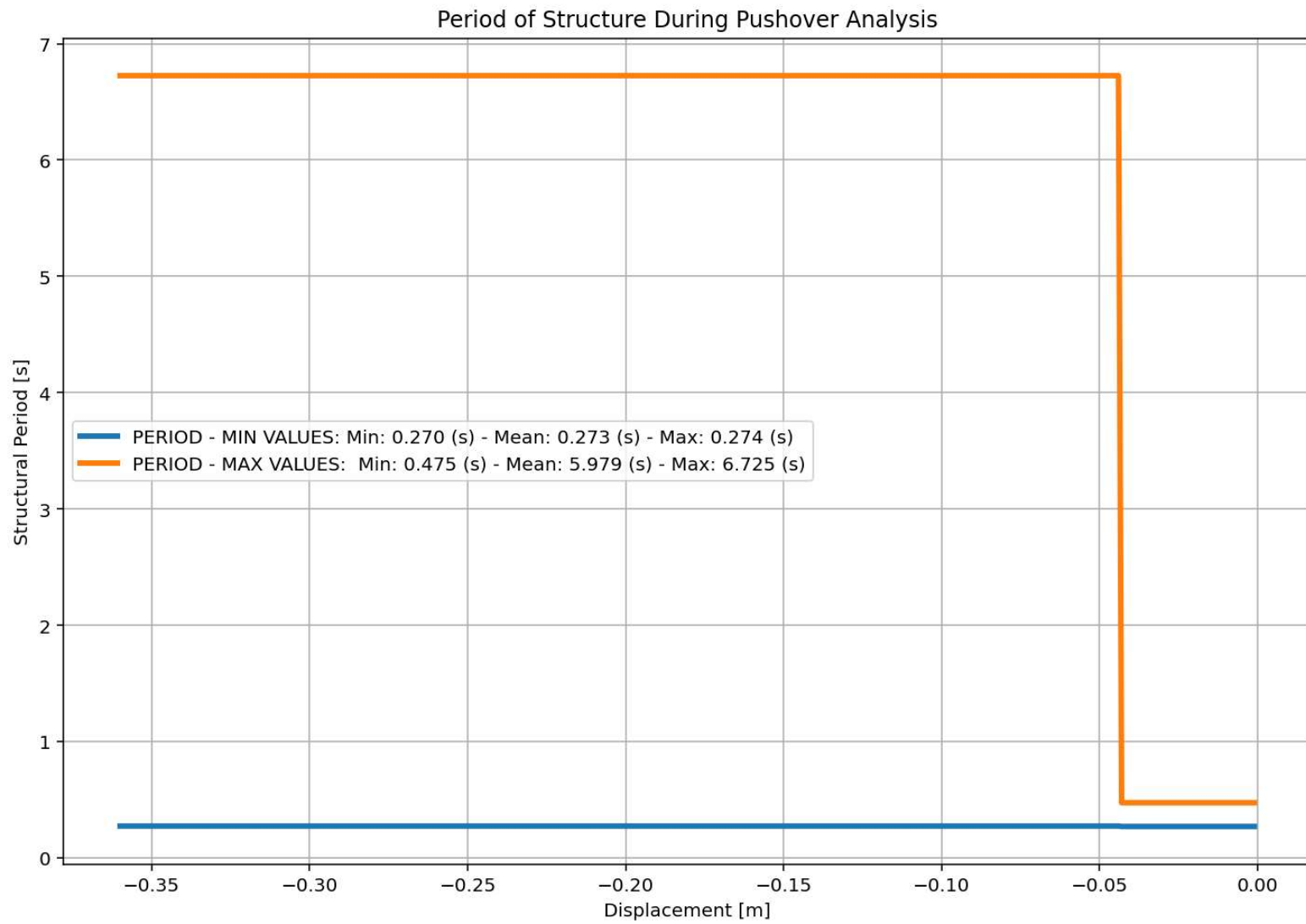
```

985  #####
986  # PUSHOVER ANALYSIS (STATIC TIME-HISTORY ANALYSIS)
987  MAT_TYPE = 'INELASTIC' # 'ELASTIC' OR 'INELASTIC'
988  ANAL_TYPE = 'PUSHOVER'
989
990  DATA = ELEVATOR_STRUCTURE_MDOF(MAT_TYPE, TOTAL_MASS, ANAL_TYPE)
991  (reaction_PUSH, disp_PUSH, DI_PUSH,
992   ele_force_PUSH, node_displacements_PUSH,
993   PERIOD_MIN_PUSH, PERIOD_MAX_PUSH) = DATA
994
995
996  XDATA = disp_PUSH
997  YDATA = reaction_PUSH
998  XLABEL = 'Displacement [m]'
999  YLABEL = 'Base Reaction [N]'
1000  TITLE = 'Base Reaction and Displacement of Structure During Pushover Analys
1001  COLOR = 'black'
1002  SEMILOGY = False
1003  PLOT(XDATA, YDATA, TITLE, XLABEL, YLABEL, COLOR, SEMILOGY)
1004
1005  DATA = S07.BILINEAR_CURVE(np.abs(disp_PUSH), np.abs(reaction_PUSH), SLOPE_NO
1006  (X_PUSH, Y_PUSH, Elastic_ST, Plastic_ST, Tangent_ST, Ductility_Rito, Over_S
1007
1008  # PLOT STRUCTURAL PERIOD DURING THE ANALYSIS
1009  plt.figure(0, figsize=(12, 8))
1010  plt.plot(disp_PUSH, PERIOD_MIN_PUSH, linewidth=3)
1011  plt.plot(disp_PUSH, PERIOD_MAX_PUSH, linewidth=3)
1012  plt.title('Period of Structure During Pushover Analysis')
1013  plt.ylabel('Structural Period [s]')
1014  plt.xlabel('Displacement [m]')
1015  #plt.semilogy()
1016  plt.grid()
1017  plt.legend([f'PERIOD - MIN VALUES: Min: {np.min(PERIOD_MIN_PUSH):.3f} (s) -
1018             f'PERIOD - MAX VALUES: Min: {np.min(PERIOD_MAX_PUSH):.3f} (s)

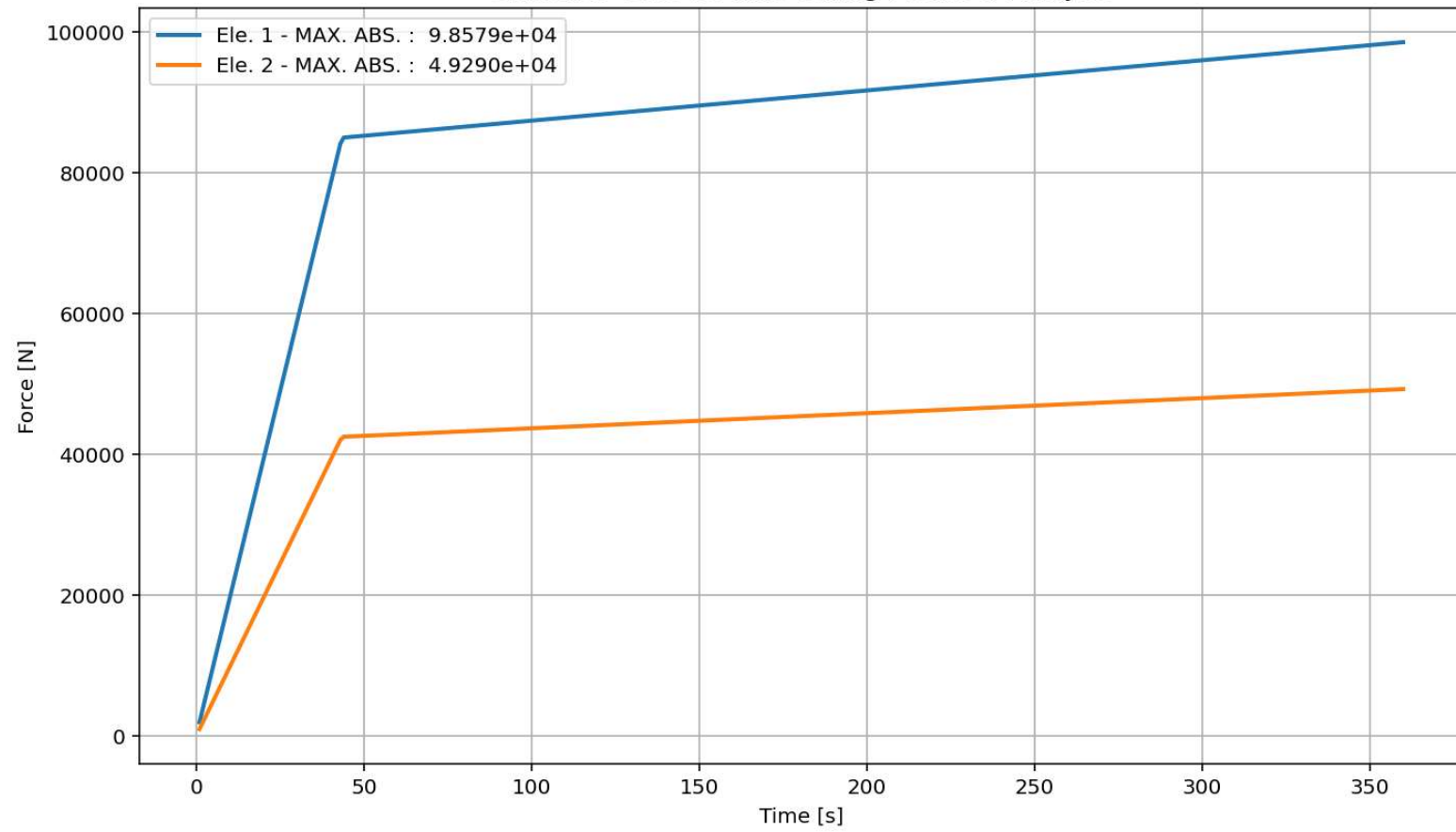
```



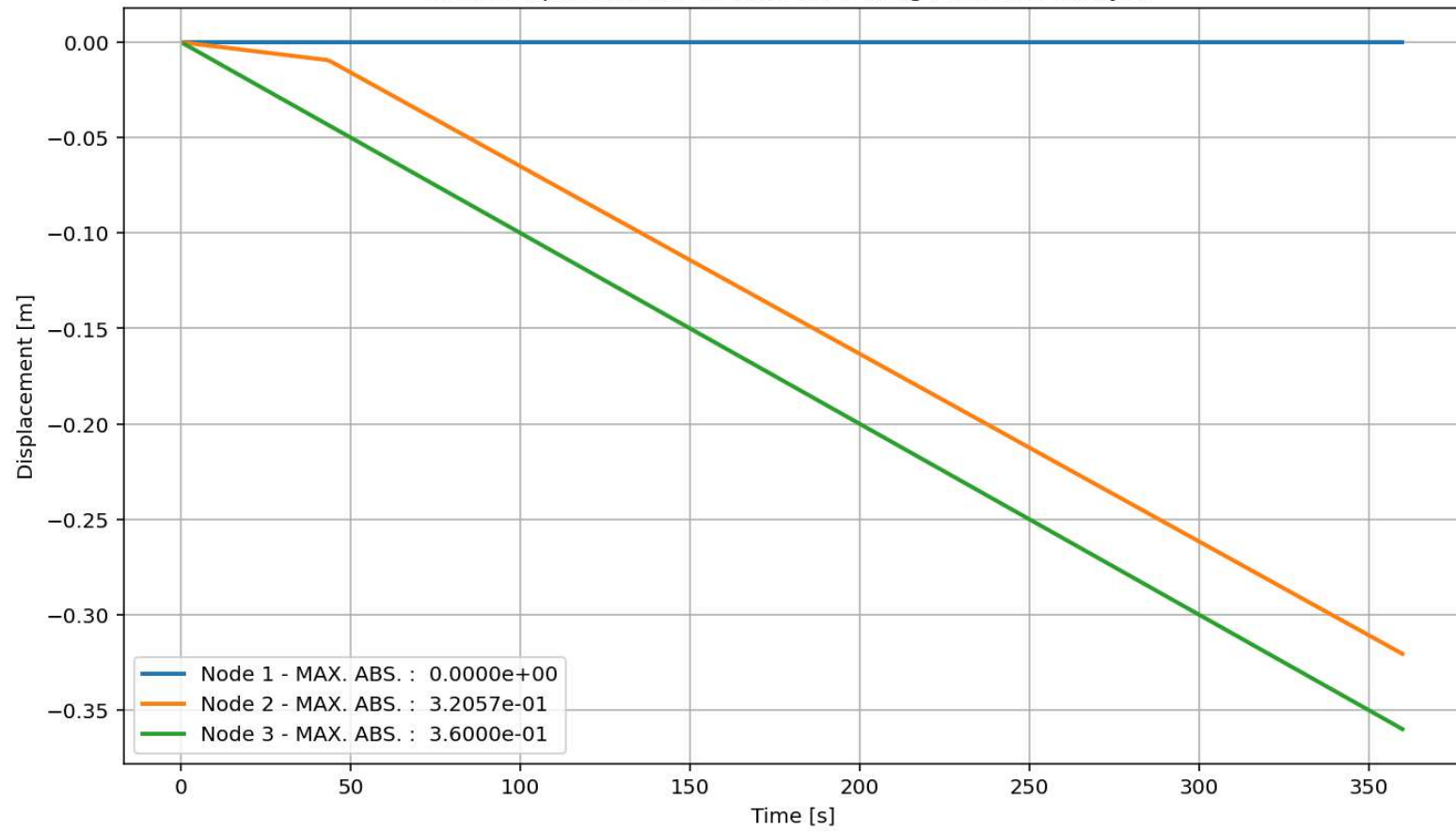


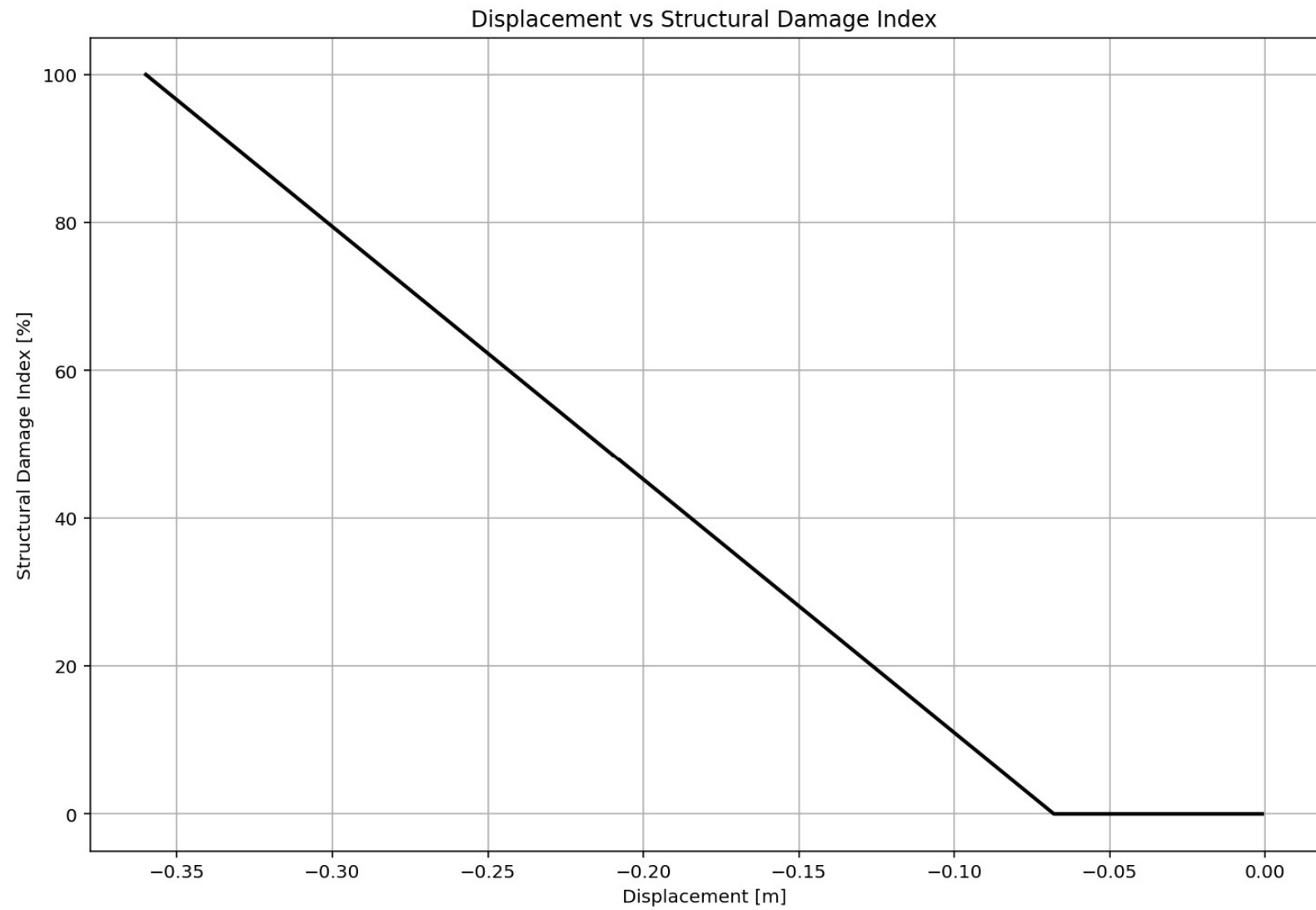


elements force vs Time During Pushover Analysis



Node Displacements vs Time for During Pushover Analysis

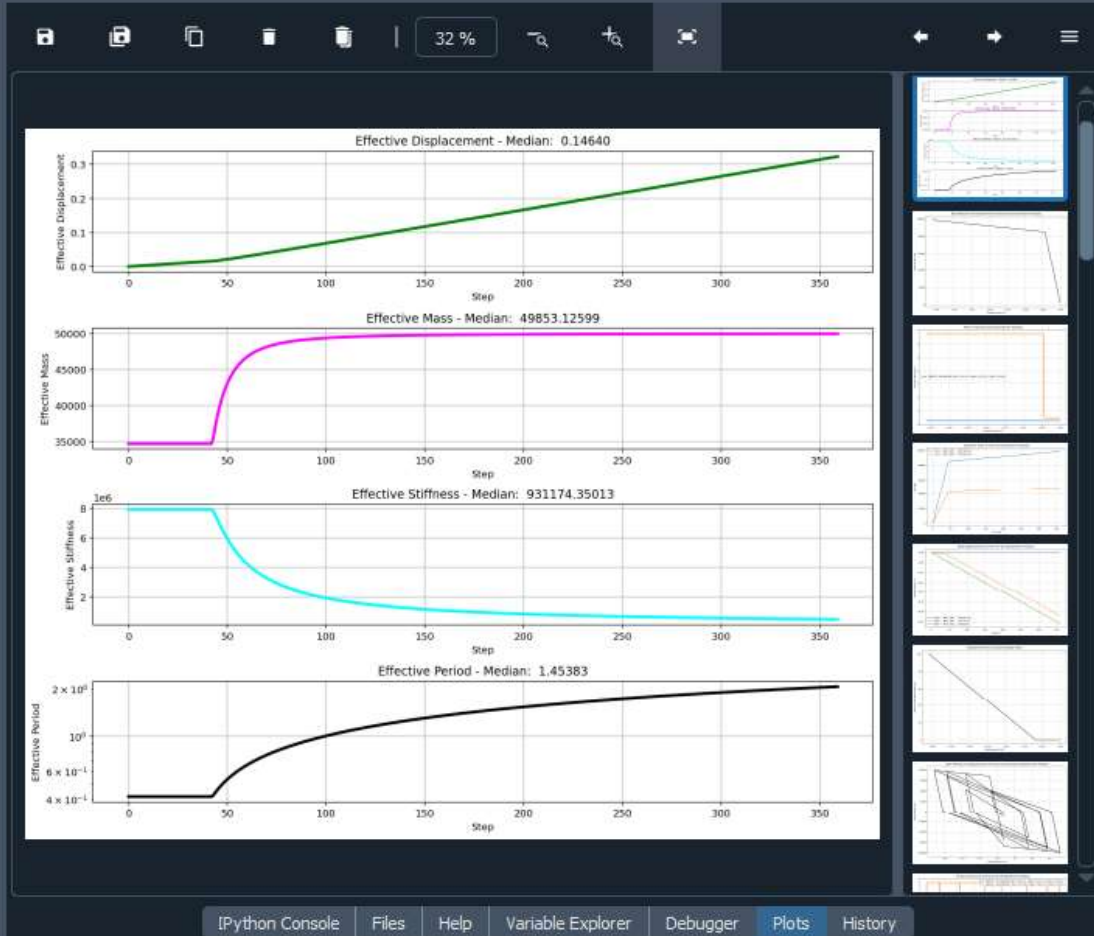


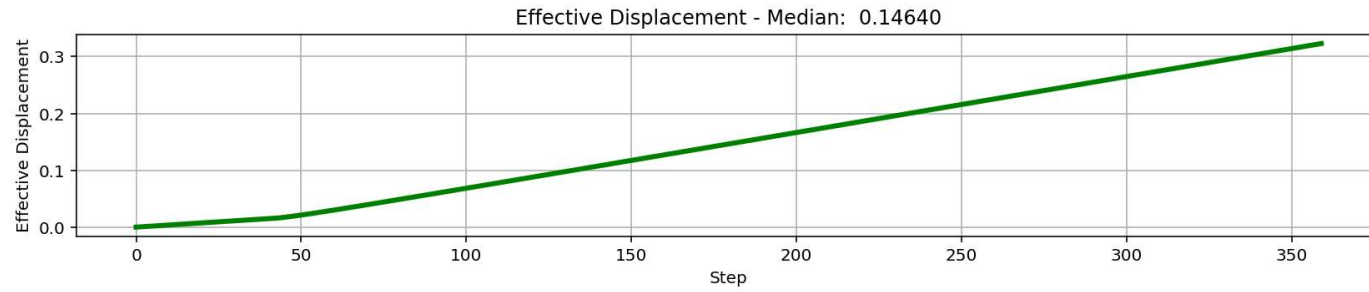


```

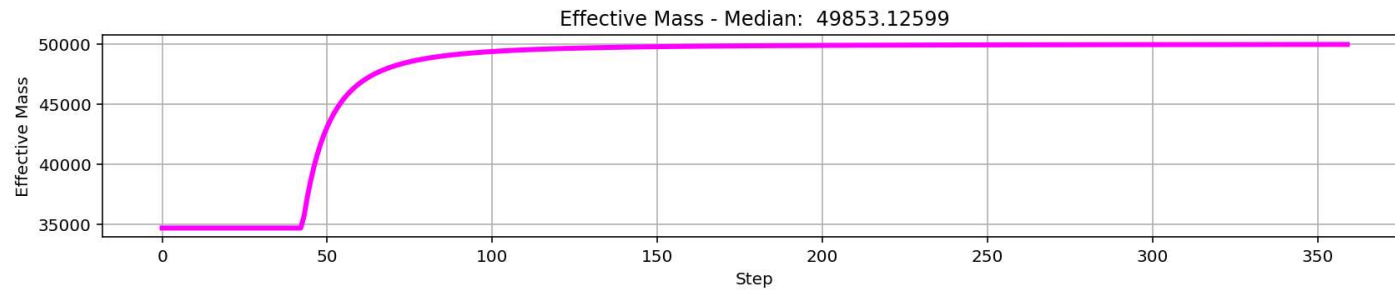
1  """ EQUIVALENT SDOF SYSTEM DERIVATION VIA DISPLACEMENT-BASED SEISMIC DESIGN P
2  # Change MDOF to SDOF System with Displacement Based Design Concept
3  # THIS PROGRAM WRITTEN BY SALAR DELAVAR GHASHGHAEE (QASHQAI)
4  """
5  This script implements a displacement-based pushover transformation,
6  converting a multi-degree-of-freedom (MDOF) system into an equivalent
7  single-degree-of-freedom (SDOF) system for seismic assessment.
8  It calculates effective modal properties-displacement, mass, and
9  stiffness-by weighting element forces and nodal displacements according
10 to a presumed deformed shape.
11 The derived effective period provides a simplified dynamic characteristic
12 for performance-based engineering. The visualizations effectively track the
13 evolution of these equivalent parameters throughout the nonlinear analysis s
14 """
15
16 -----
17 displacement_X_1 = np.array(list(node_displacements.values())[1]) # DOF 02
18 displacement_X_2 = np.array(list(node_displacements.values())[2]) # DOF 03
19
20 ele_force_01 = np.array(list(ele_force.values())[0]) # ELEMENT 01
21 ele_force_02 = np.array(list(ele_force.values())[1]) # ELEMENT 02
22
23 STIFF_X_1 = np.abs(ele_force_01 / displacement_X_1)
24 STIFF_X_2 = np.abs(ele_force_02 / displacement_X_2)
25
26 MX2 = (MASS[0] * np.square(displacement_X_1) +
27        MASS[1] * np.square(displacement_X_2))
28
29 MX = (MASS[0] * np.array(displacement_X_1) +
30        MASS[1] * np.array(displacement_X_2))
31 EFFECTIVE_DISP_X = MX2 / np.abs(MX)
32
33 EFFECTIVE_MASS_X = np.abs(MX) / EFFECTIVE_DISP_X
34

```

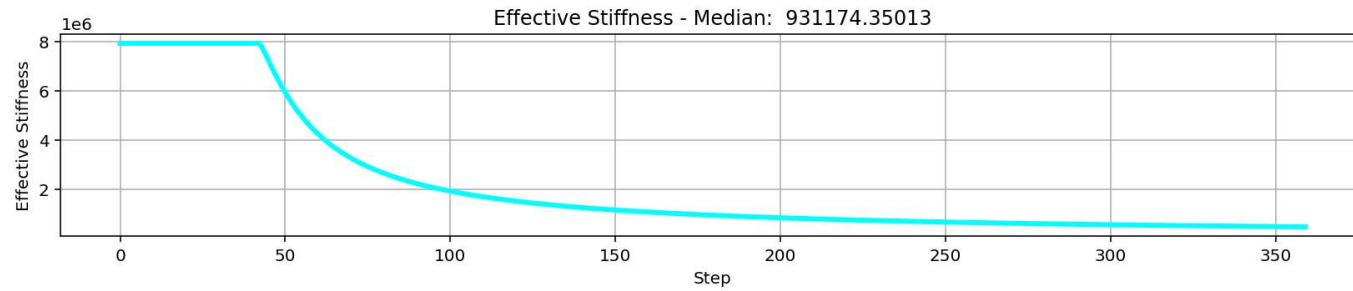




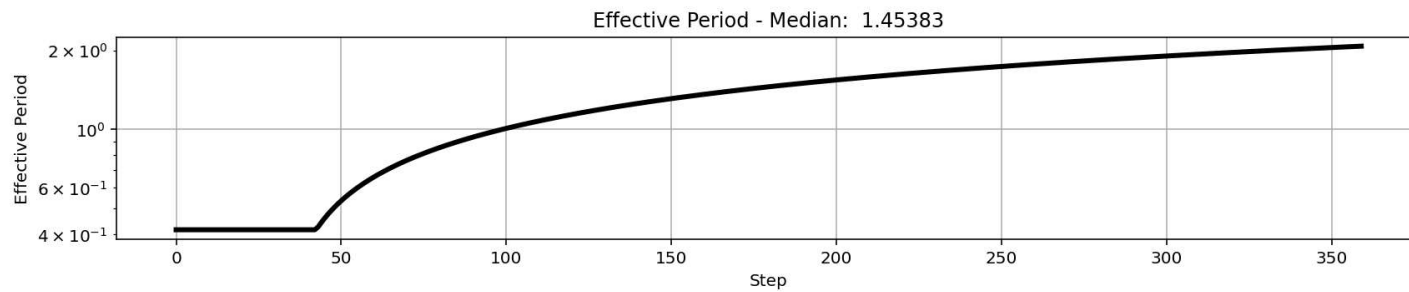
$$\Delta_d = \frac{\sum_{i=1}^n m_i \Delta_i^2}{\sum_{i=1}^n m_i \Delta_i}$$



$$m_{eff} = \frac{\sum_{i=1}^n m_i \Delta_i}{\Delta_d}$$



$$k_{eff} = \frac{\sum_{i=1}^n k_i \Delta_i}{\Delta_d}$$



$$T_{eff} = \frac{2\pi}{\sqrt{\frac{k_{eff}}{m_{eff}}}}$$

CYCLIC DISPLACEMENT ANALYSIS

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY..._MDOF_8_ANA\p(t)_ELEVATOR_STRUCTURE_MDOF_8_ANA.py

P(t)_ELEVATOR_STRUCTURE_MDOF_8_ANA.py COMPUTE_EFFECTIVE_PROPERTIES_FUN.py

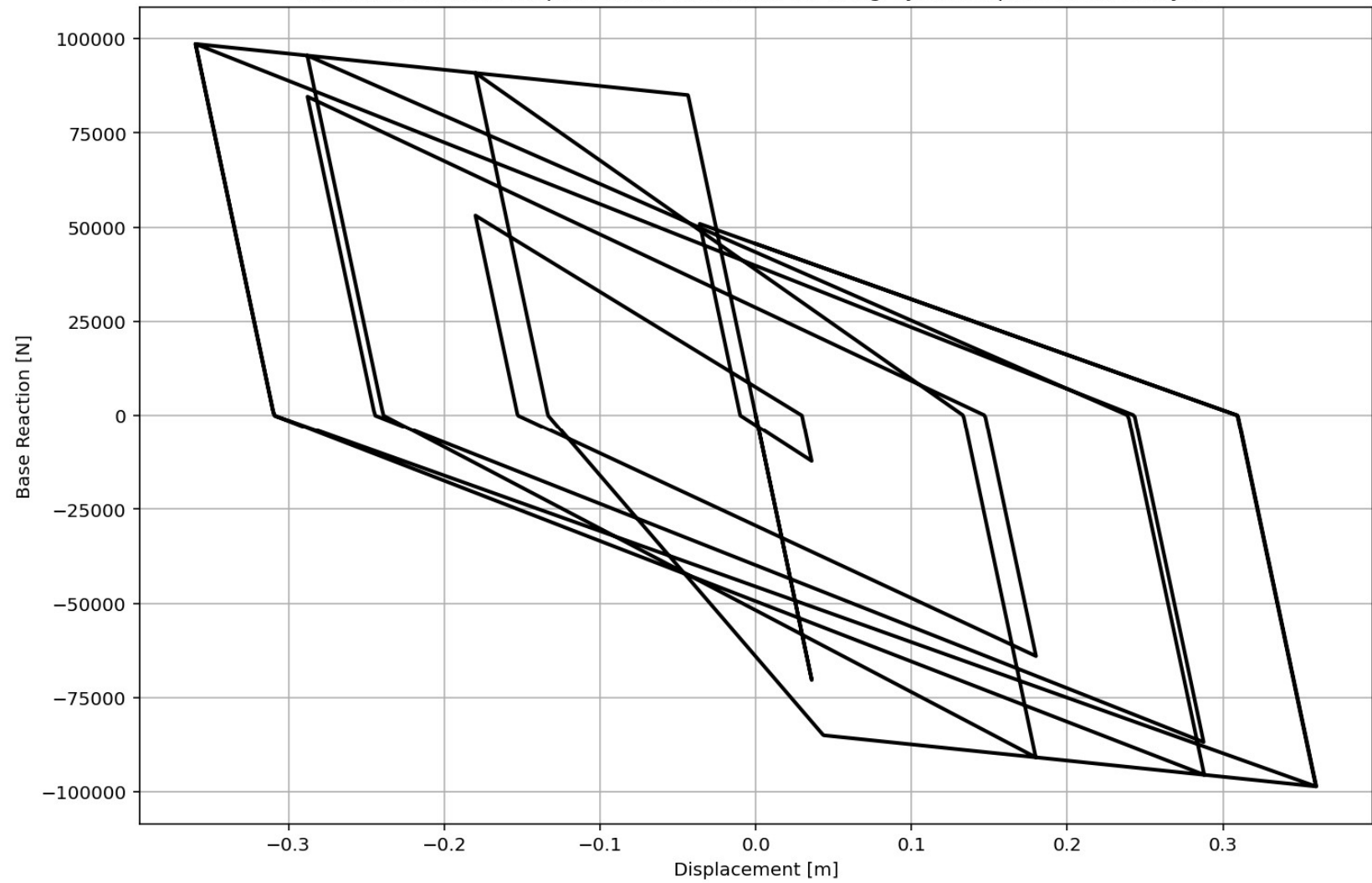
```
366 if ANAL_TYPE == 'CYCLIC_DISPLACEMENT': # STATIC TIME-HISTORY ANALYSIS
367     IDctrlDOF = 1 # 1: Horizontal Displacement - 2: Vertical Dispalce
368     DMAX = -DSU # [m] Max. Displacement
369     n_points = 10000
370     # 4. CYCLIC DISPALCEMENT PROTOCOL
371     # Key strain points (same logic as your protocol)
372     key_disp = np.array([
373         0.0,
374         0.1*DMAX, -0.1*DMAX,
375         0.5*DMAX, -0.5*DMAX,
376         0.8*DMAX, -0.8*DMAX,
377         DMAX, -DMAX,
378         0.1*DMAX, -0.1*DMAX,
379         0.5*DMAX, -0.5*DMAX,
380         0.8*DMAX, -0.8*DMAX,
381         DMAX, -DMAX,
382         0.0
383     ])
384
385     # Generate 1000-point displacement protocol
386     disp_protocol = np.interp(
387         np.linspace(0, len(key_disp) - 1, n_points),
388         np.arange(len(key_disp)),
389         key_disp
390     )
391
392     ops.analysis('Static')
393     STEP = 0.0
394     for target_disp in disp_protocol:
395         current_disp = ops.nodeDisp(center_node, 1) # DISPALCEMENT APPL
396         dU = target_disp - current_disp
397         ops.integrator('DisplacementControl', center_node, IDctrlDOF, d
398         OK = ops.analyze(1)
399         S02.ANALYSIS(OK, 1, MAX_TOLERANCE, MAX_ITERATIONS)
```

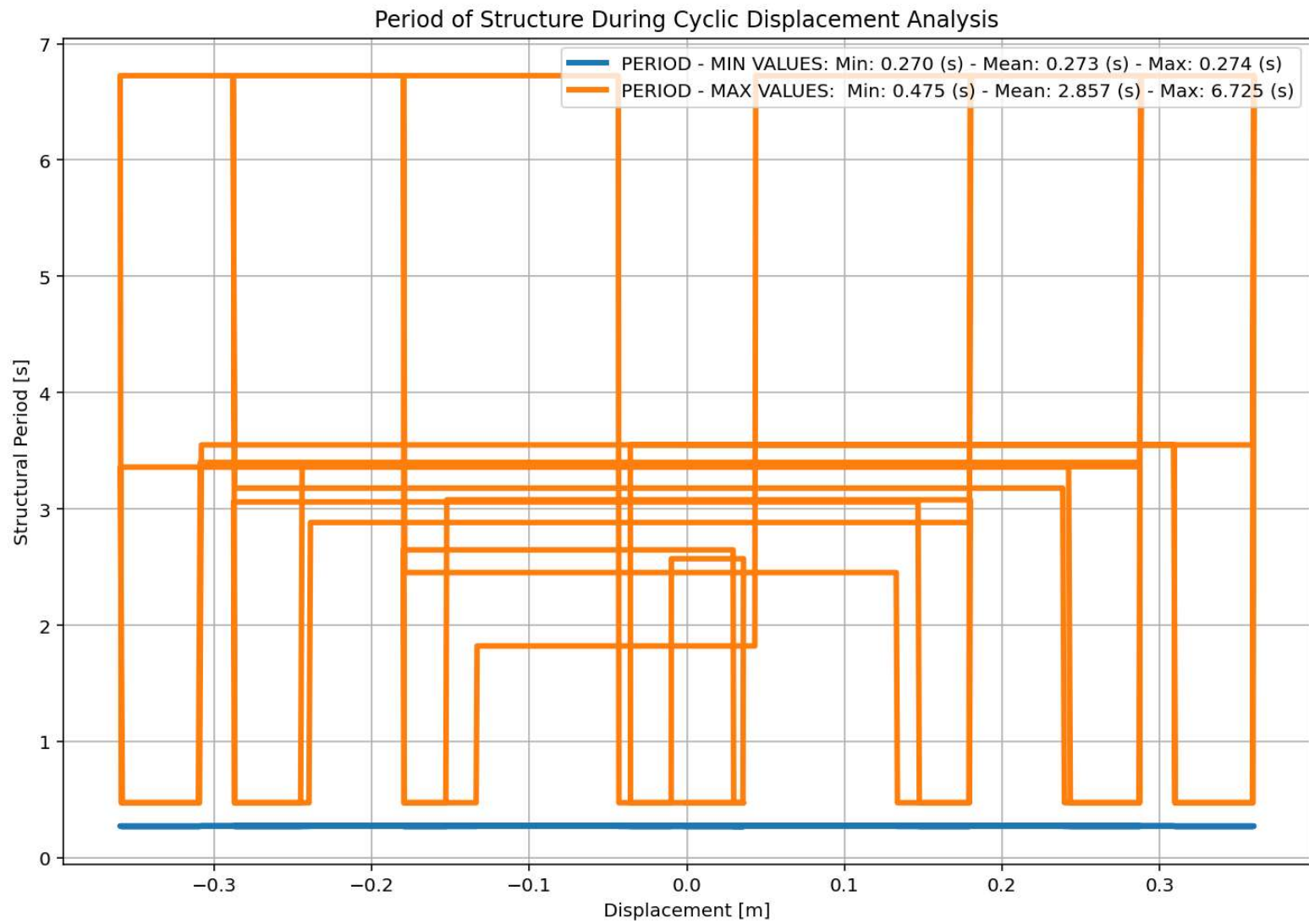
Base Reaction and Displacement of Structure During Cyclic-Displacement Analysis

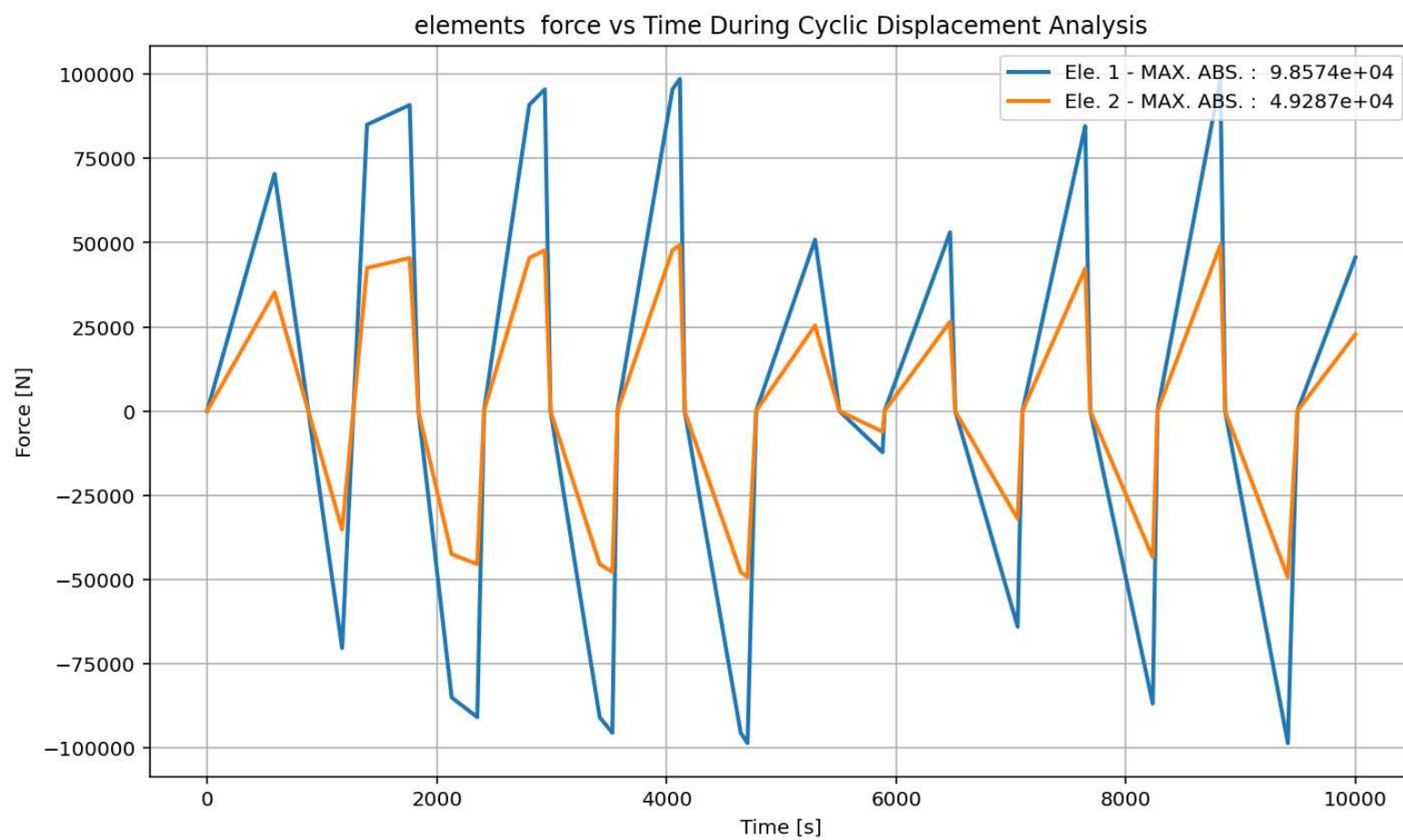
Python Console Files Help Variable Explorer Debugger Plots History

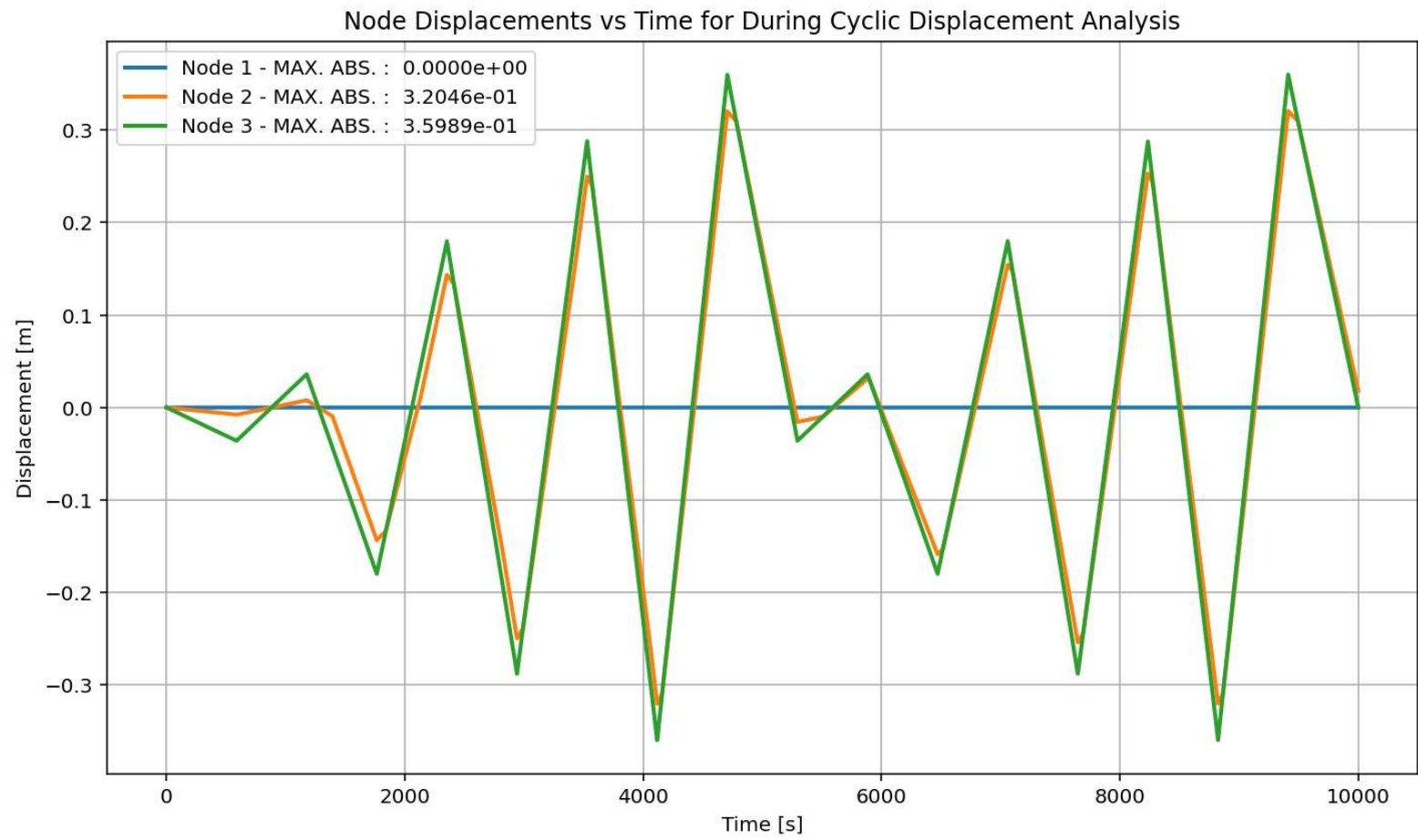
Inline Conda: anaconda3 (Python 3.12.7) LSP: Python Line 362, Col 44 UTF-8 CRLF RW Mem 51%

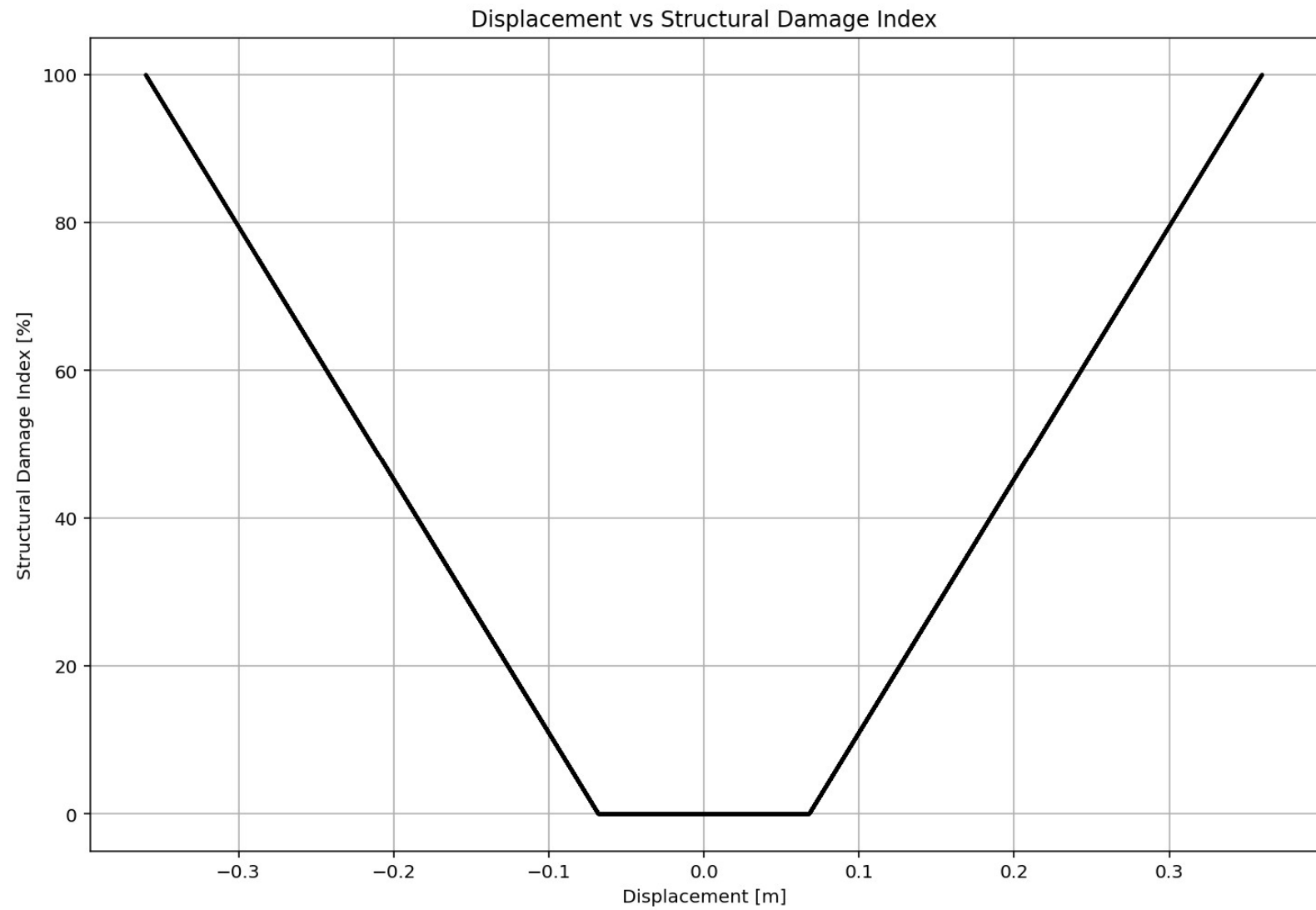
Base Reaction and Displacement of Structure During Cyclic-Displacement Analysis



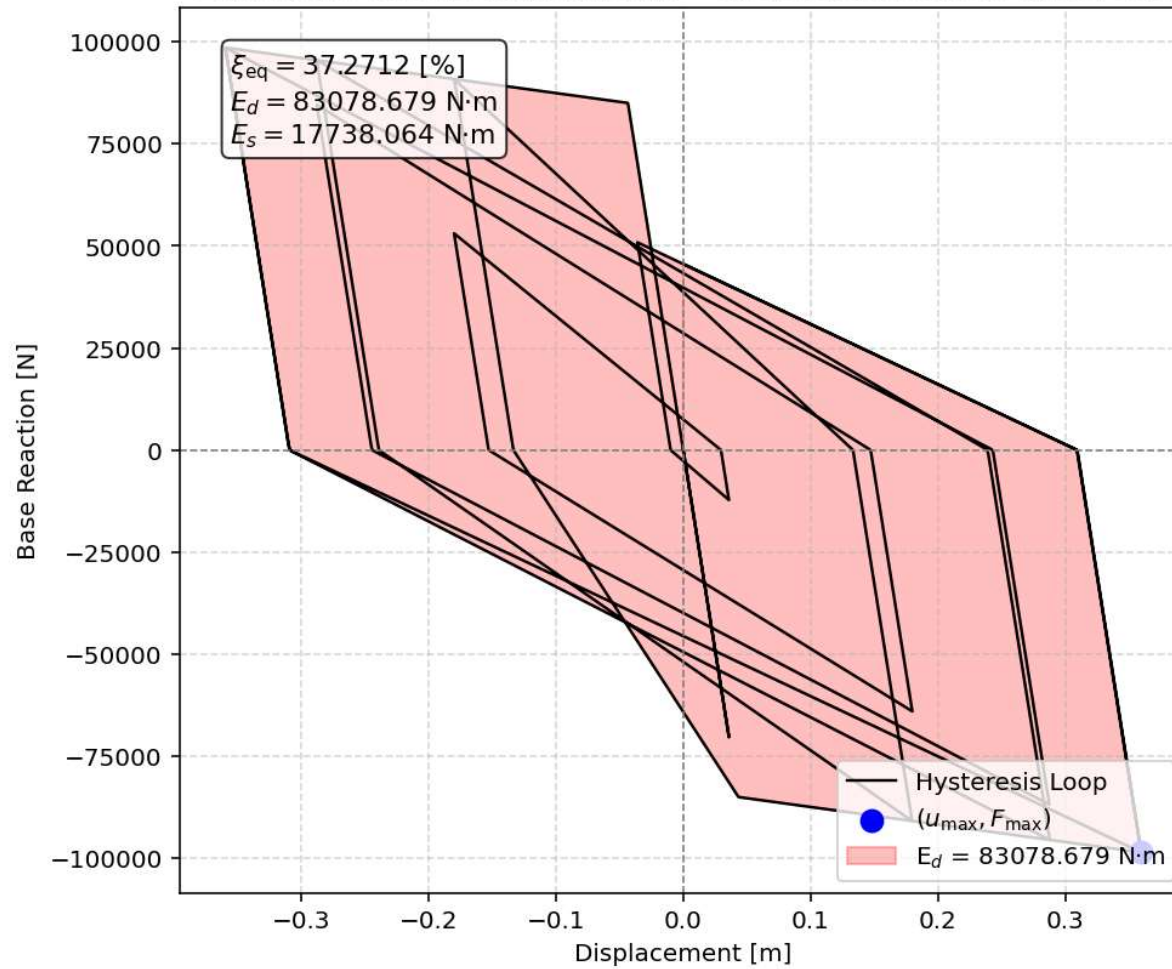








Equivalent viscous damping ratio - Cyclic Displacement Hysteresis



STATIC TIME-HISTORY WITH EXTERNAL TIME-DEPENDENT LOADING ANALYSIS

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\ DELL\Desktop\OPENSEES_FILES\+EIGHT_ANALY..._MDOF_8_ANA\P(t)_ELEVATOR_STRUCTURE_MDOF_8_ANA.py

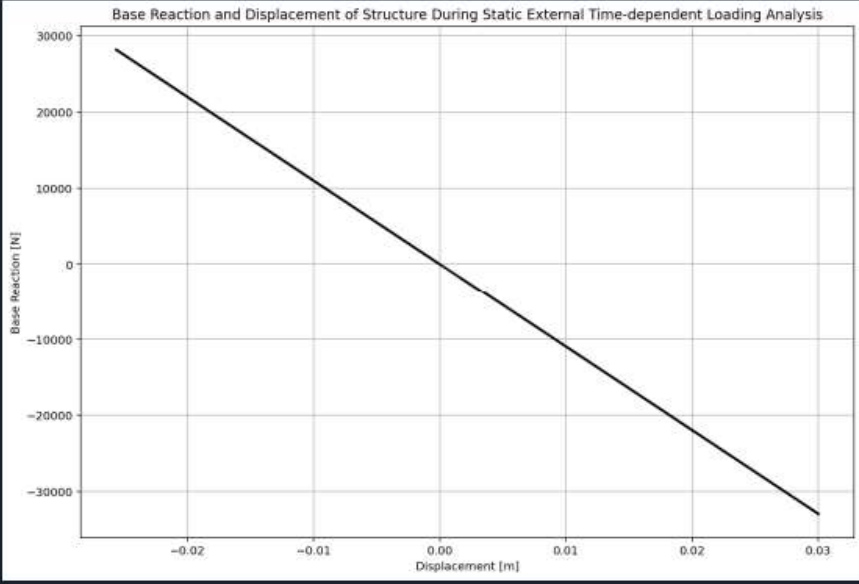
P(t)_ELEVATOR_STRUCTURE_MDOF_8_ANA.py x COMPUTE_EFFECTIVE_PROPERTIES_FUN.py x

```
431 if ANAL_TYPE == 'STATIC EXTERNAL TIME-DEPENDENT LOADING': # STATIC TIME
432     %% DEFINE EXTERNAL TIME-DEPENDENT LOADING PROPERTIES
433     # IN HERE ANALYSIS TIME AND DURATION ARE LOAD STEPS
434     duration = 20.0 # [s] Analysis duration
435     dt = 0.01 # [s] Time step
436     DT = dt # [s] Time step
437     DT_time = 5.0 # [s] Total external Load Analysis Dura
438     W_kg = MASS[1]*2500 # [kg] Total weight (cabin + capacity +
439     v_mps = 2.0 # [m/s] Impact velocity
440     S_m = 0.45 # [m] Buffer stroke
441     g = 9.81 # [m/s^2] standard acceleration due to g
442     W = W_kg * g # Weight in Newtons
443     # Maximum impact force using buffer formula
444     R_max = W * (1 + (v_mps**2) / (2 * g * S_m))
445     force_amplitude = R_max # [N] Amplitude Force
446     omega_DT = 5.0715 # [rad/s] Natural angular frequency
447     time_steps = int(duration/dt)
448
449     # Check function
450     def CHECK_FUN(DT_time ,duration):
451         if DT_time > duration:
452             print('\n\nAnalysis Duration Must be greater than External
453                 exit()
454             return -1
455
456     CHECK_FUN(DT_time ,duration)
457     def EXTERNAL_TIME_DEPENDENT(force_amplitude, omega_DT, DT, DT_time)
458         import numpy as np
459         import matplotlib.pyplot as plt
460         # External Load Durations
461         num_steps = int(DT_time / DT)
462         load_time = np.linspace(0, DT_time, num_steps)
463         target_frequency = 1.0 * omega_DT # Target excitation frequenc
464         DT_load = force_amplitude * np.sin(target_frequency * load_time
```

ISSIPATION_CAPACITY_INDEX\54_ELEVATOR_STRUCTURE_MDOF_8_ANA

37 %

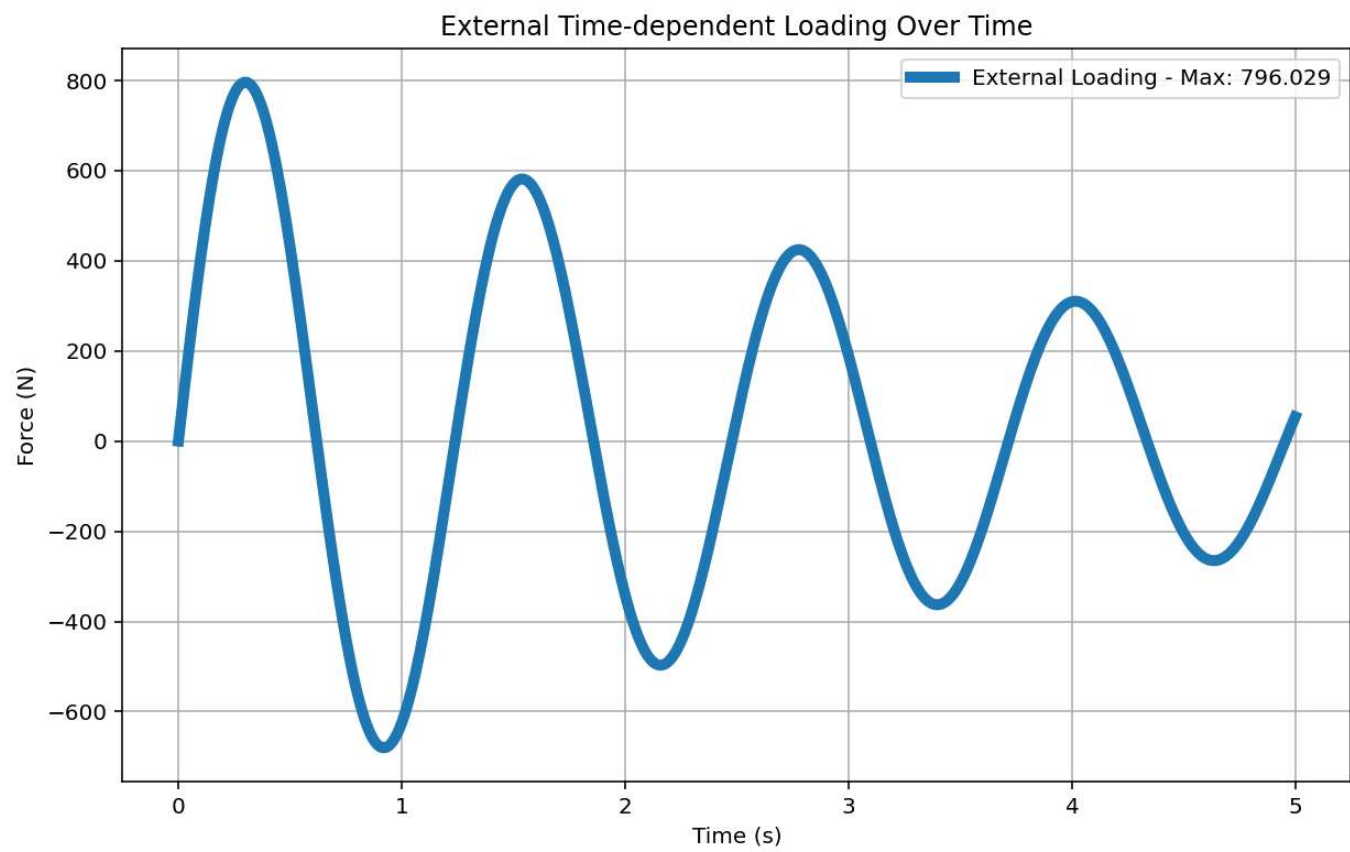
Base Reaction and Displacement of Structure During Static External Time-dependent Loading Analysis



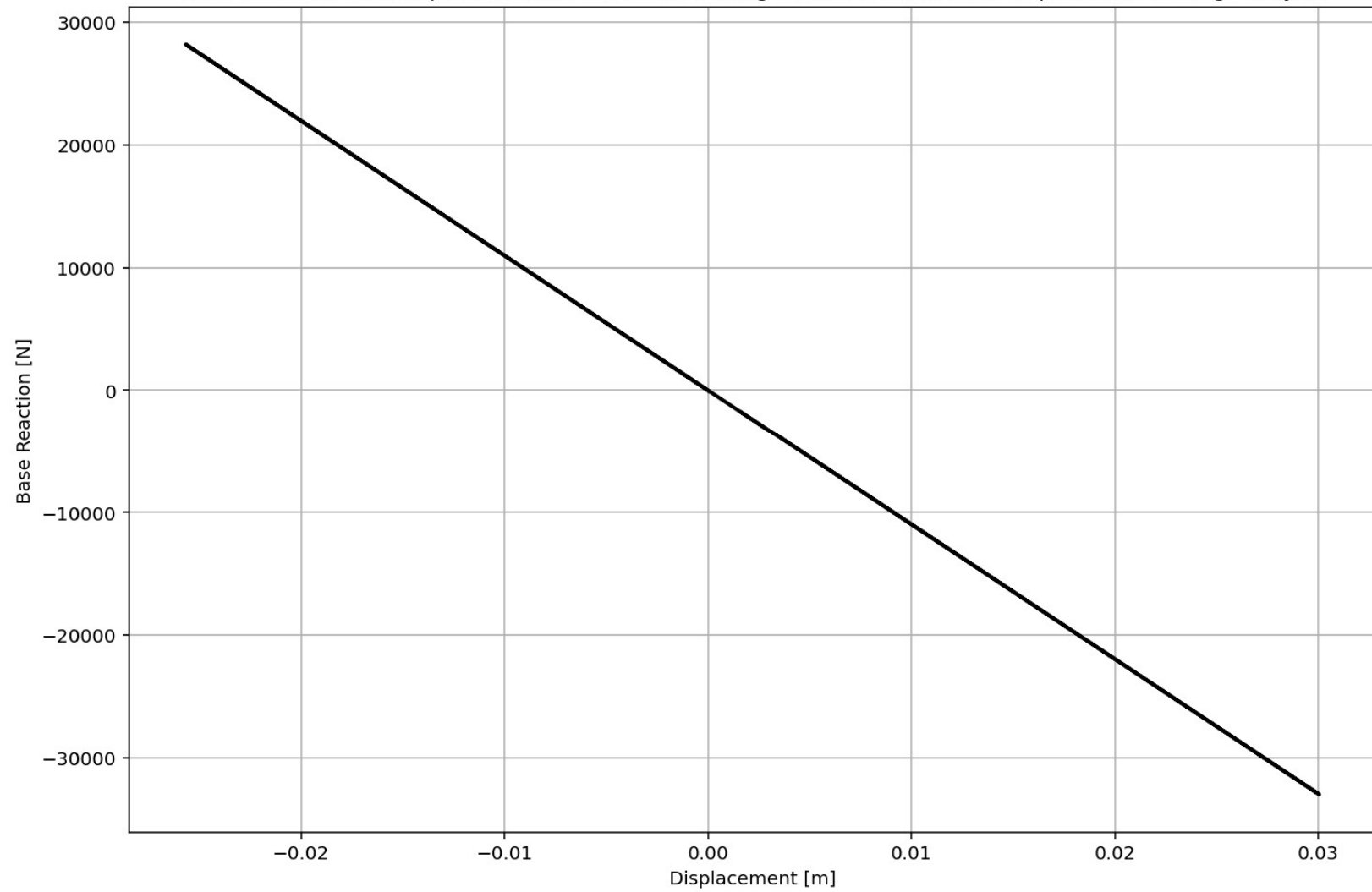
IPython Console Files Help Variable Explorer Debugger Plots History

Browse a working directory

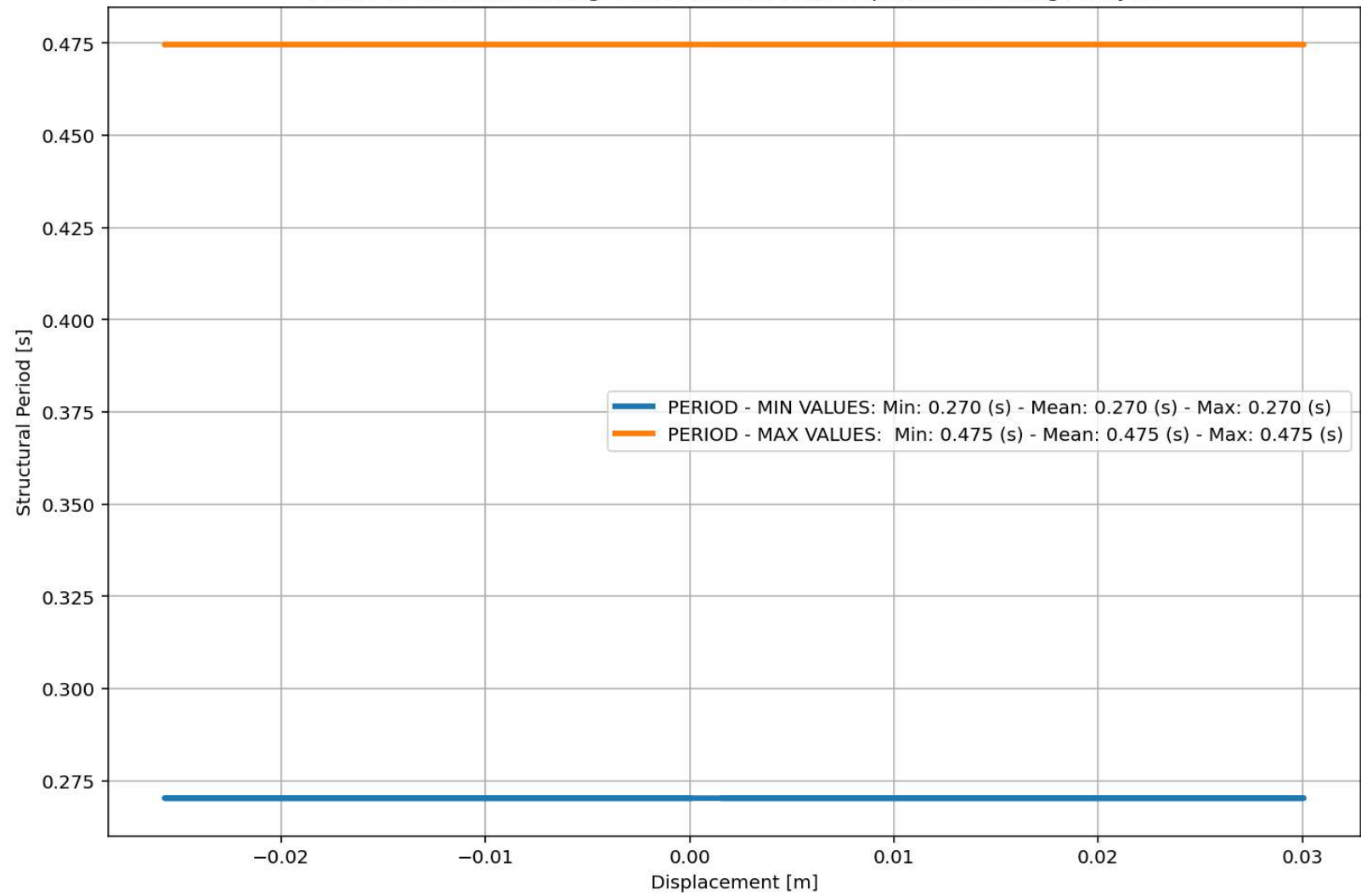
Inline Conda: anaconda3 (Python 3.12.7) LSP: Python Line 362, Col 44 UTF-8 CRLF RW Mem 51

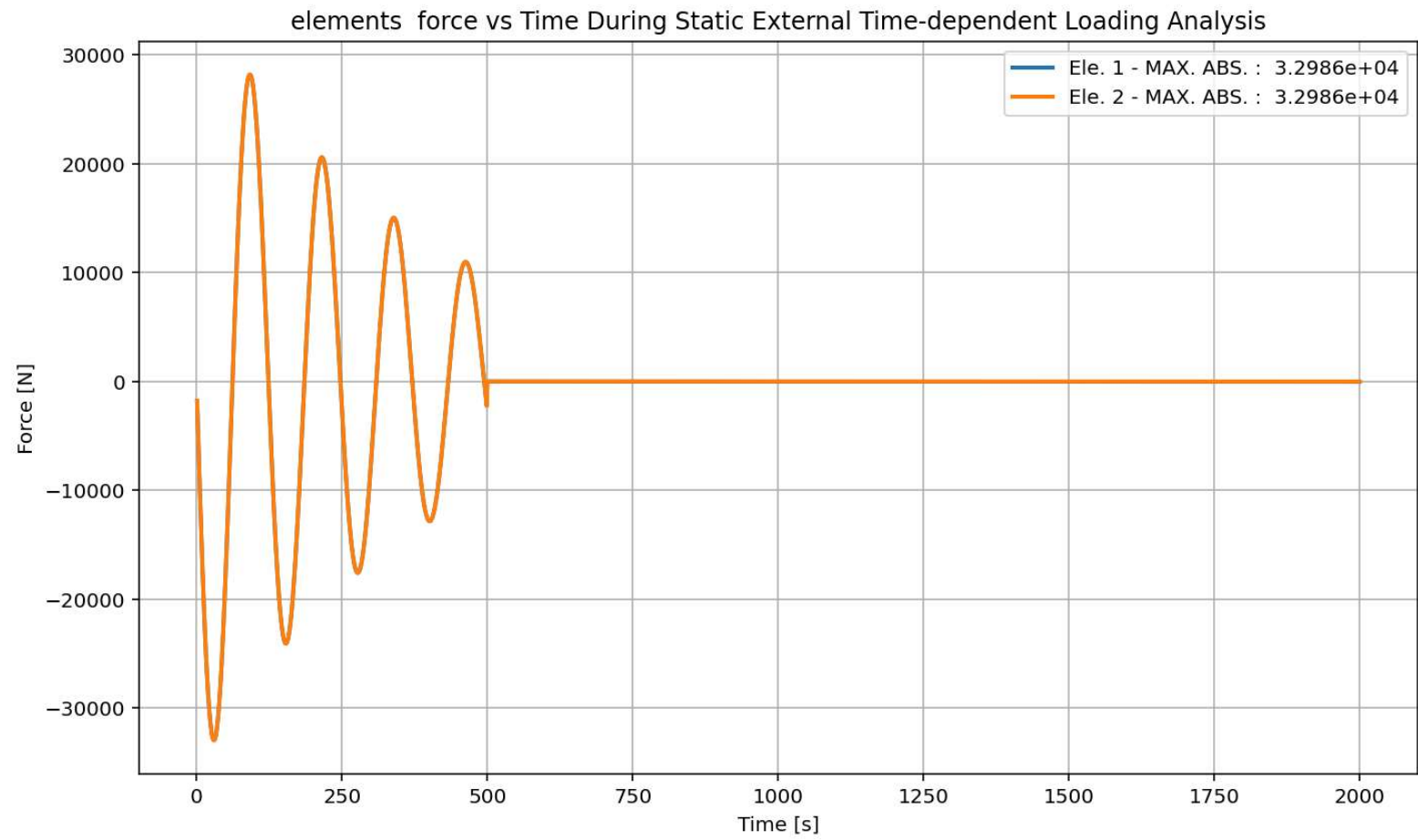


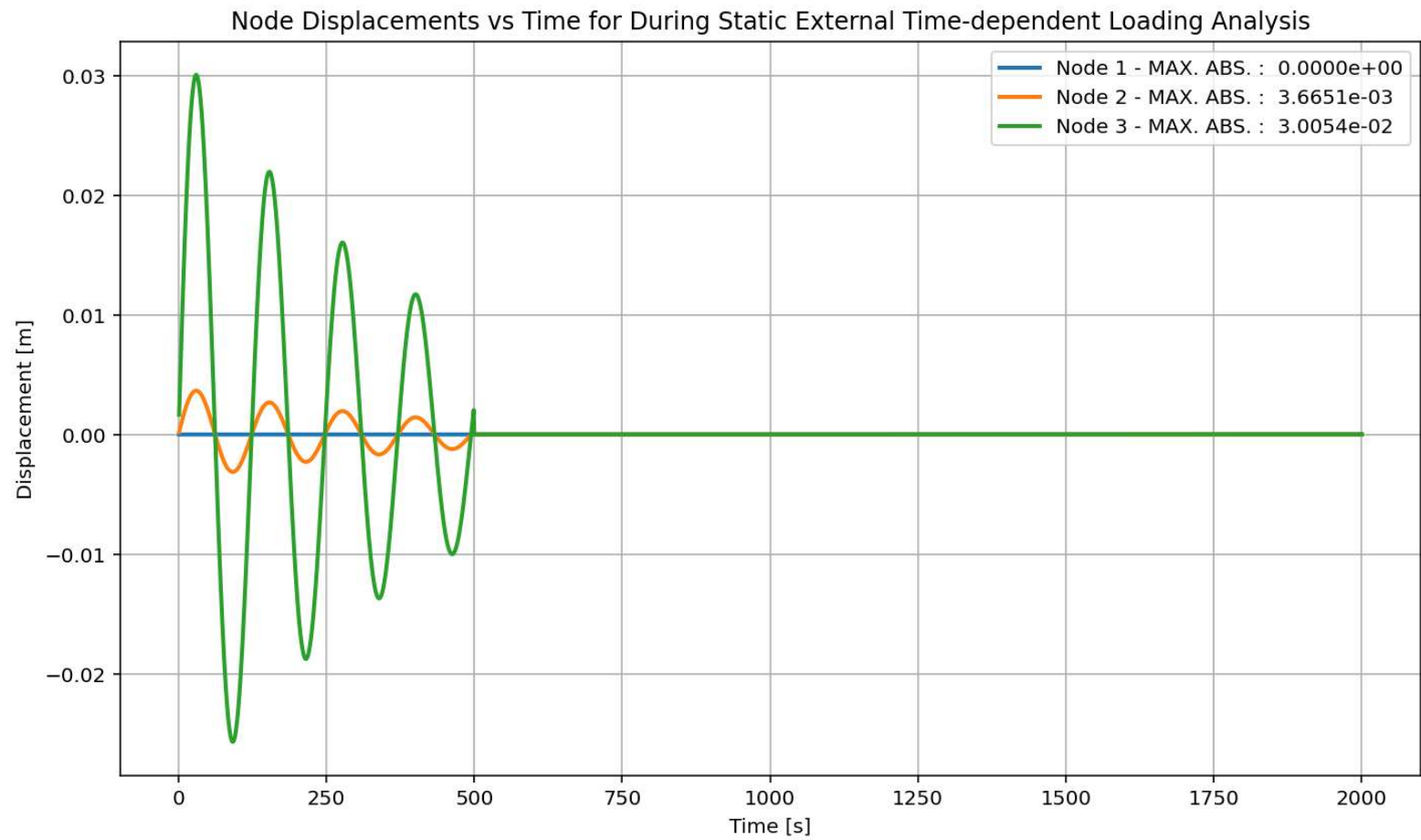
Base Reaction and Displacement of Structure During Static External Time-dependent Loading Analysis

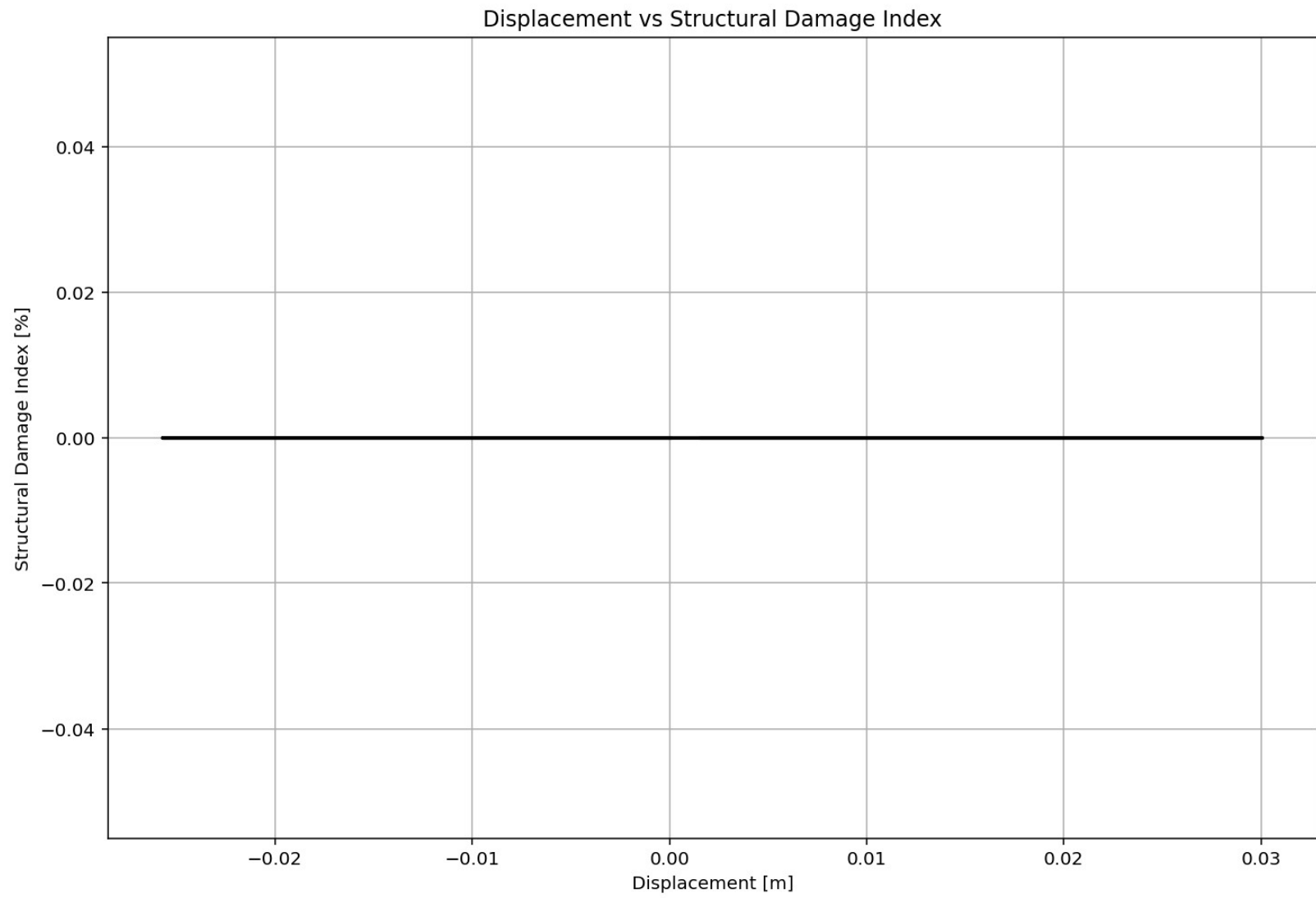


Period of Structure During Static External Time-dependent Loading Analysis

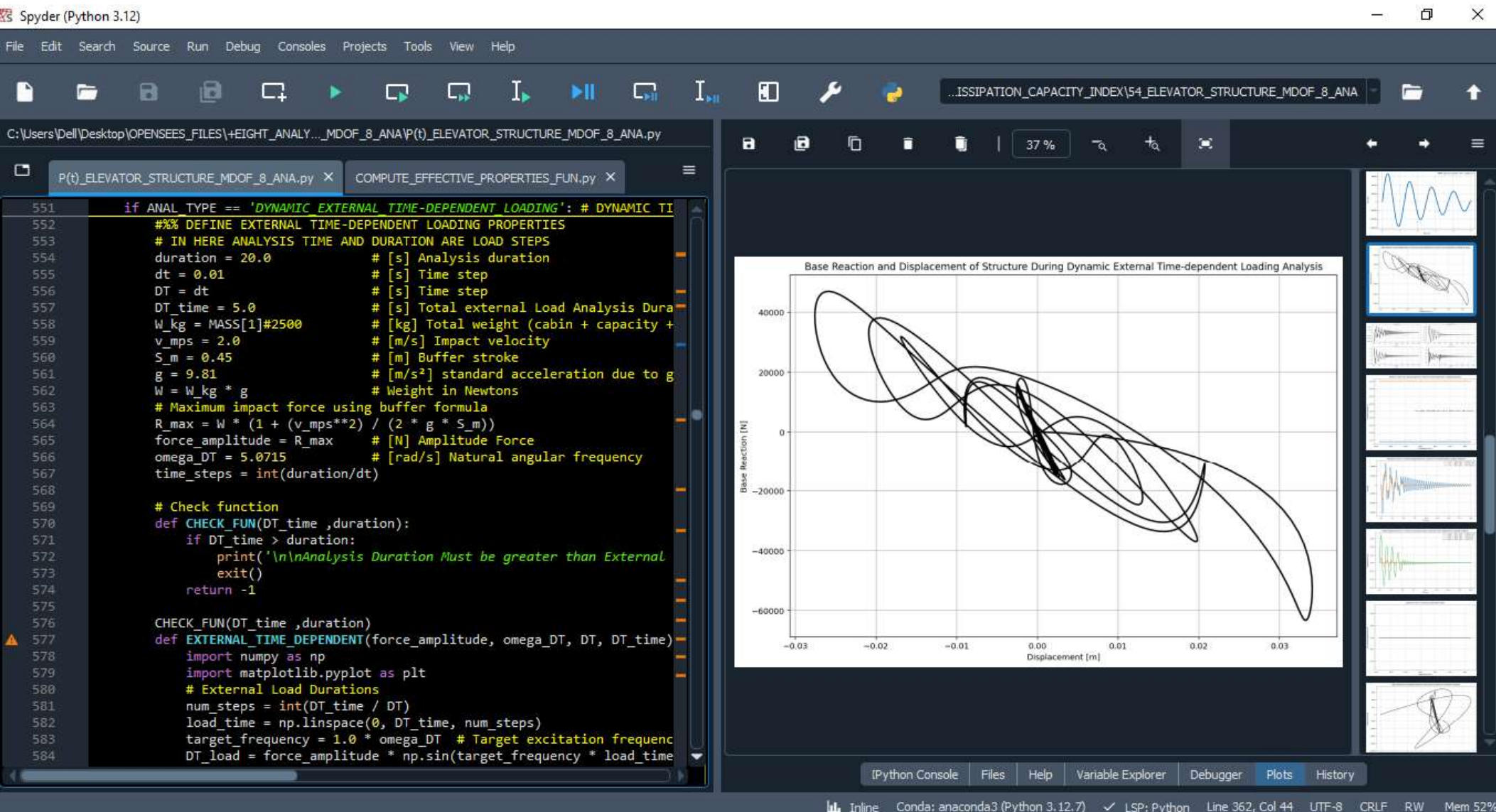


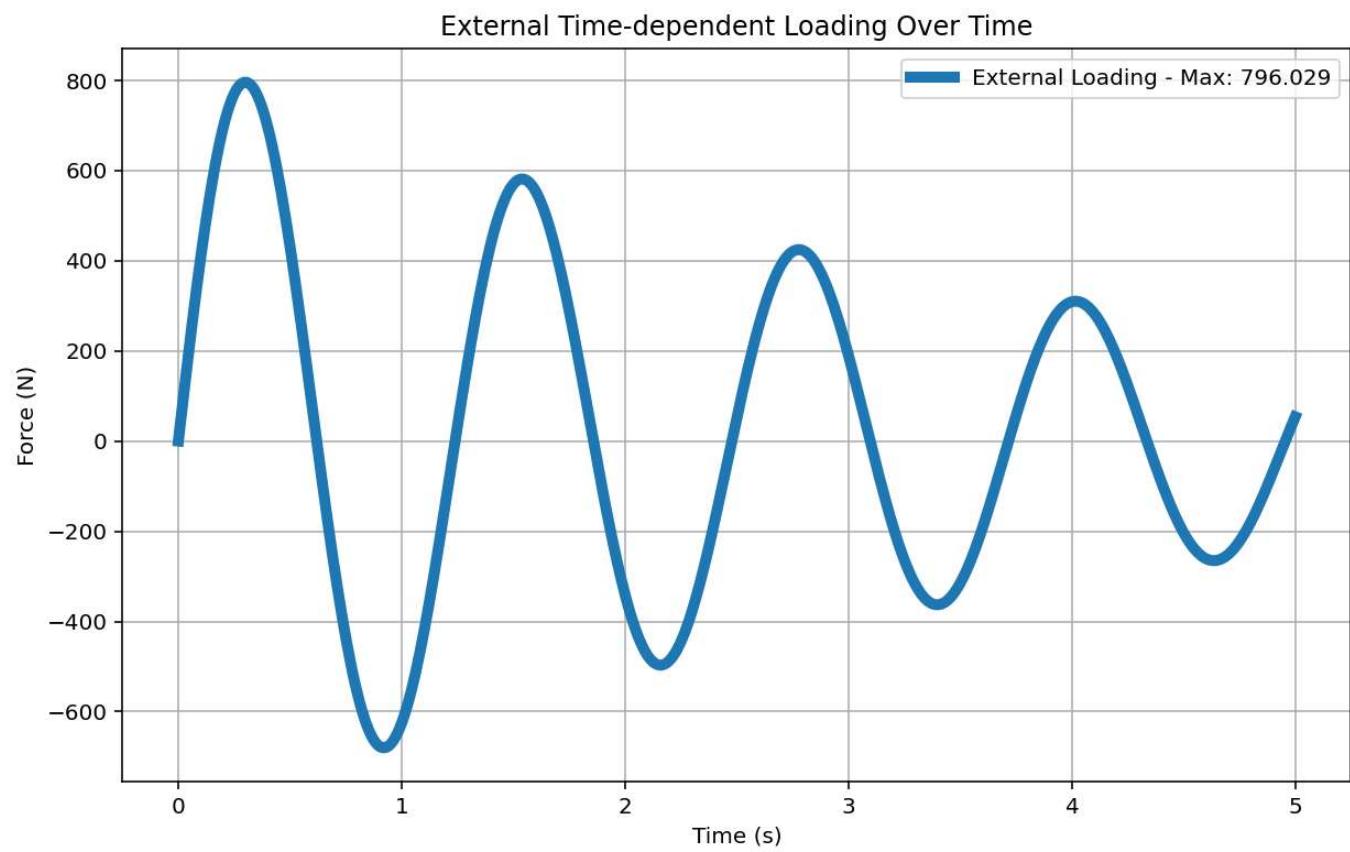


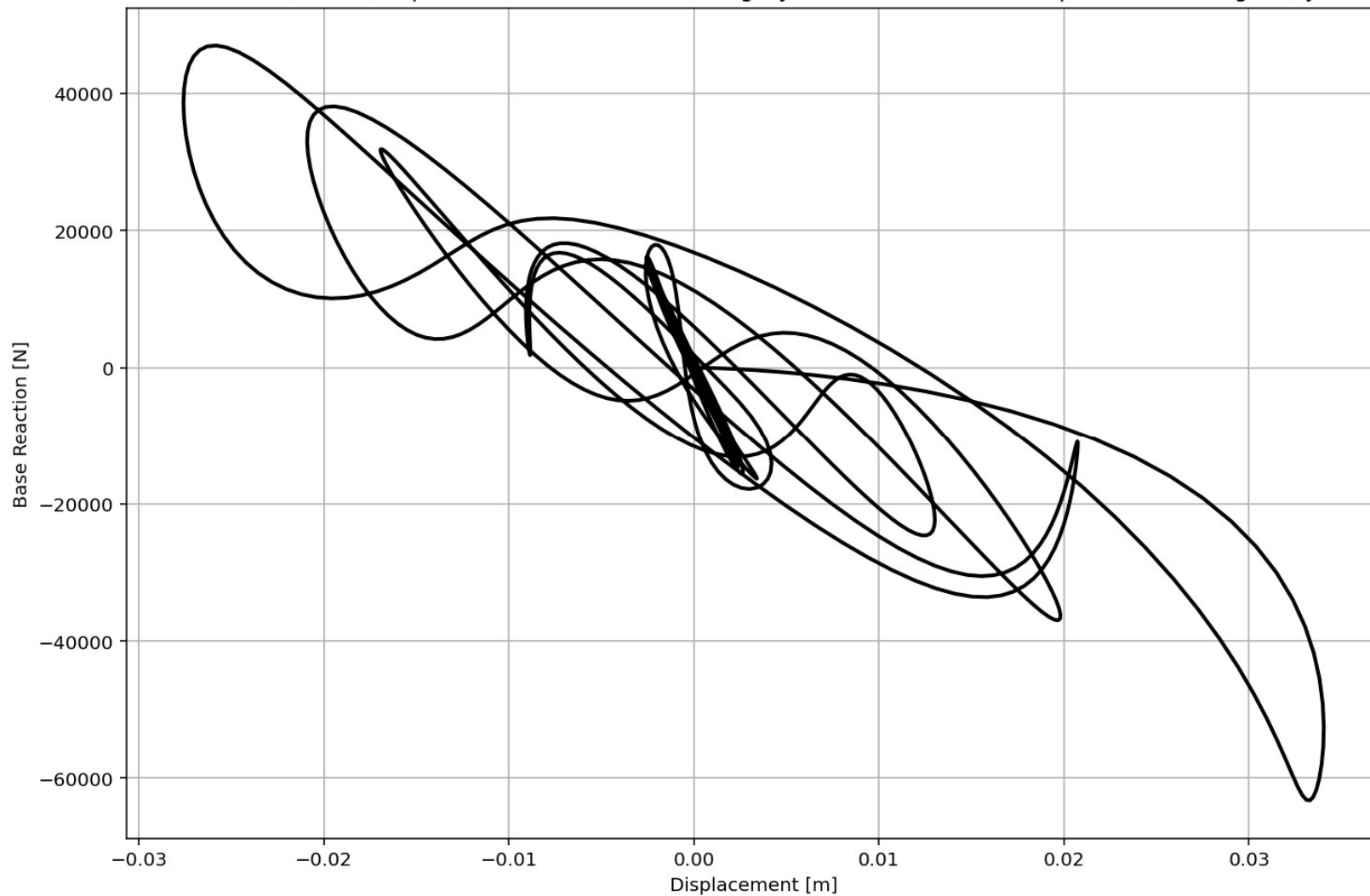


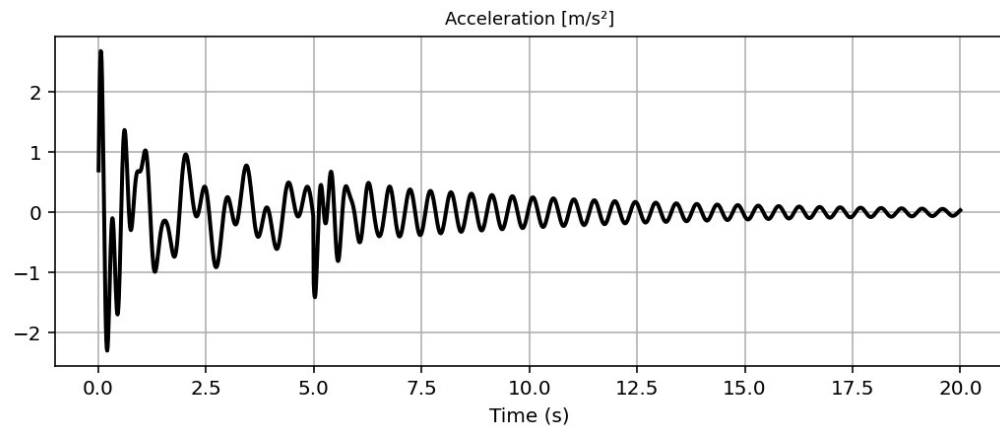
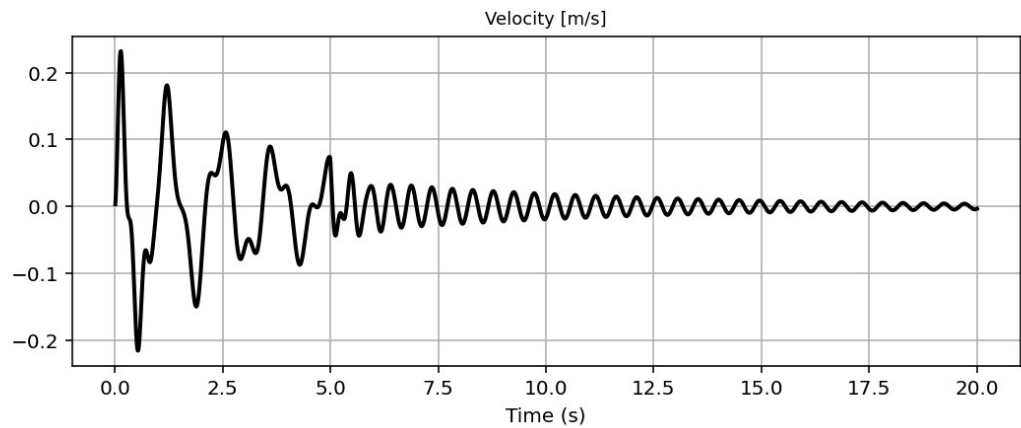
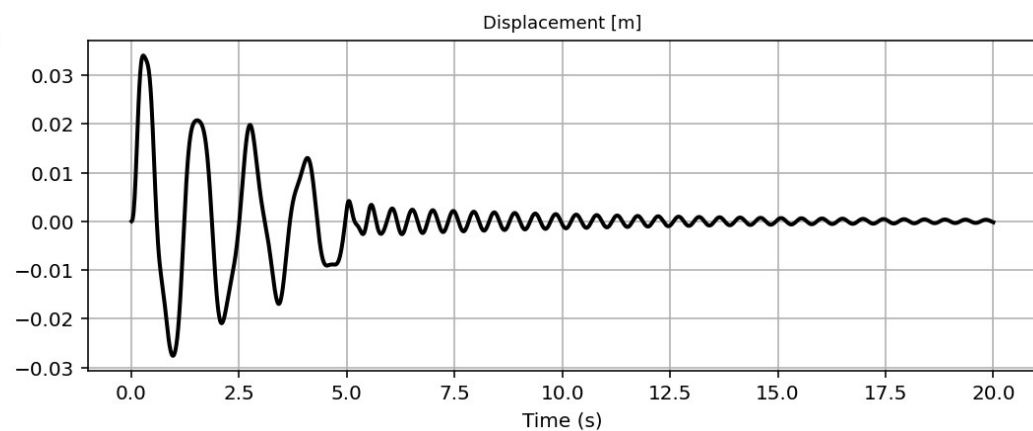
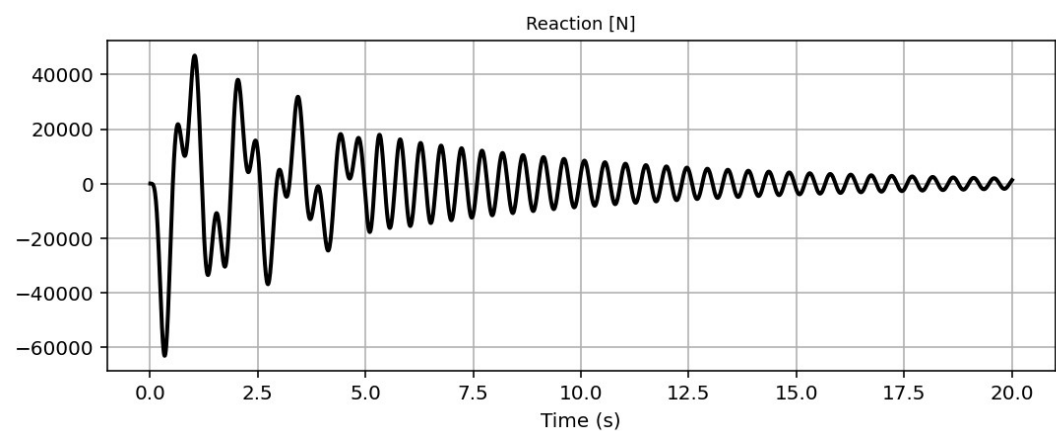


DYNAMIC TIME-HISTORY WITH EXTERNAL TIME-DEPENDENT LOADING ANALYSIS

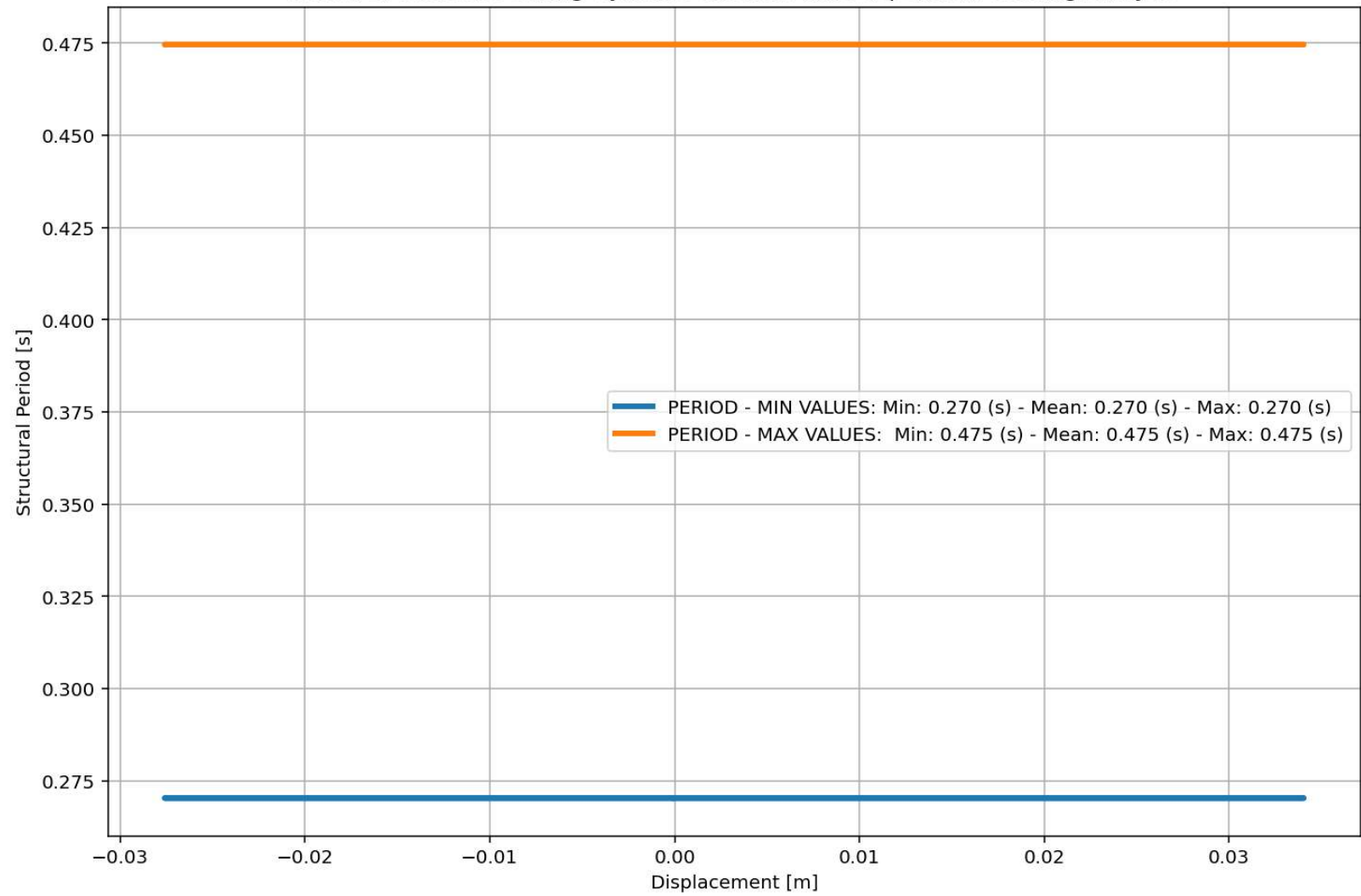


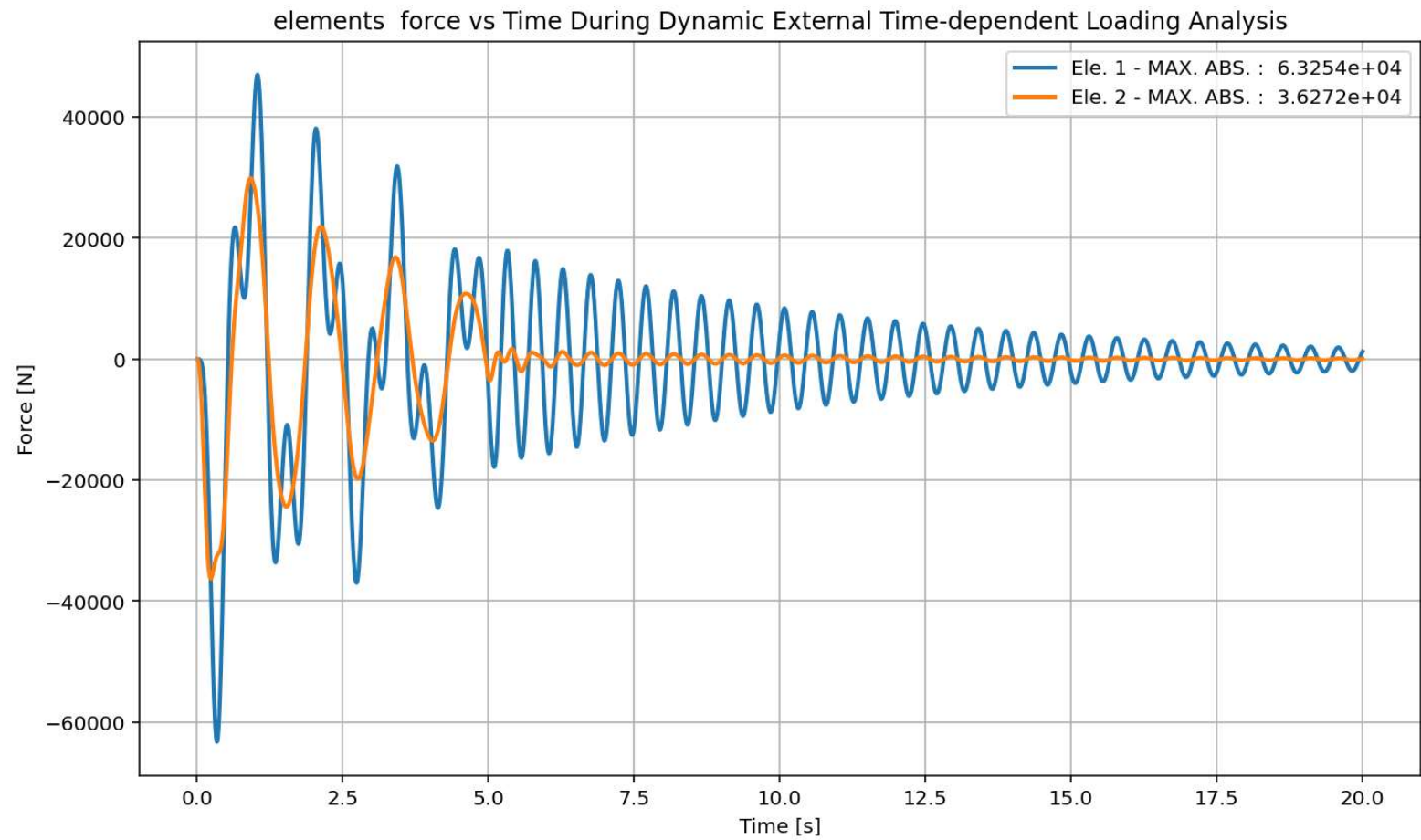




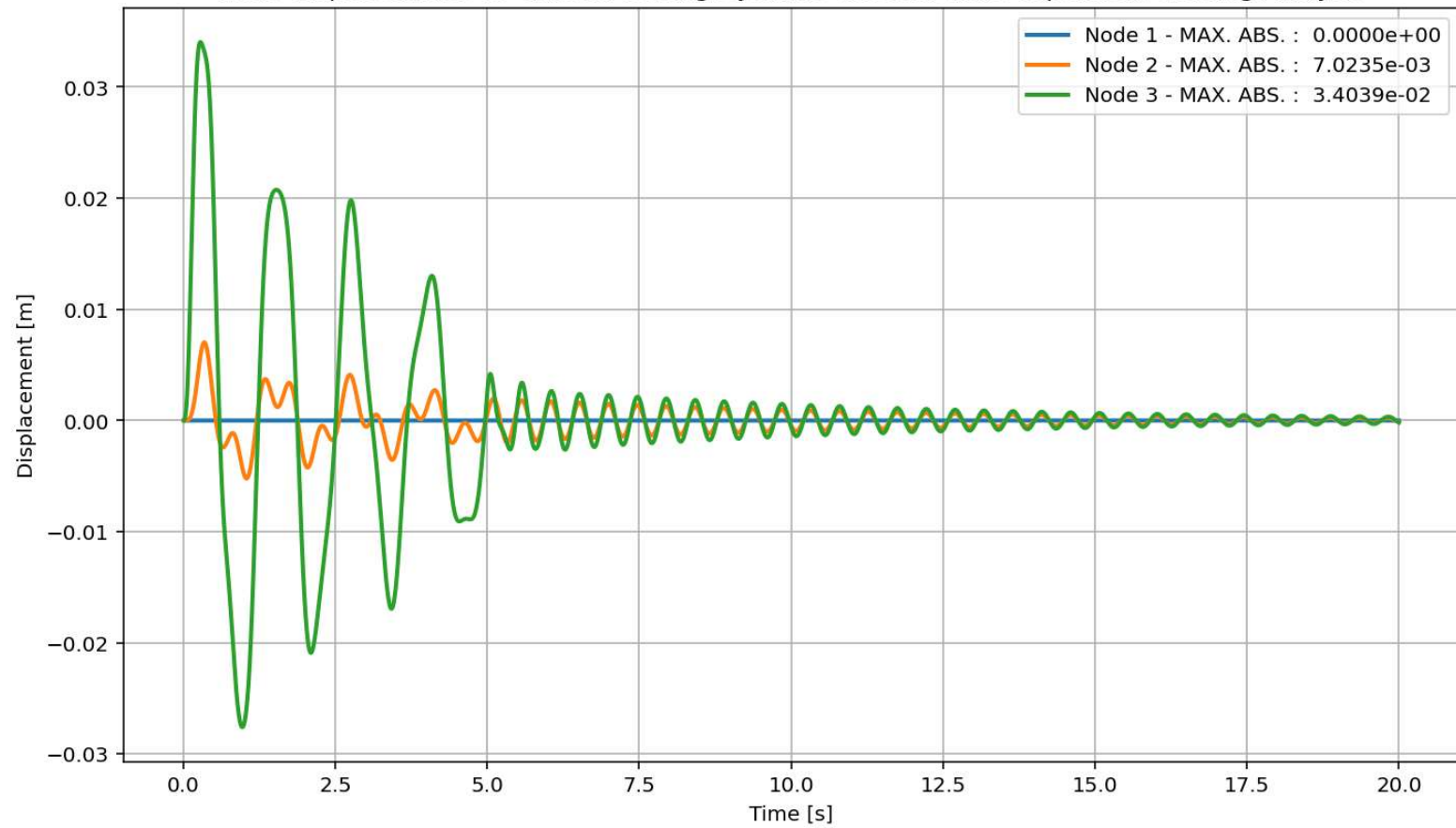


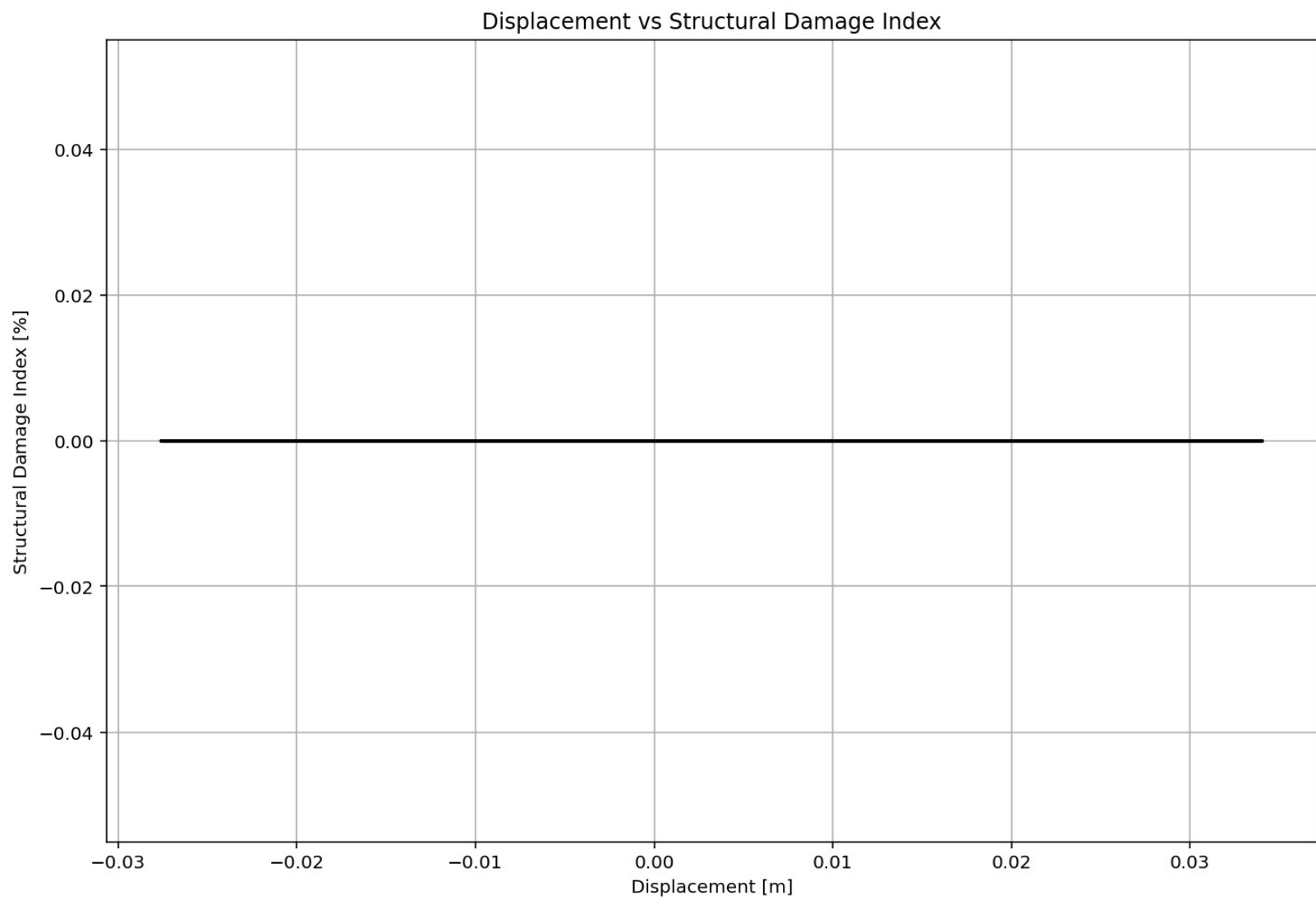
Period of Structure During Dynamic External Time-dependent Loading Analysis





Node Displacements vs Time for During Dynamic External Time-dependent Loading Analysis





FREE-VIBRATION ANALYSIS

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\De\l\Desktop\OPENSEES_FILES\+EIGHT_ANALY..._MDOF_8_ANA\P(t)_ELEVATOR_STRUCTURE_MDOF_8_ANA.py

P(t)_ELEVATOR_STRUCTURE_MDOF_8_ANA.py x COMPUTE_EFFECTIVE_PROPERTIES_FUN.py x

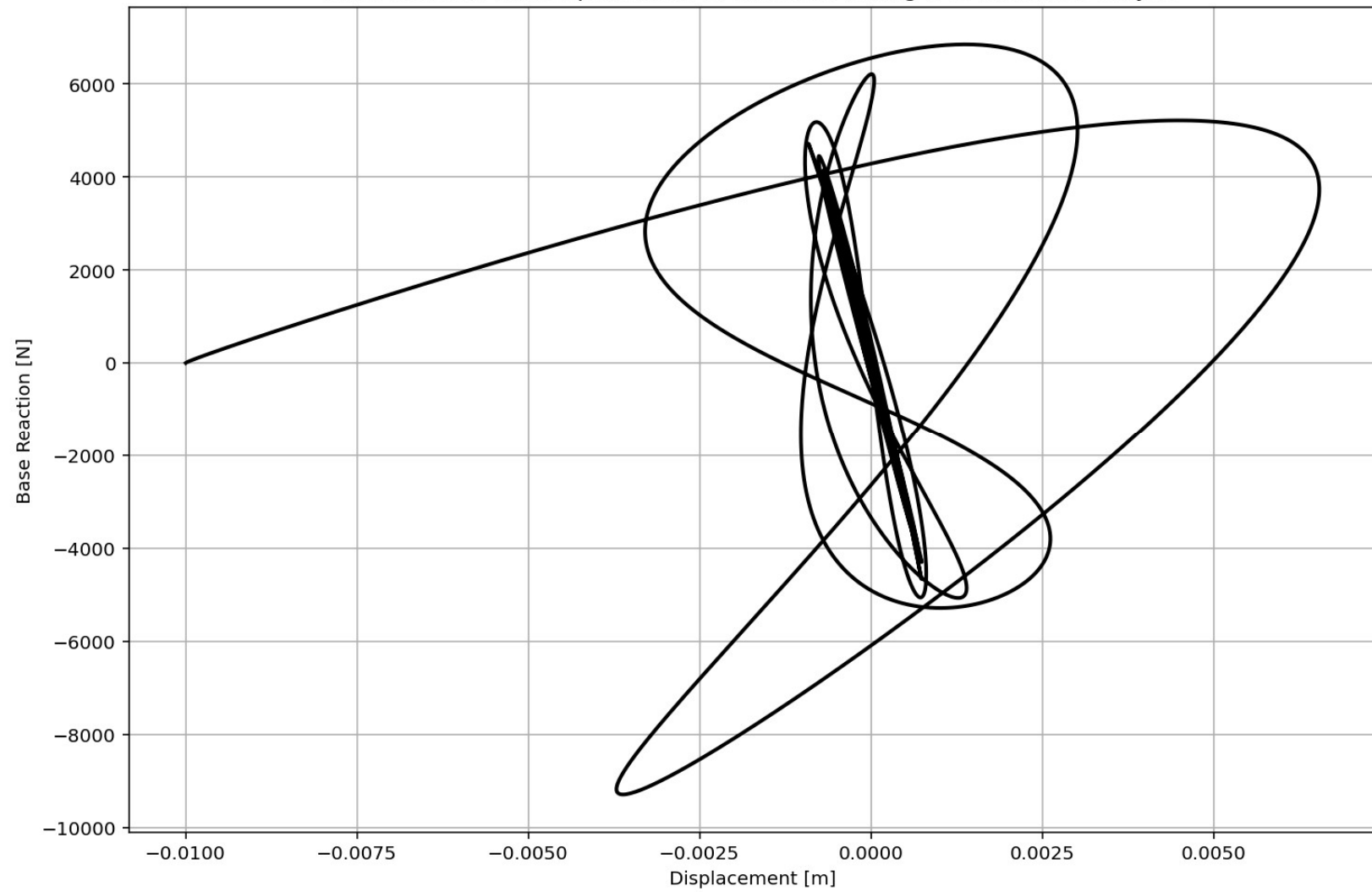
```
687
688 if ANAL_TYPE == 'FREE-VIBRATION': # DYNAMIC TIME-HISTORY ANALYSIS
689     %% DEFINE PARAMETERS FOR FREE-VIBRATION ANALYSIS
690     u0 = -0.010 # [m] Initial displacement
691     v0 = 0.000015 # [m/s] Initial velocity
692     a0 = 0.000065 # [m/s^2] Initial acceleration
693     IU = True # Free Vibration with Initial Di
694     IV = True # Free Vibration with Initial Ve
695     IA = True # Free Vibration with Initial Ac
696     duration = 20.0 # [s] Analysis duration
697     dt = 0.001 # [s] Time step
698     DR = 0.03 # Damping Ratio
699
700 if IU == True:
701     # Define initial displacment
702     ops.setNodeDisp(center_node, 1, u0, '-commit') # INFO LINK: htt
703 if IV == True:
704     # Define initial velocity
705     ops.setNodeVel(center_node, 1, v0, '-commit') # INFO LINK: htt
706 if IA == True:
707     # Define initial acceleration
708     ops.setNodeAccel(center_node, 1, a0, '-commit') # INFO LINK: ht
709
710 ops.constraints('Plain')
711 ops.numberer('Plain')
712 ops.system('BandGeneral')
713 ops.test('NormDispIncr', MAX_TOLERANCE, MAX_ITERATIONS) # INFO LINK
714 #ops.integrator('CentralDifference') # JUST FOR LINEAR ANALYSIS -
715 alpha=0.5; beta=0.25;
716 ops.integrator('Newmark', alpha, beta) # INFO LINK: https://opensee
717 #alpha=2/3; gamma=1.5-alpha; gamma=1.5-alpha; beta=(2-alpha)**2/4;
718 #ops.integrator('HHT', alpha, gamma, beta) # INFO LINK: https://ope
719 ops.algorithm('Newton') # INFO LINK: https://openseespydoc.readthe
720 ops.analysis('Transient') # INFO LINK: https://openseespydoc.readth
```

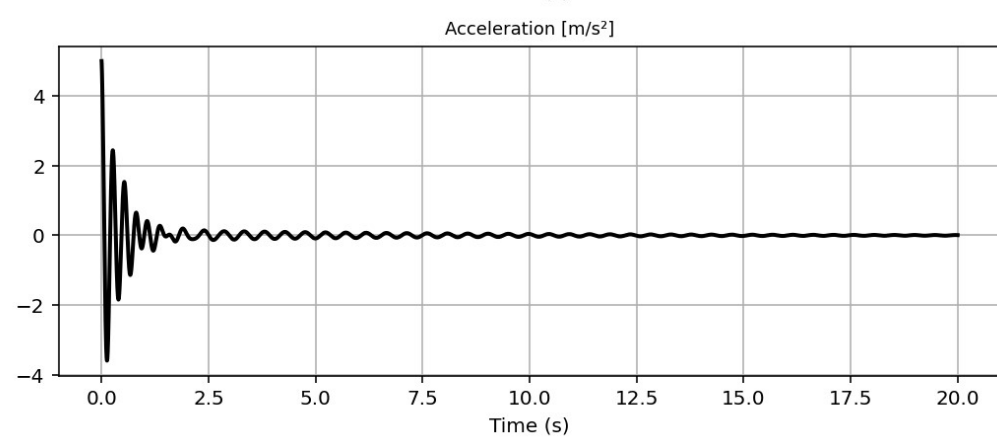
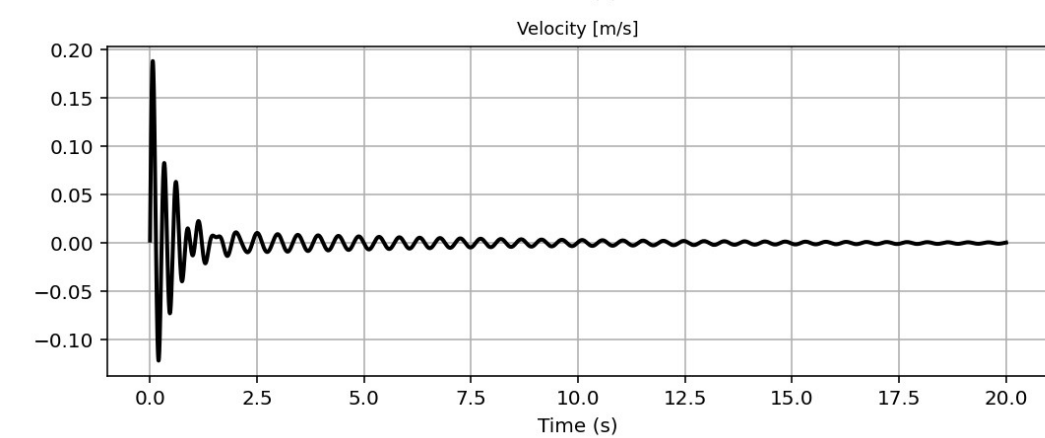
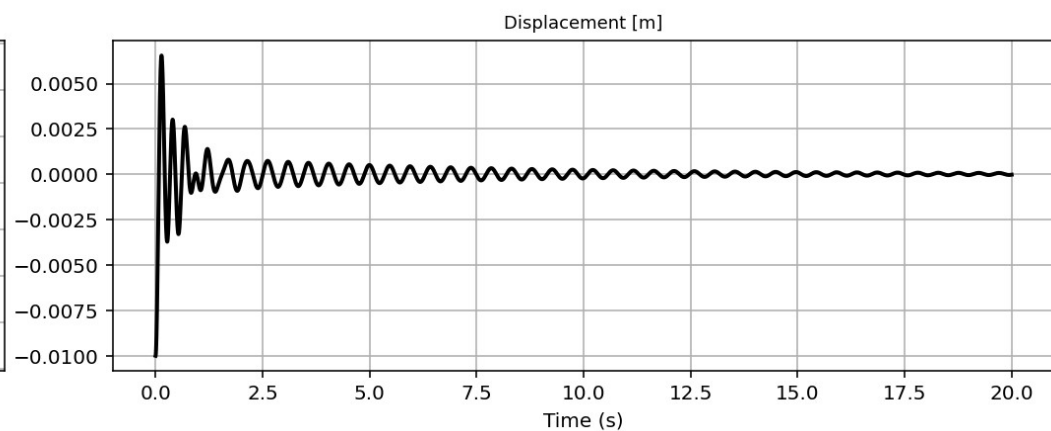
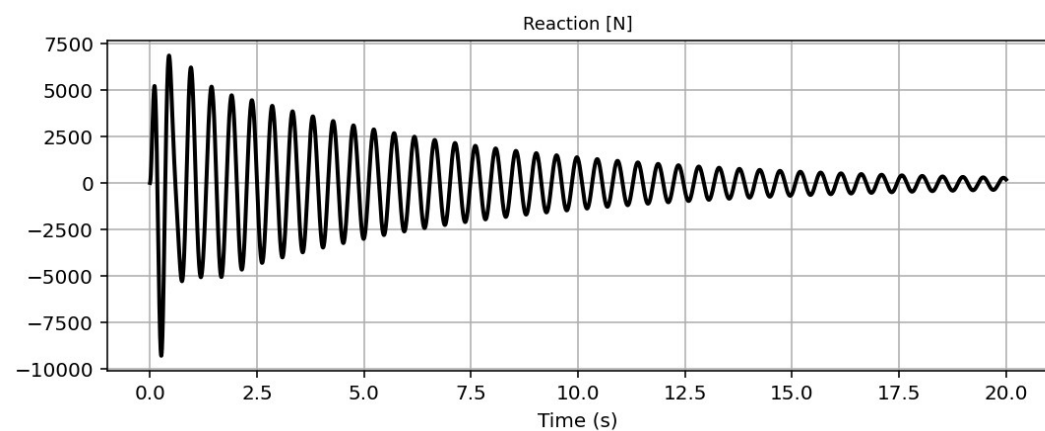
Base Reaction and Displacement of Structure During Free-vibration Analysis

IPython Console Files Help Variable Explorer Debugger Plots History

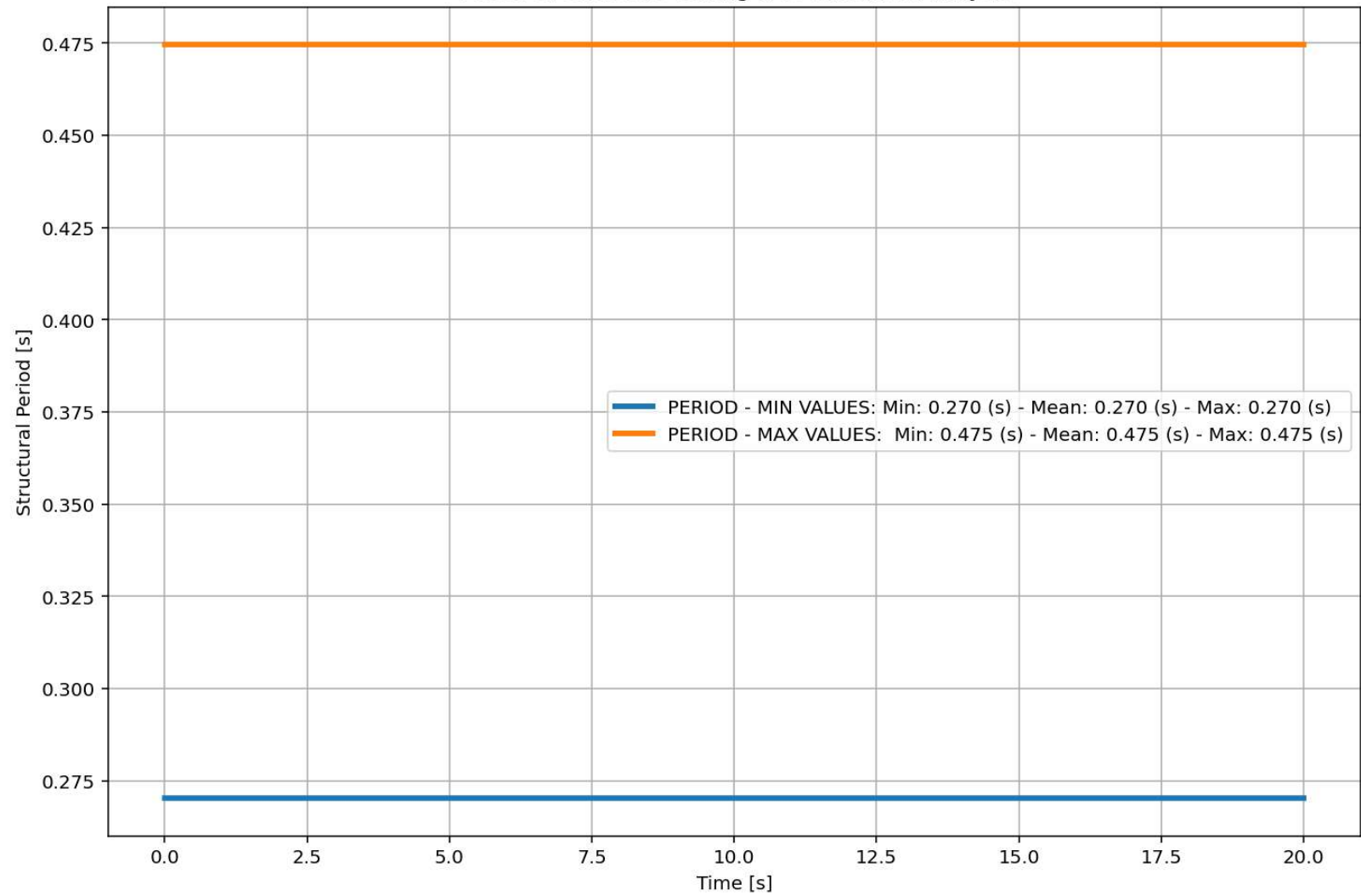
Inline Conda: anaconda3 (Python 3.12.7) ✓ LSP: Python Line 362, Col 44 UTF-8 CRLF RW Mem 53%

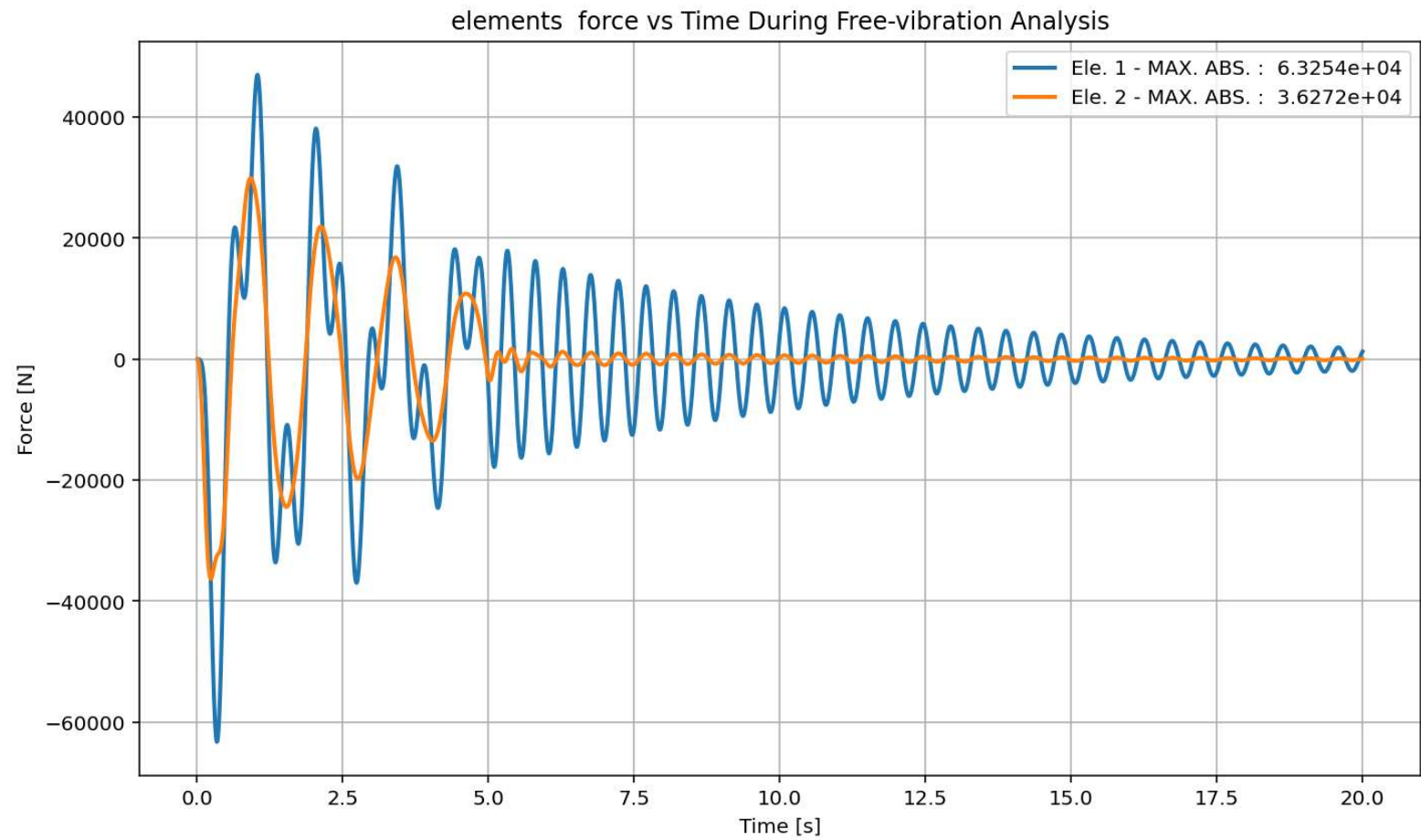
Base Reaction and Dispalcement of Structure During Free-vibration Analysis



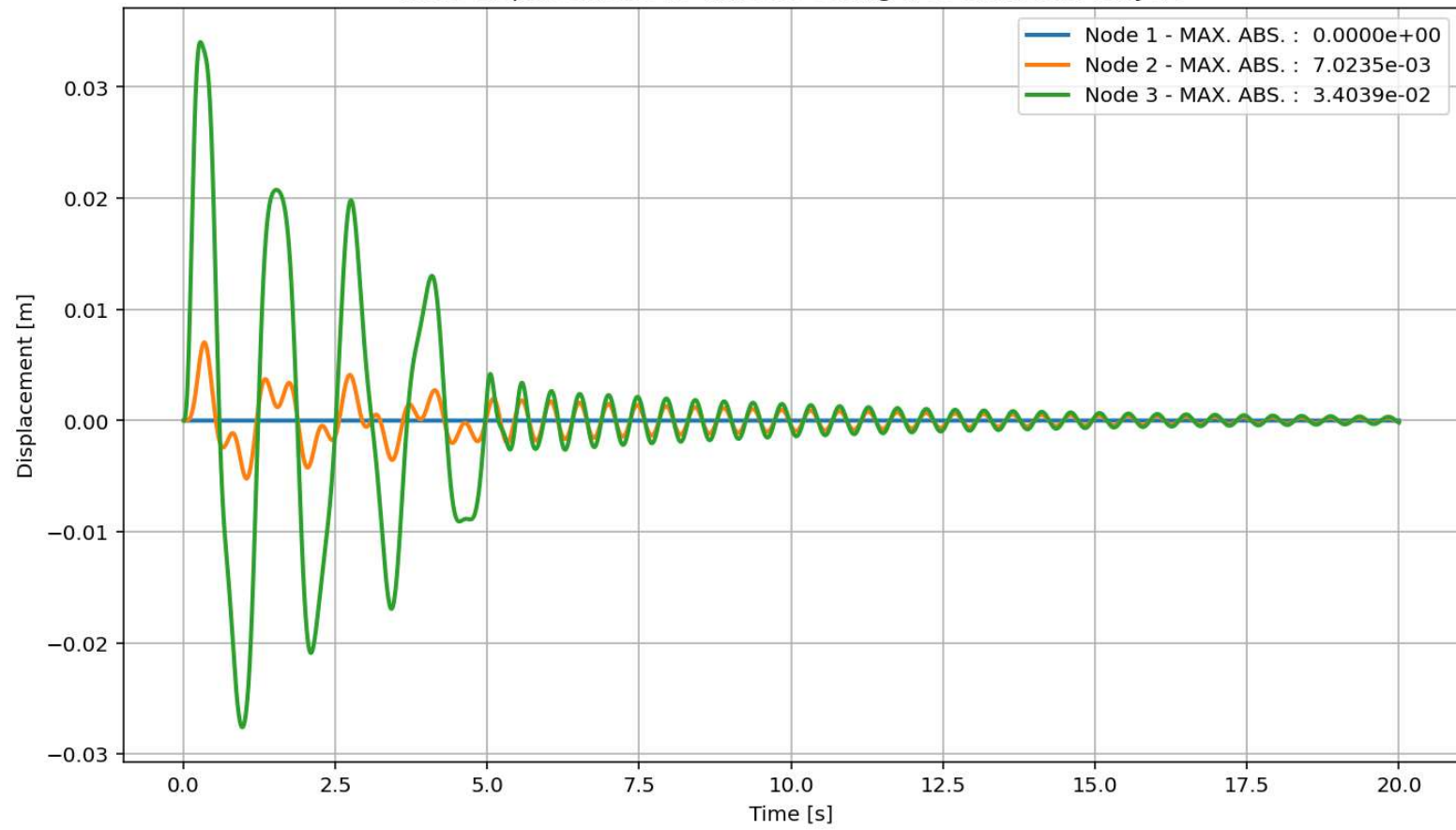


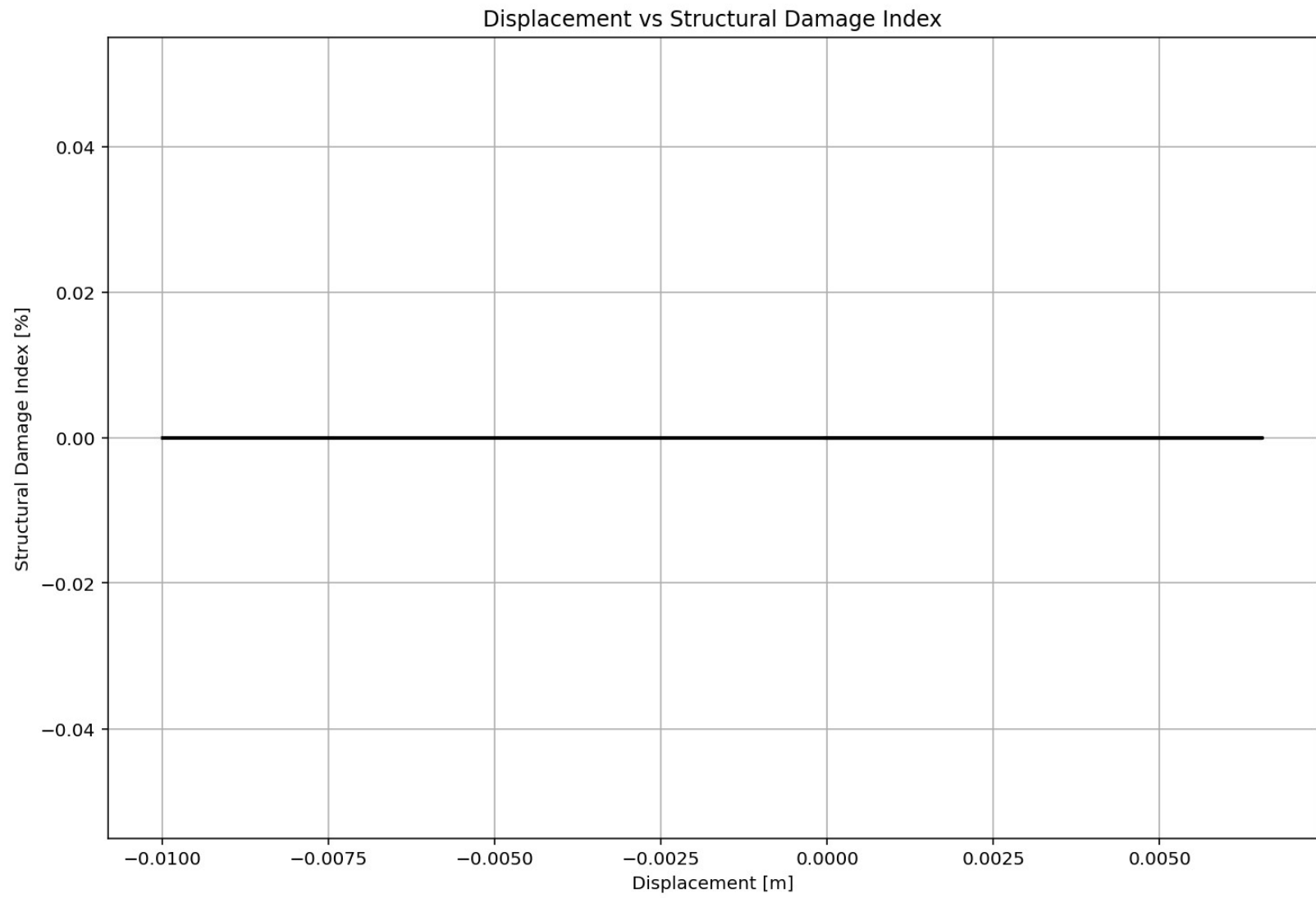
Period of Structure During Free-vibration Analysis



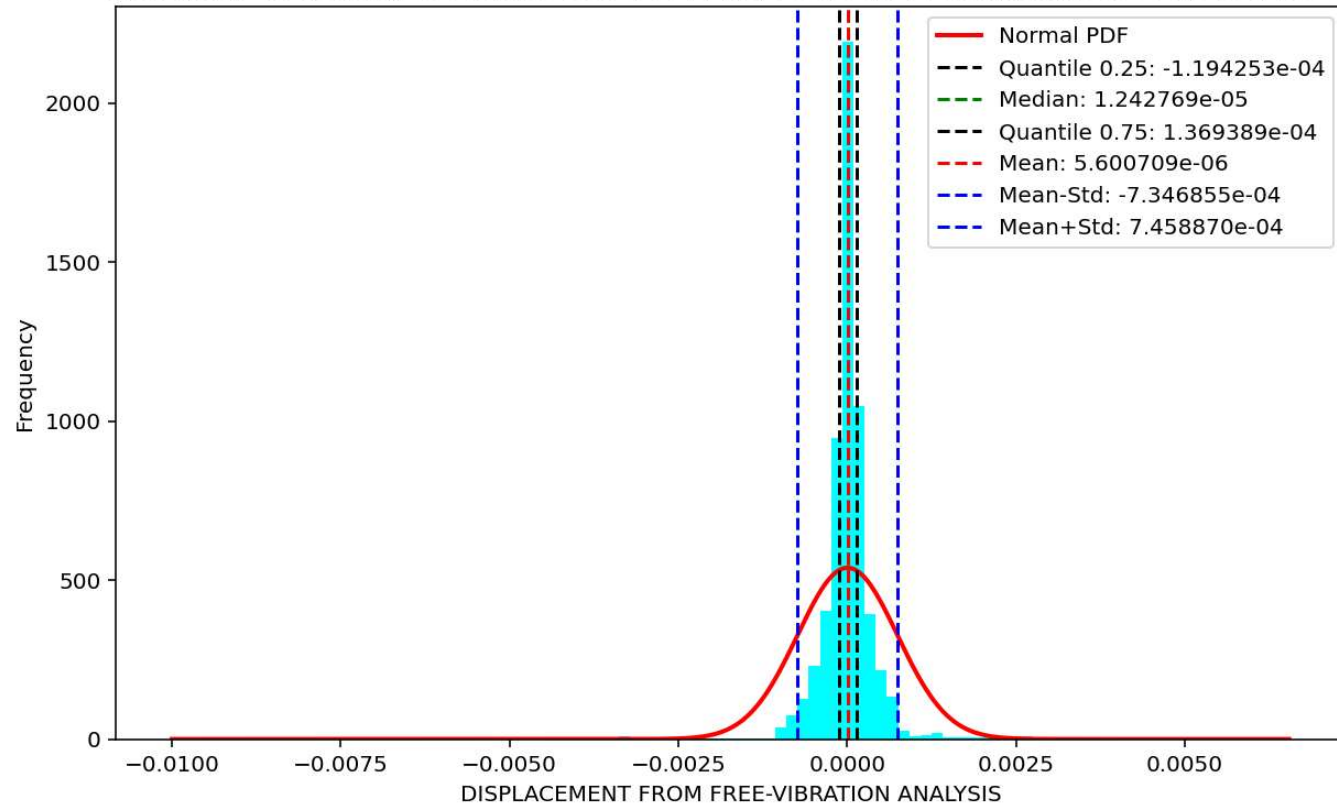


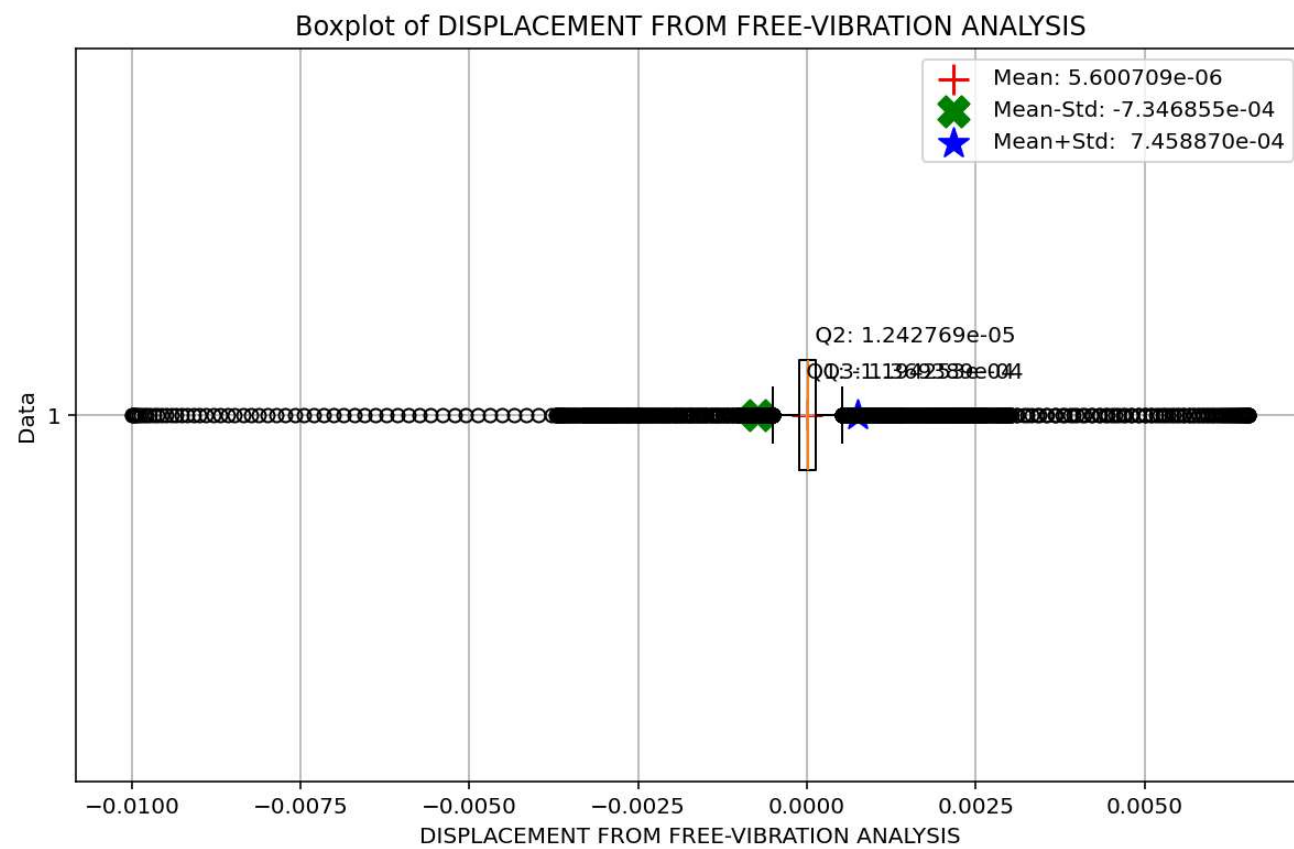
Node Displacements vs Time for During Free-vibration Analysis



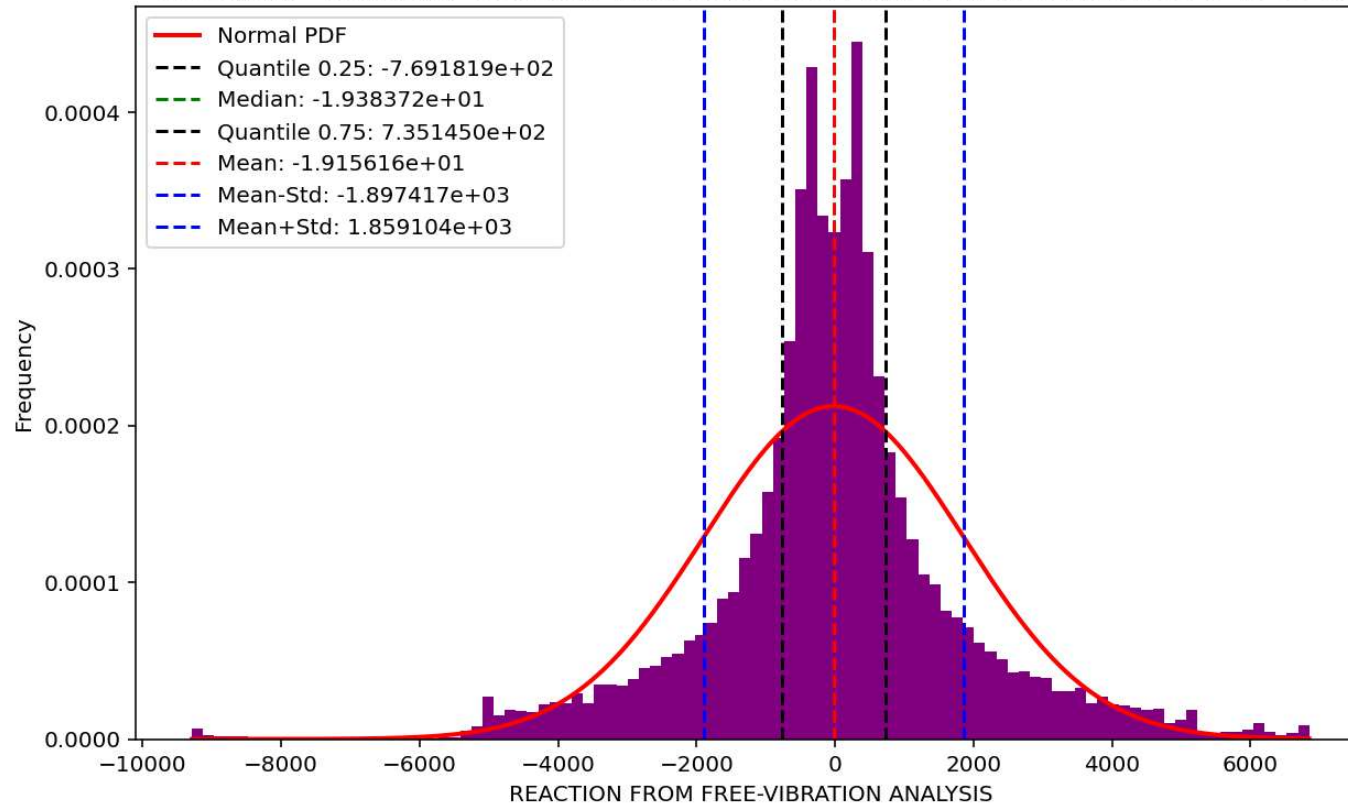


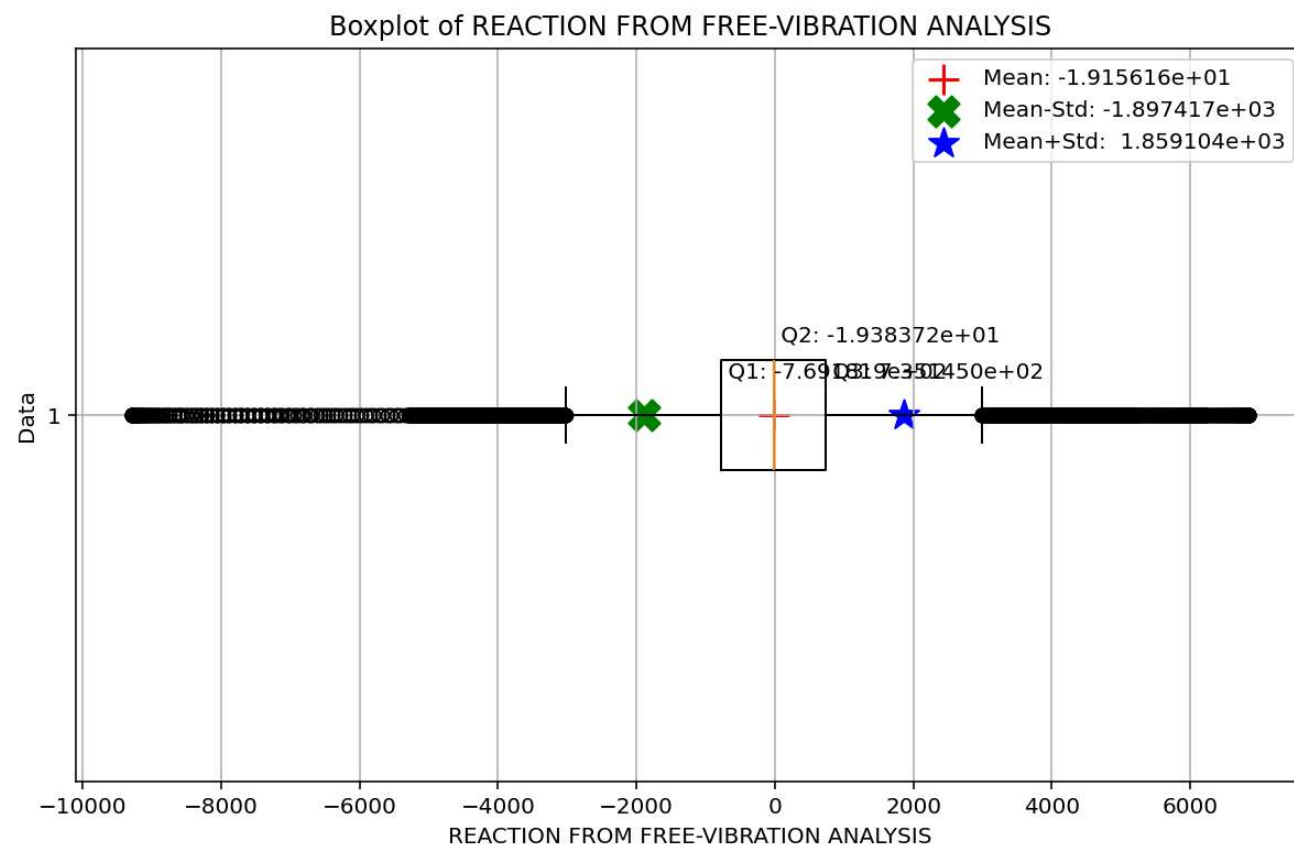
Histogram - Probability of Positive DISPLACEMENT FROM FREE-VIBRATION ANALYSIS is 52.35 %



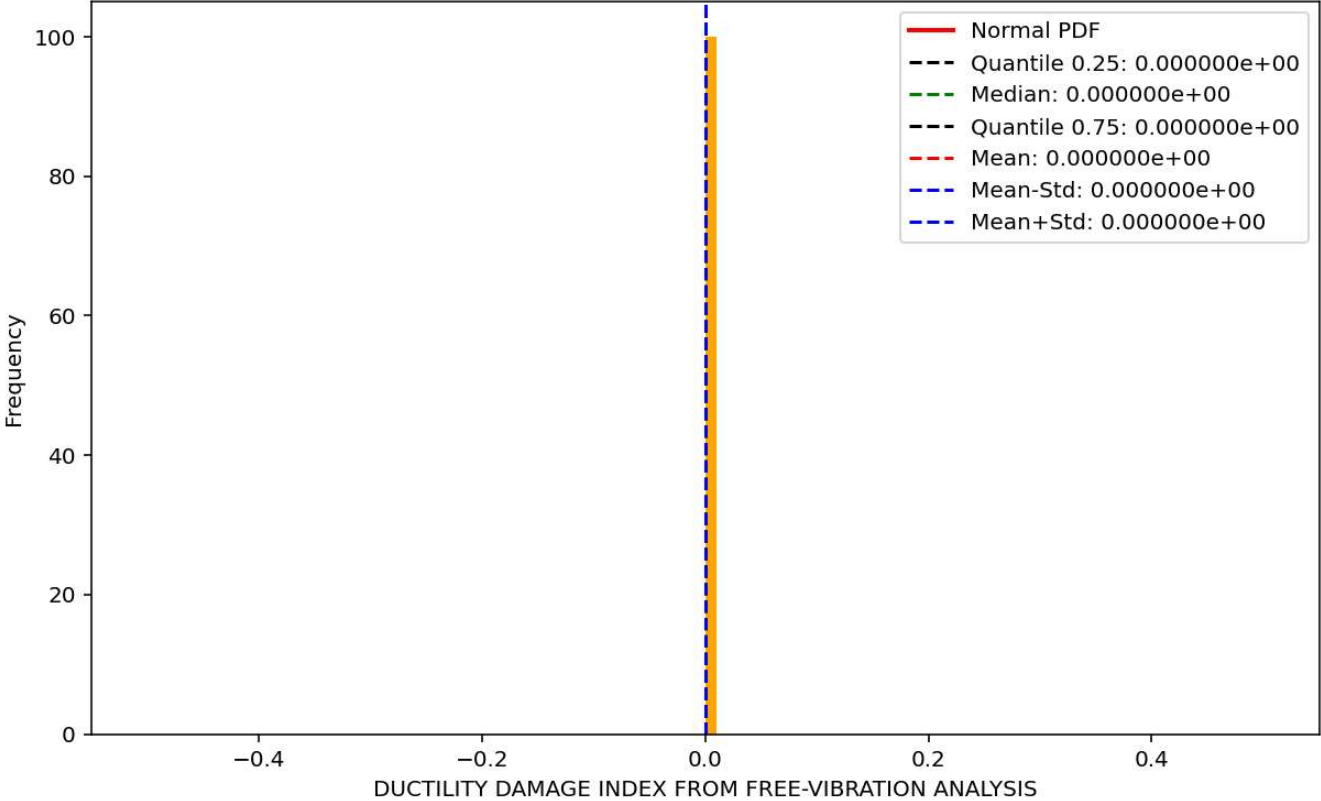


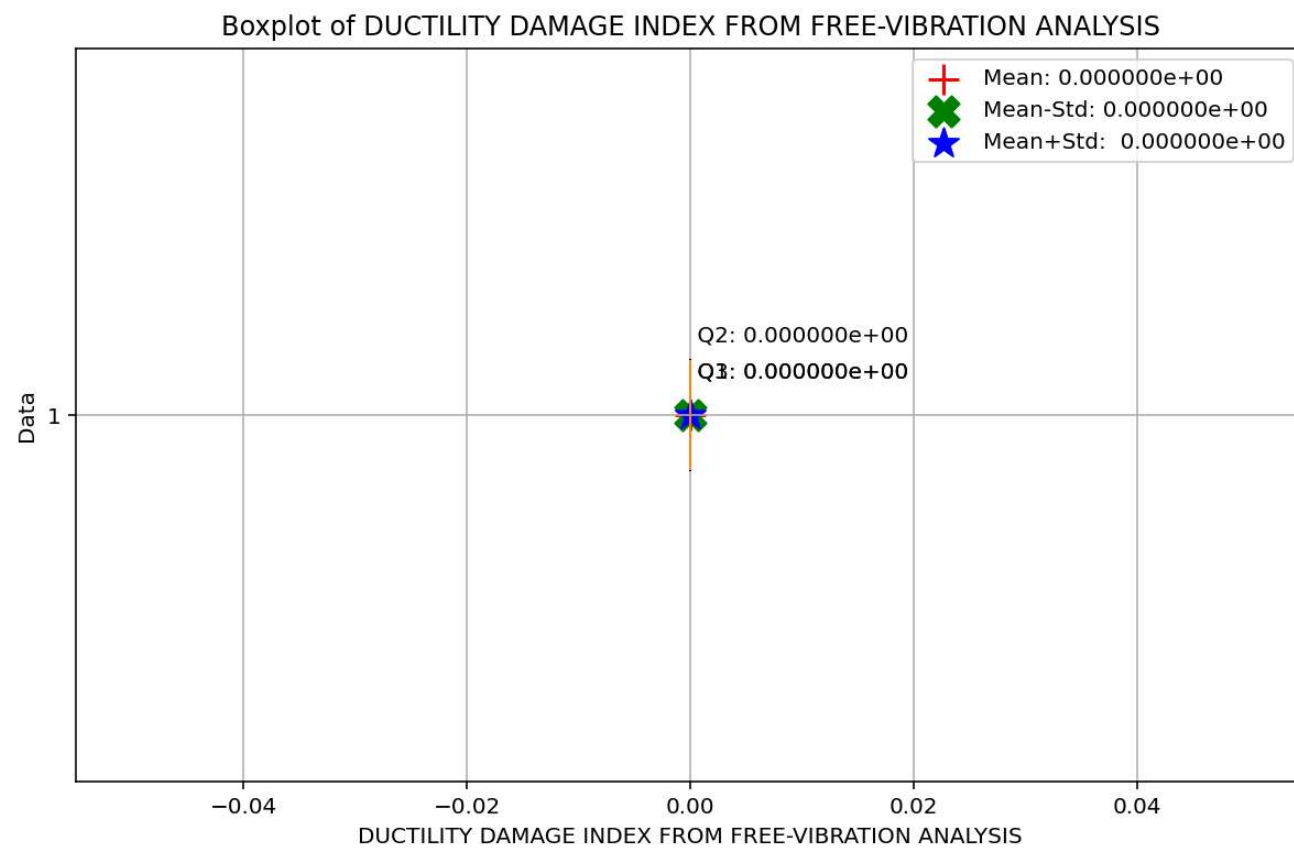
Histogram - Probability of Positive REACTION FROM FREE-VIBRATION ANALYSIS is 49.38 %



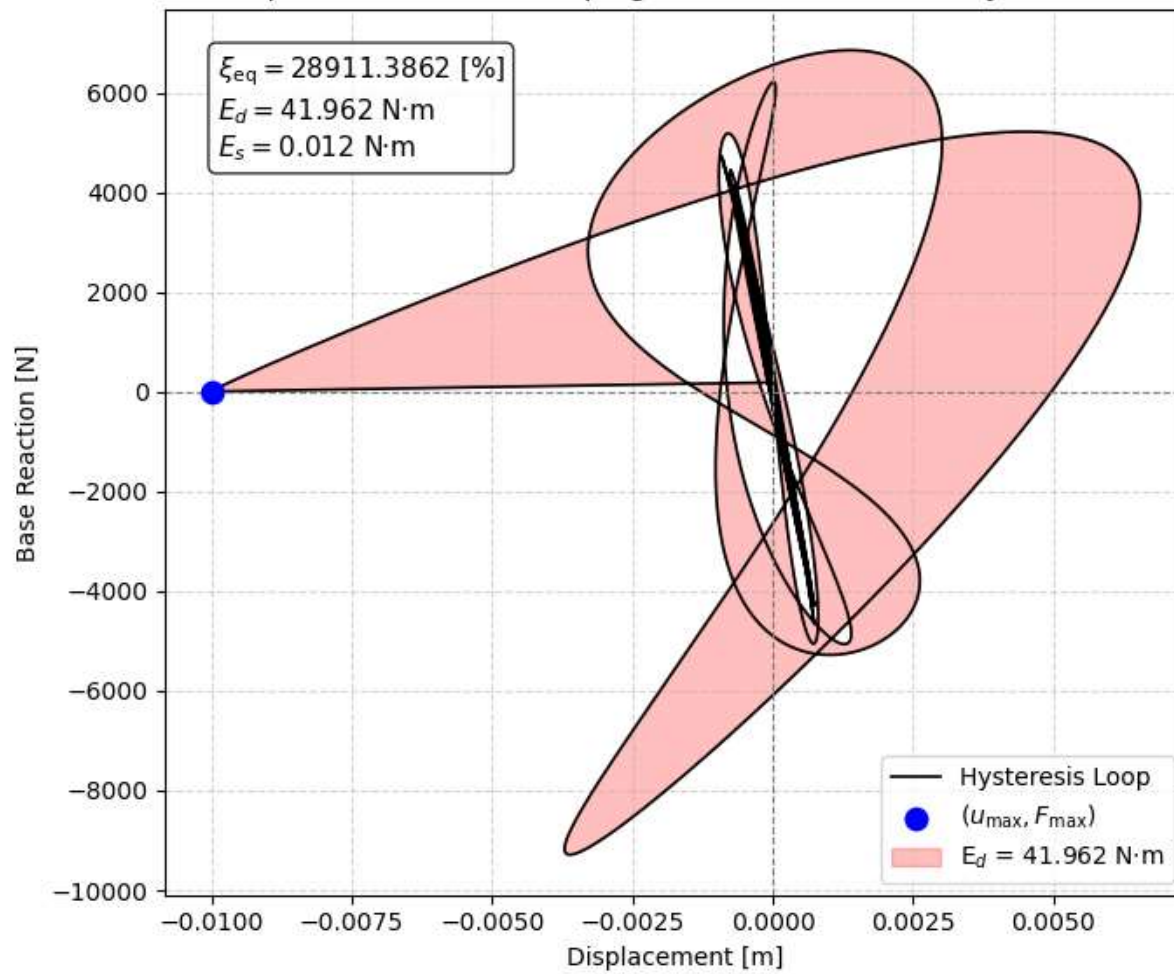


Histogram - Probability of Positive DUCTILITY DAMAGE INDEX FROM FREE-VIBRATION ANALYSIS is 0.00 %





Equivalent viscous damping ratio - Free-vibration Hysteresis



SEISMIC ANALYSIS

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY..._MDOF_8_ANA\P(t)_ELEVATOR_STRUCTURE_MDOF_8_ANA.py

P(t)_ELEVATOR_STRUCTURE_MDOF_8_ANA.py x COMPUTE_EFFECTIVE_PROPERTIES_FUN.py x

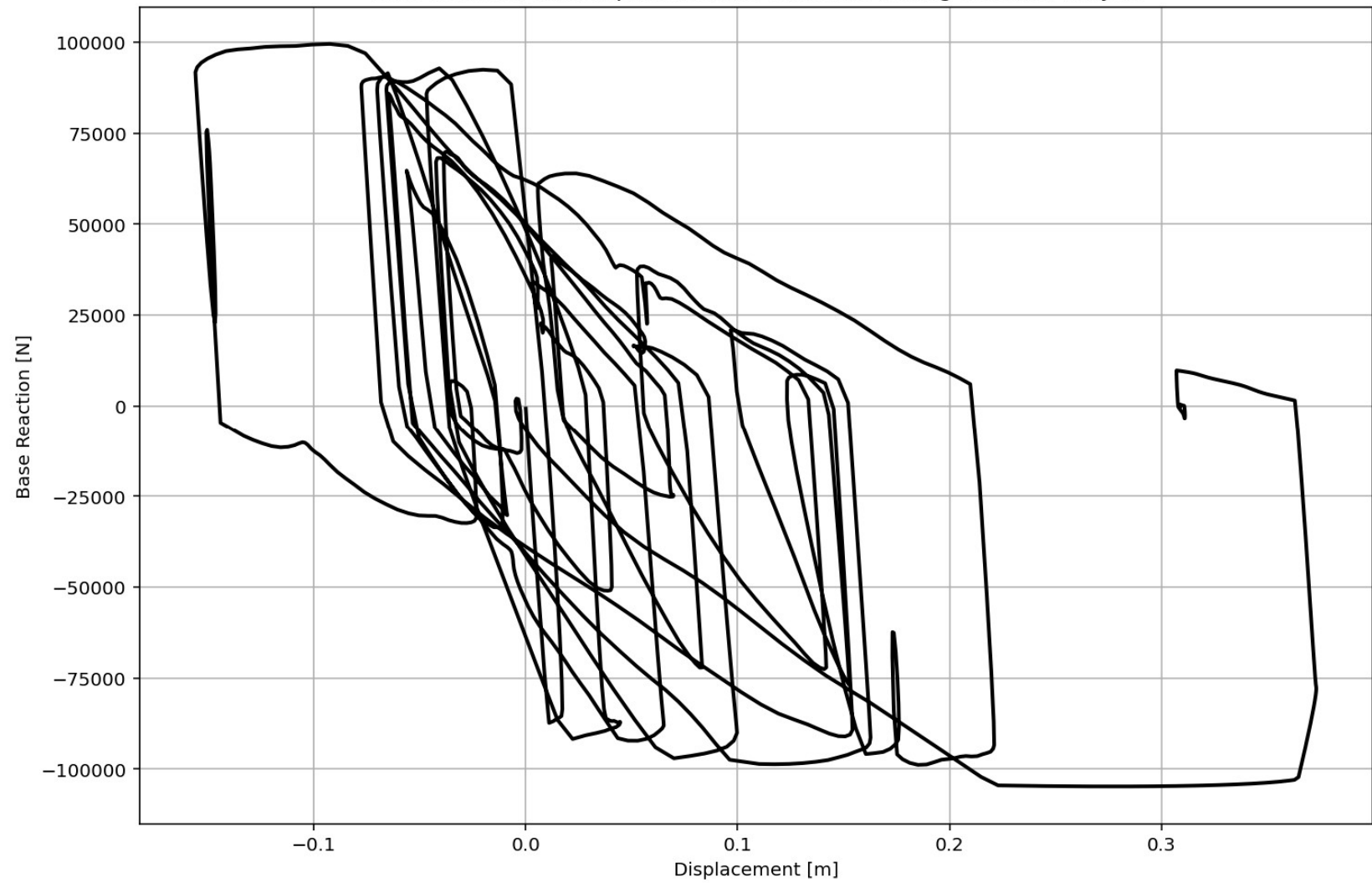
```
770 if ANAL_TYPE == 'SEISMIC': # DYNAMIC TIME-HISTORY ANALYSIS
771     %% DEFINE PARAMETERS FOR SEISMIC ANALYSIS
772     duration = 20.0           # [s] Analysis duration
773     dt = 0.01                # [s] Time step
774     GMfact = 9810            # [mm/s²] standard acceleration o
775     SSF_X = 0.05             # Seismic Acceleration Scale Fac
776     SSF_Y = 0.05             # Seismic Acceleration Scale Fac
777     iv0_X = 0.0005           # [mm/s] Initial velocity applie
778     iv0_Y = 0.0005           # [mm/s] Initial velocity applie
779     st_iv0 = 0.0             # [s] Initial velocity applied s
780     SEI = 'X'                # Seismic Direction
781     DR = 0.03                # Damping ratio
782
783     # Define time series for input motion (Acceleration time history)
784     if SEI == 'X':
785         SEISMIC_TAG_01 = 100
786         ops.timeSeries('Path', SEISMIC_TAG_01, '-dt', dt, '-filePath',
787             # Define load patterns
788             # pattern UniformExcitation $patternTag $dof -accel $tsTag <-ve
789         ops.pattern('UniformExcitation', SEISMIC_TAG_01, 1, '-accel', S
790     if SEI == 'Y':
791         SEISMIC_TAG_02 = 200
792         ops.timeSeries('Path', SEISMIC_TAG_02, '-dt', dt, '-filePath',
793             ops.pattern('UniformExcitation', SEISMIC_TAG_02, 2, '-accel', S
794     if SEI == 'XY':
795         SEISMIC_TAG_01 = 100
796         ops.timeSeries('Path', SEISMIC_TAG_01, '-dt', dt, '-filePath',
797             # Define load patterns
798             # pattern UniformExcitation $patternTag $dof -accel $tsTag <-ve
799         ops.pattern('UniformExcitation', SEISMIC_TAG_01, 1, '-accel', S
800         SEISMIC_TAG_02 = 200
801         ops.timeSeries('Path', SEISMIC_TAG_02, '-dt', dt, '-filePath',
802         ops.pattern('UniformExcitation', SEISMIC_TAG_02, 2, '-accel', S
803     print('Seismic Defined Done.')
```

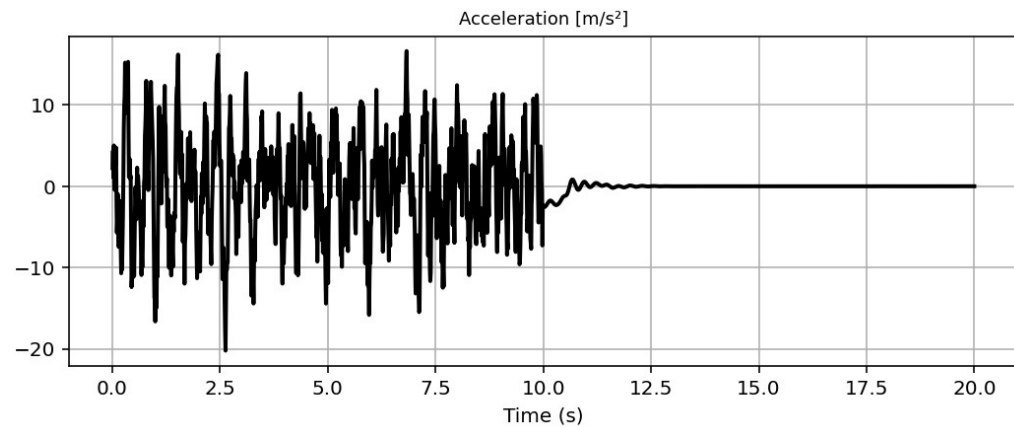
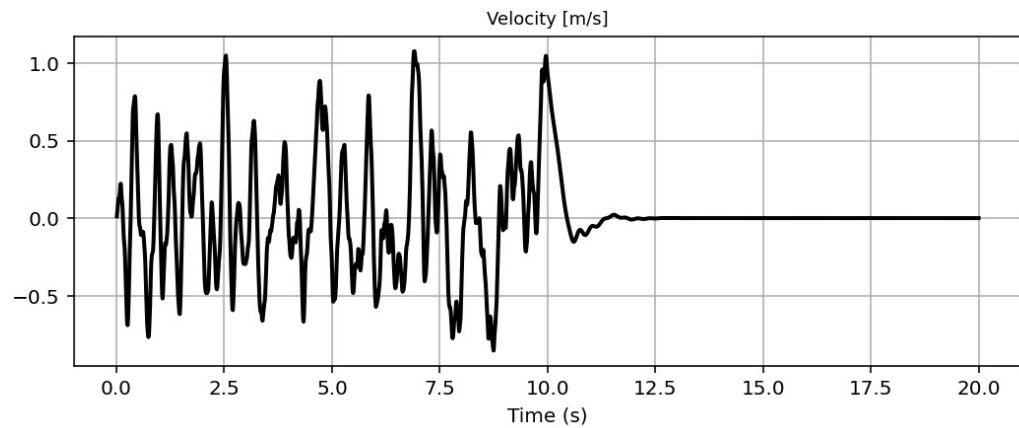
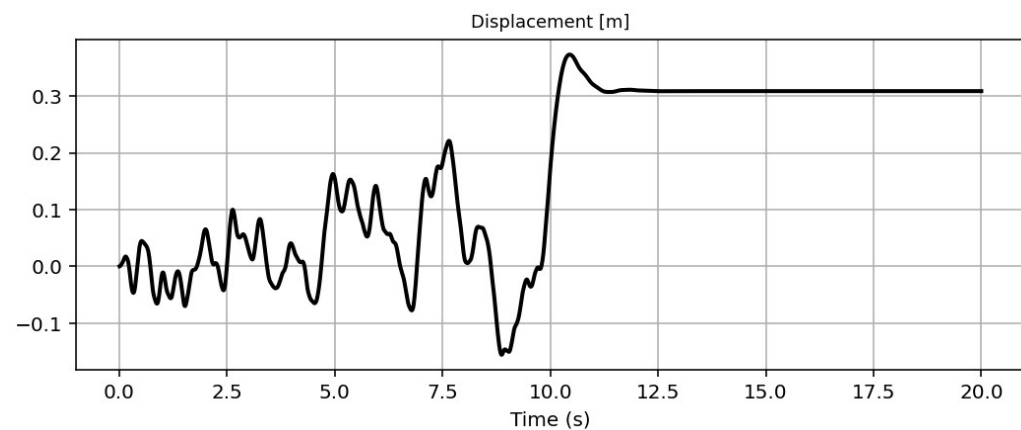
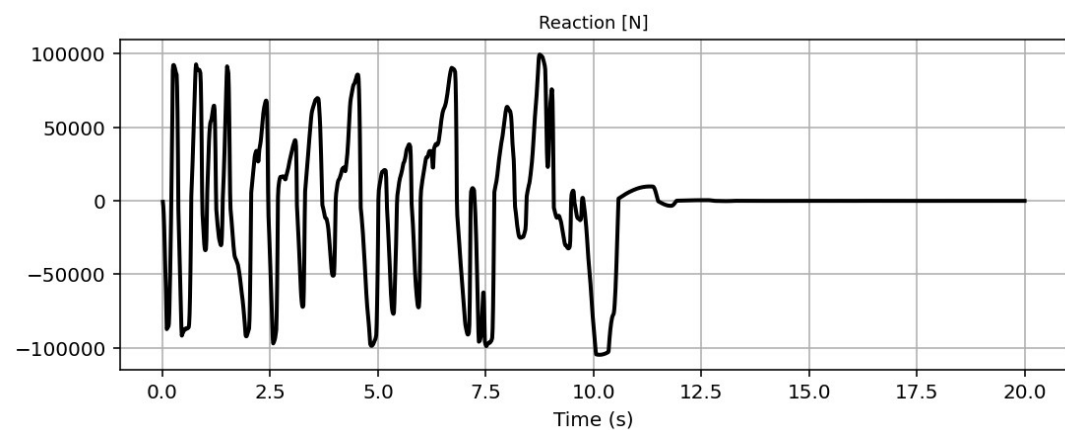
Base Reaction and Displacement of Structure During Seismic Analysis

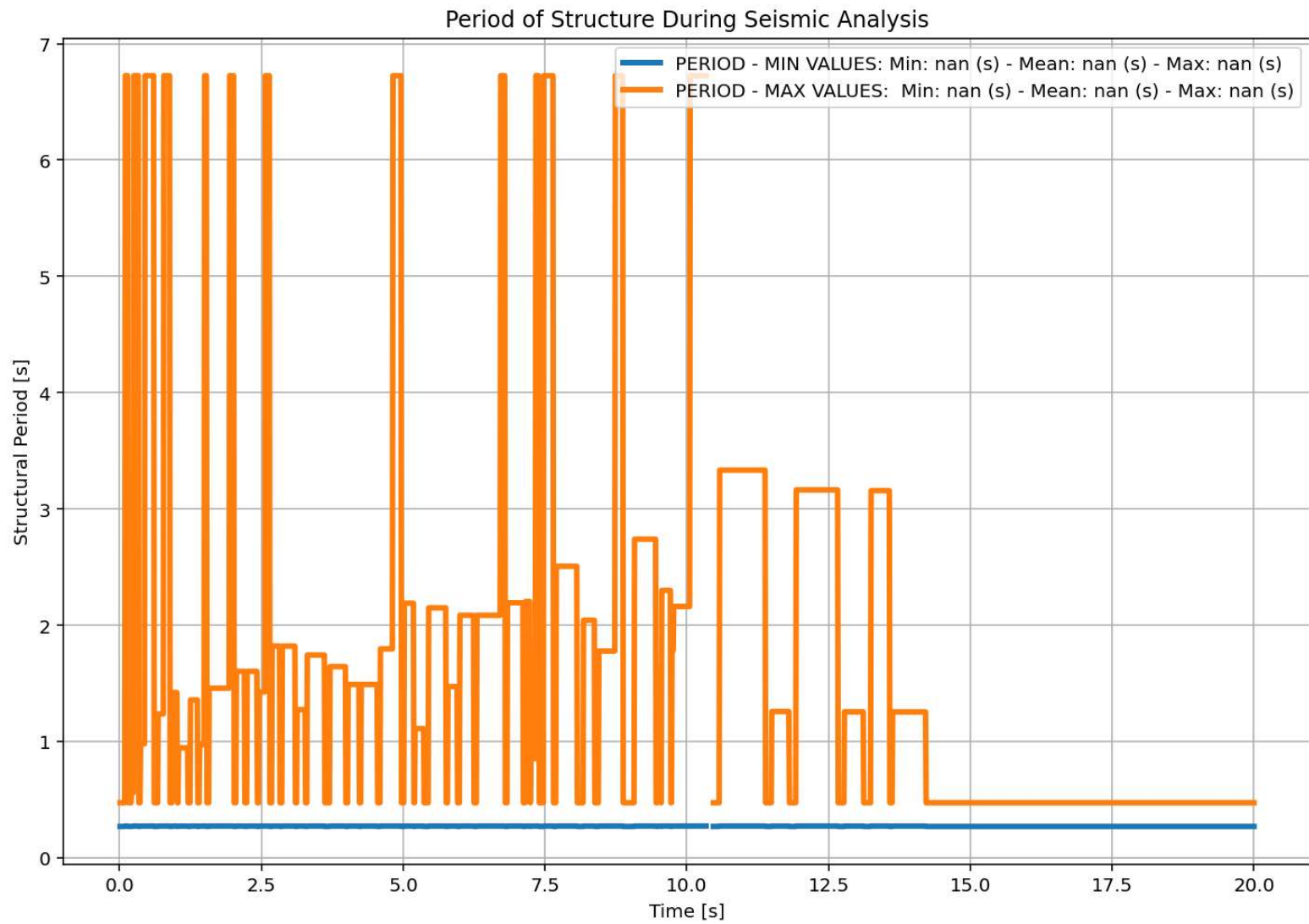
[Python Console] Files Help Variable Explorer Debugger Plots History

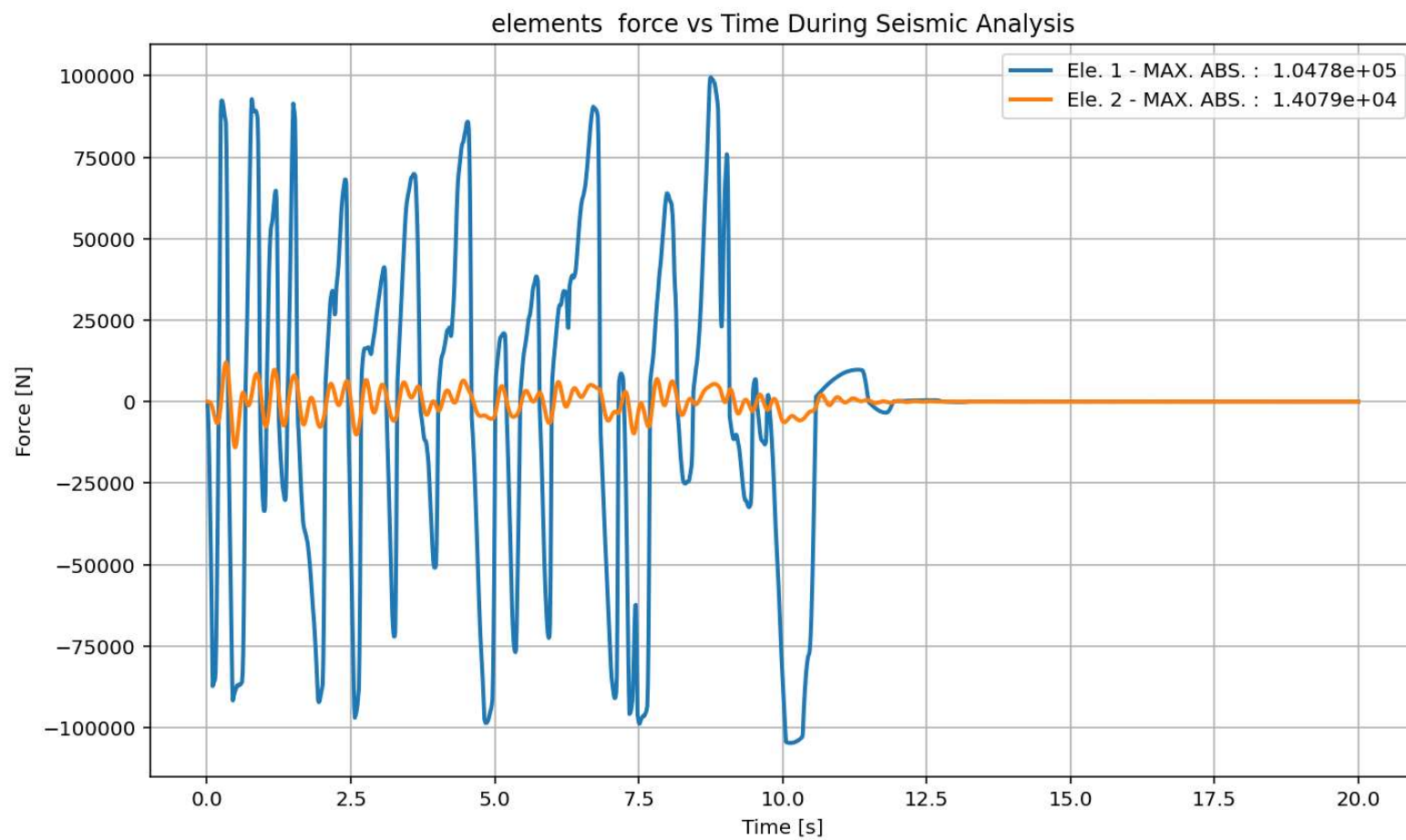
Inline Conda: anaconda3 (Python 3.12.7) ✓ LSP: Python Line 769, Col 5 UTF-8 CRLF RW Mem 51%

Base Reaction and Displacment of Structure During Seismic Analysis

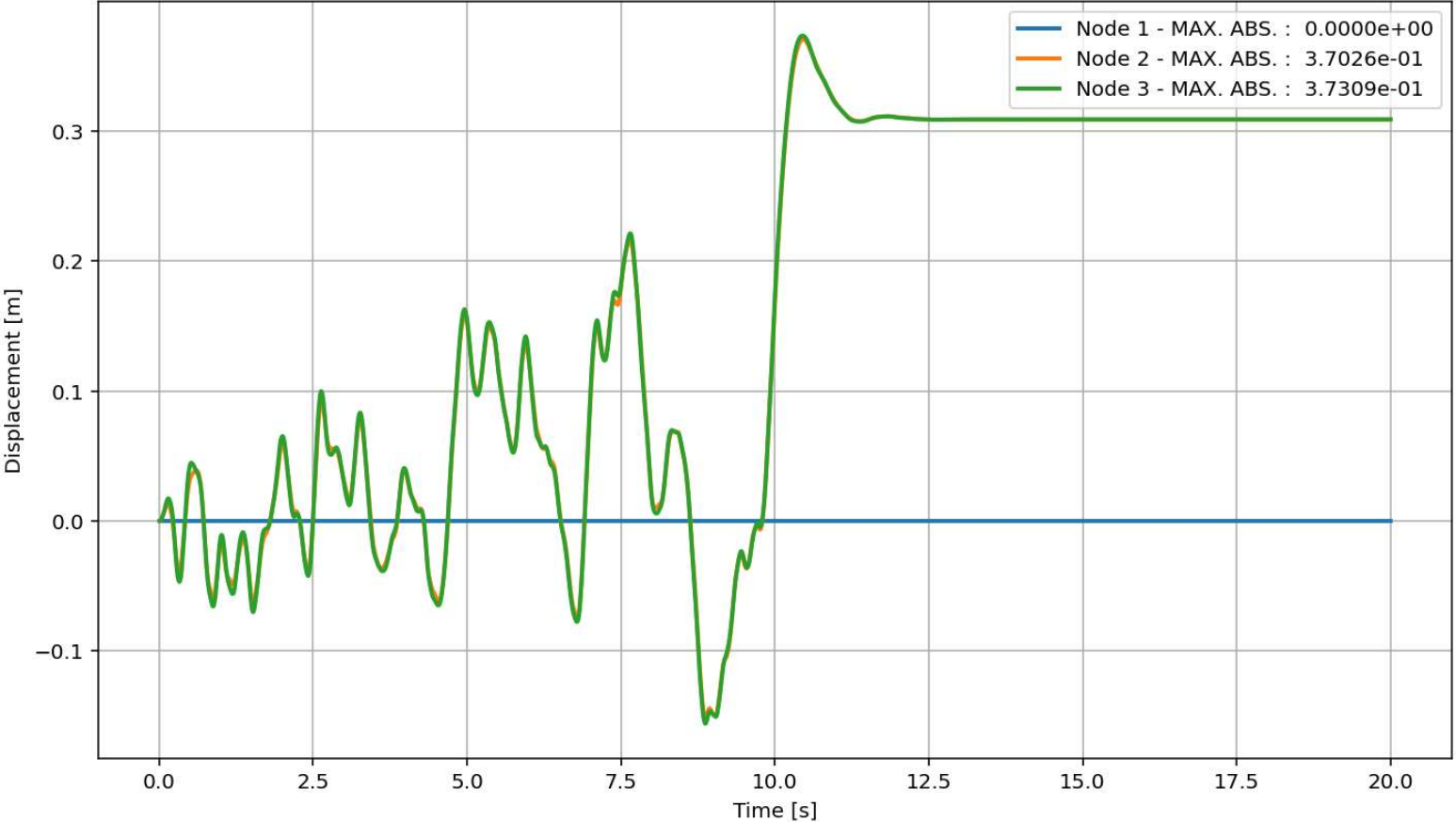


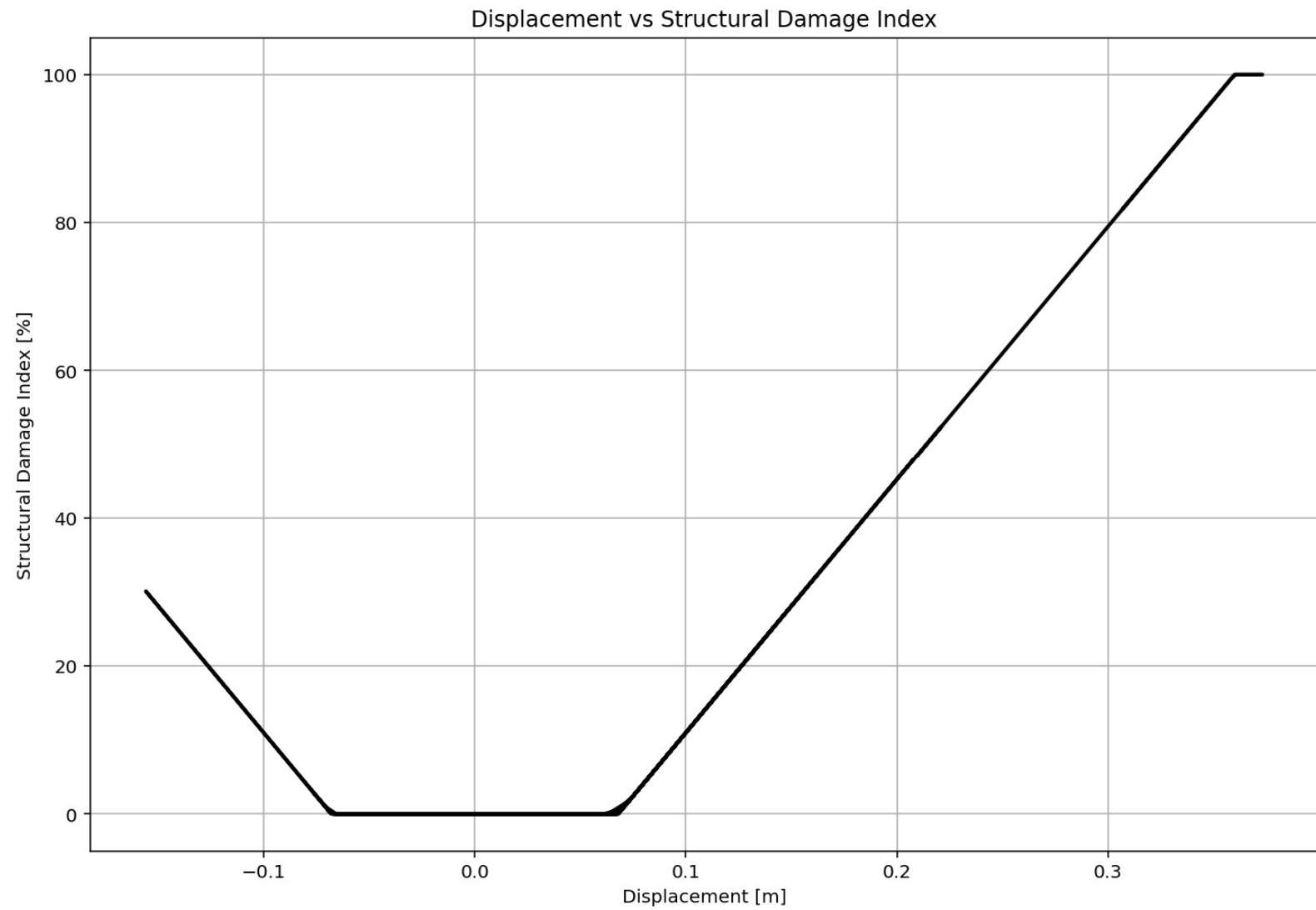


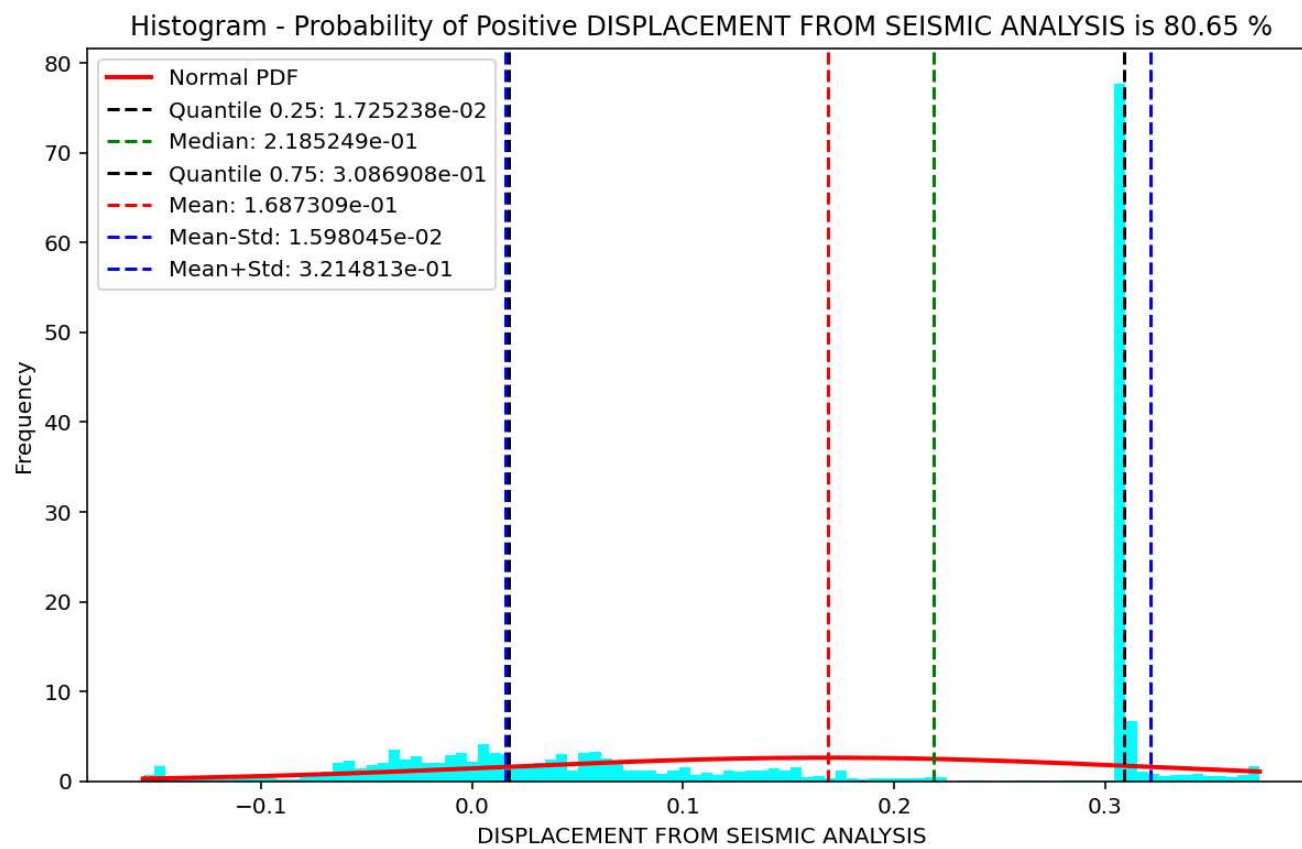




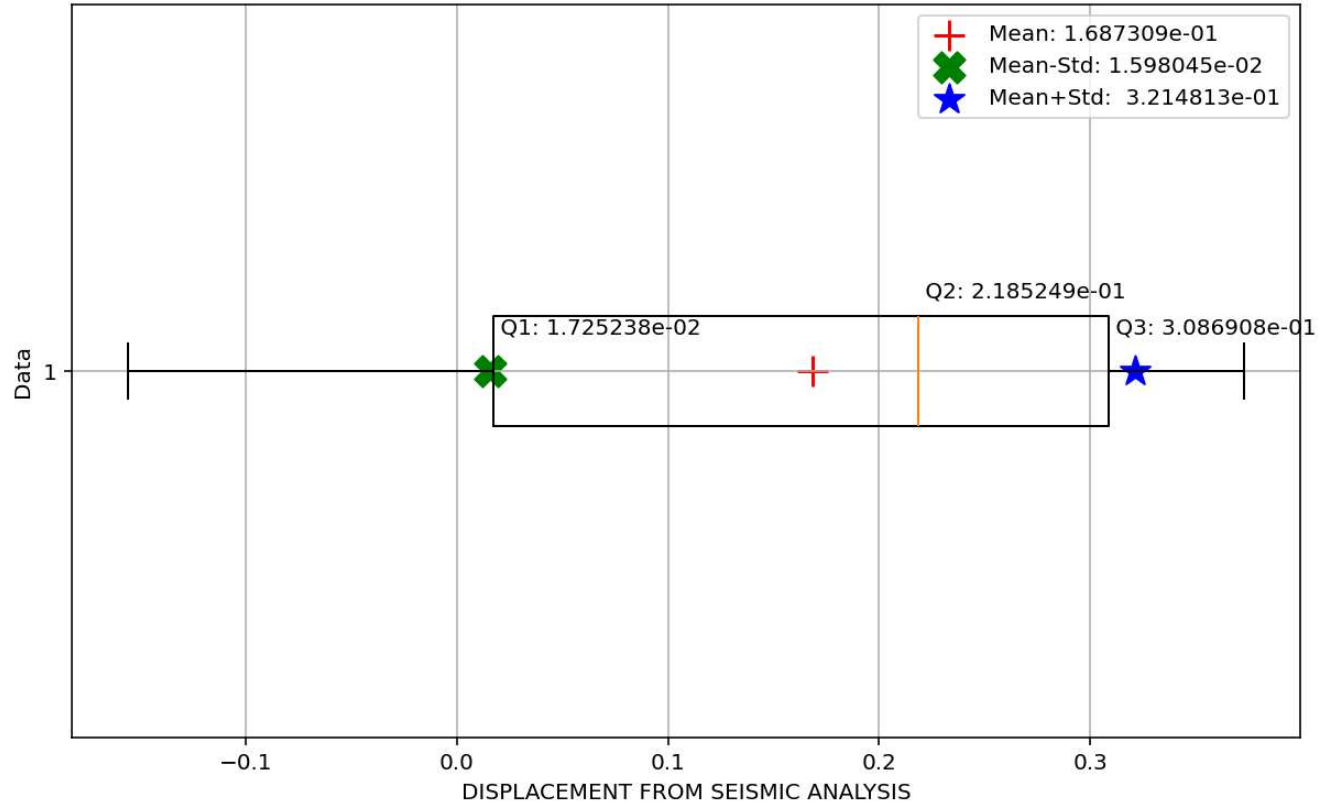
Node Displacements vs Time for During Seismic Analysis

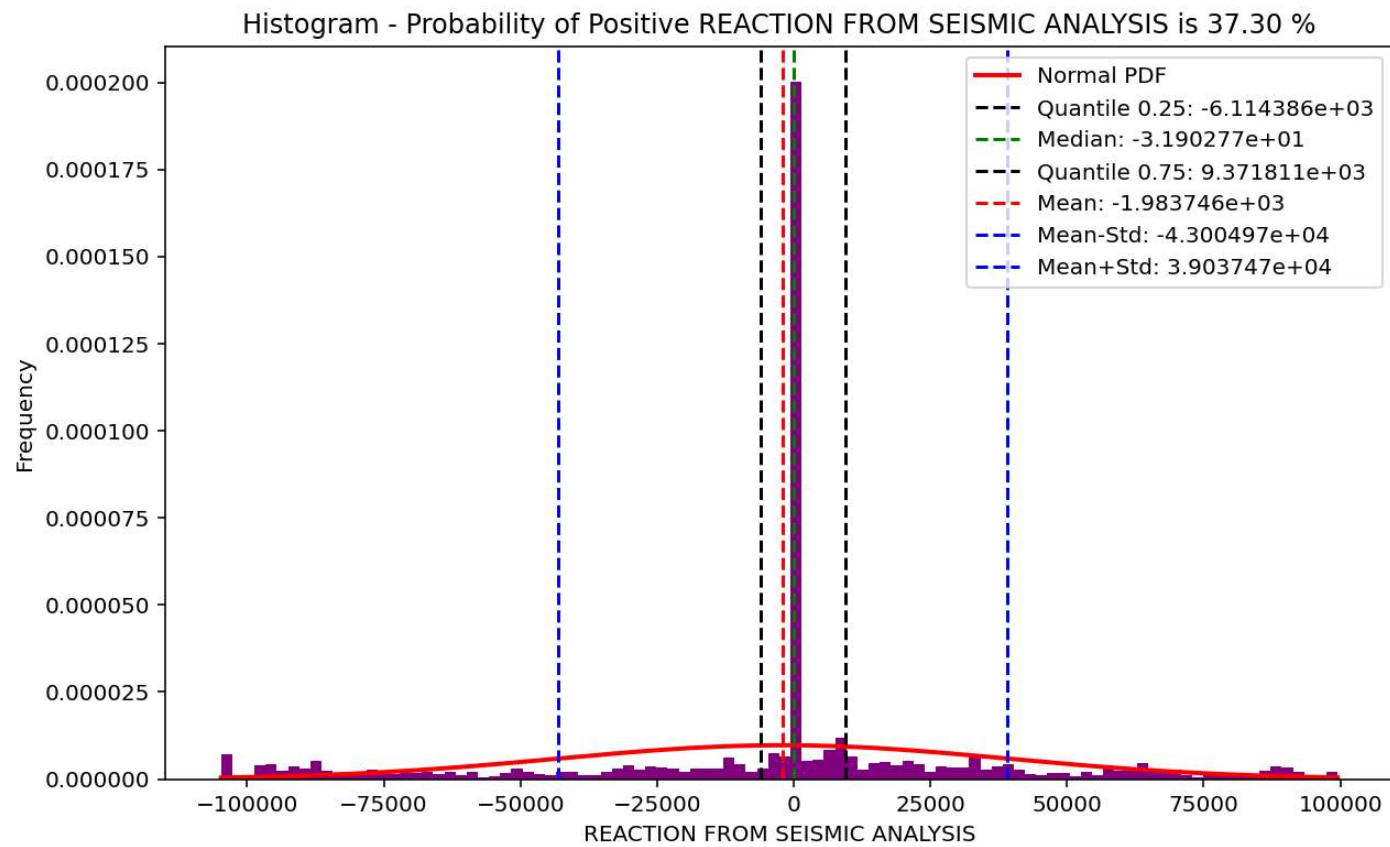




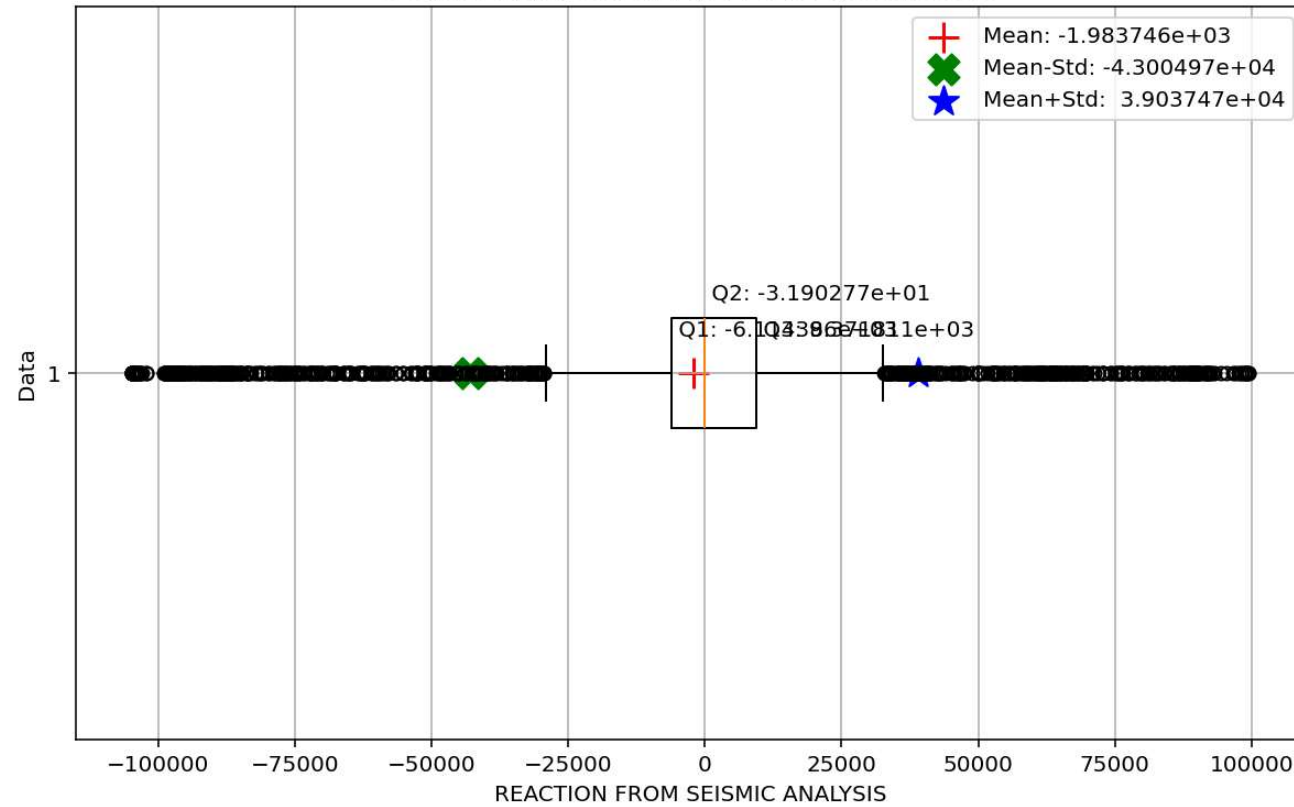


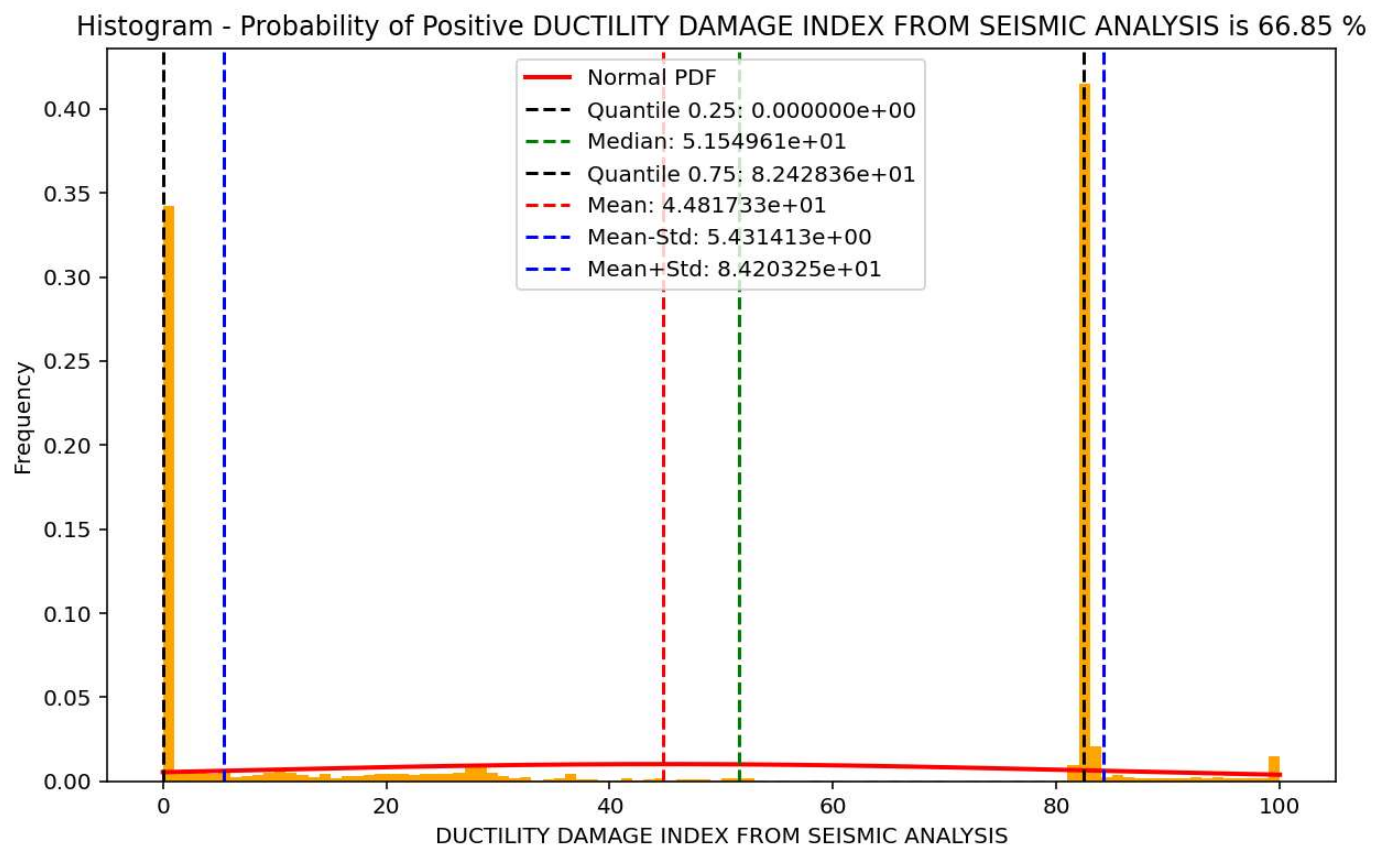
Boxplot of DISPLACEMENT FROM SEISMIC ANALYSIS

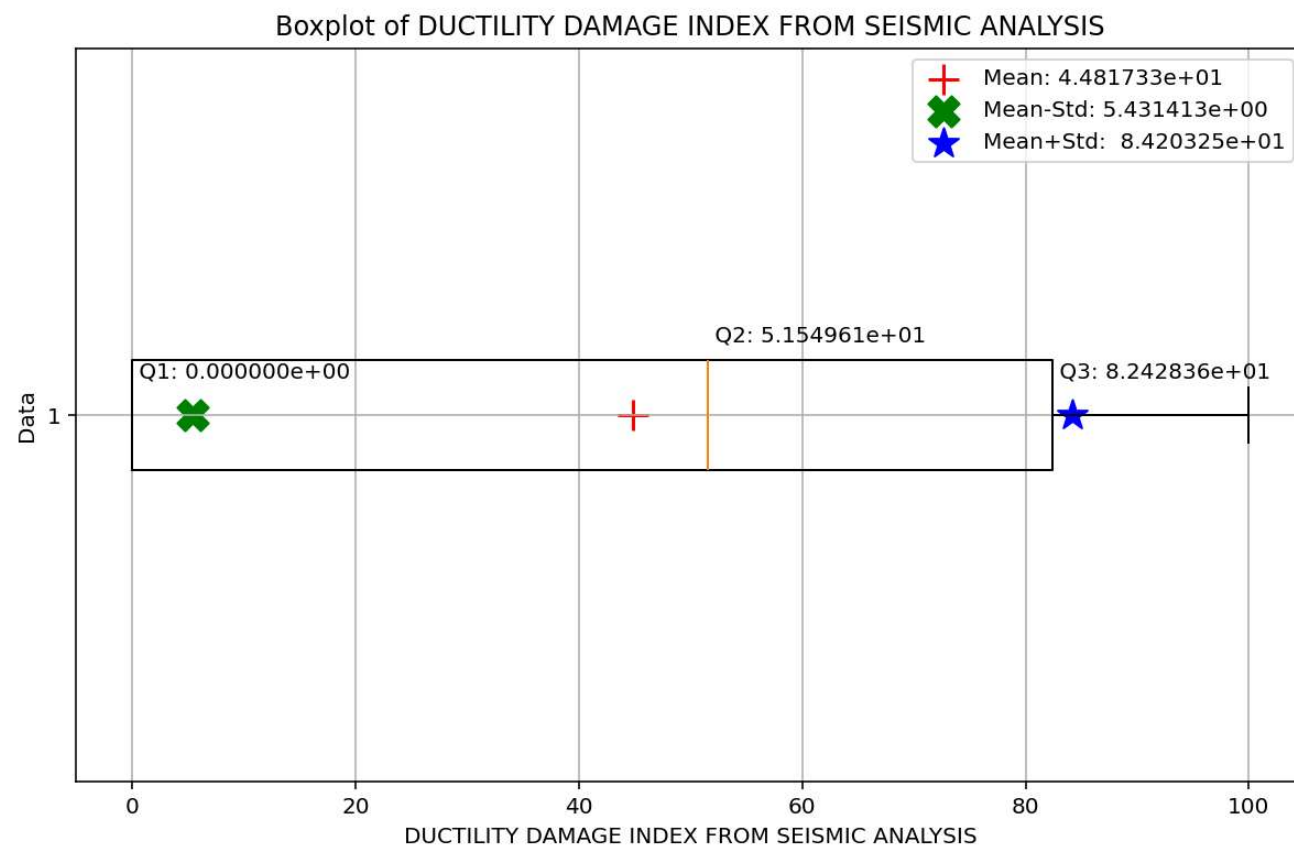




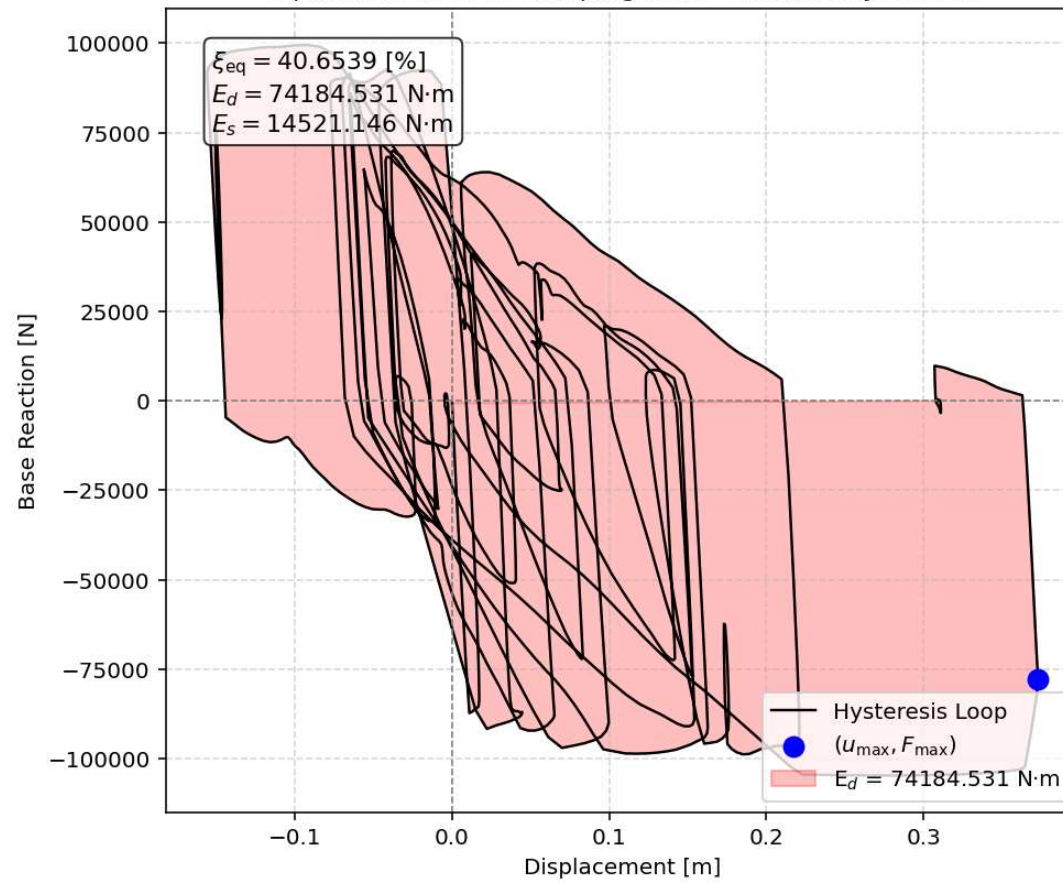
Boxplot of REACTION FROM SEISMIC ANALYSIS



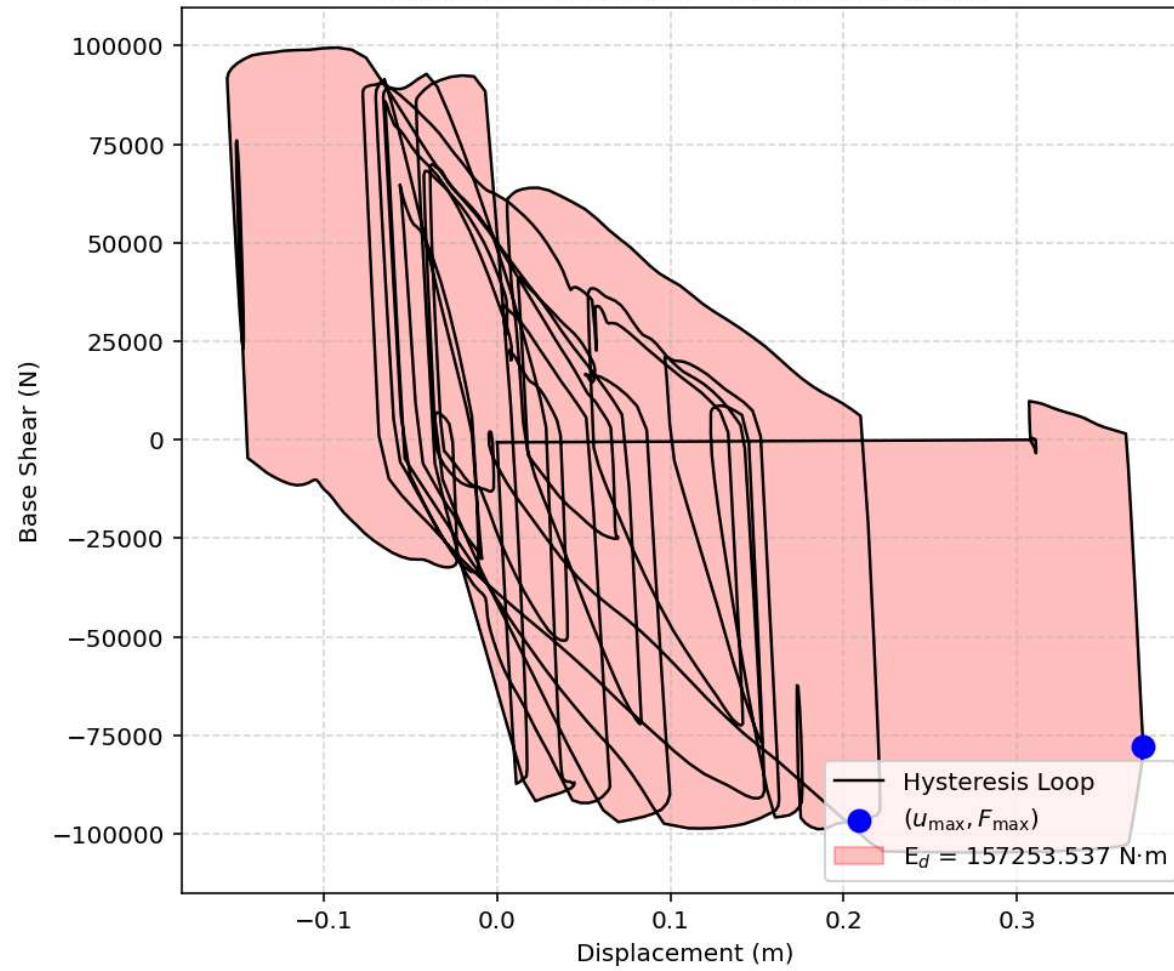


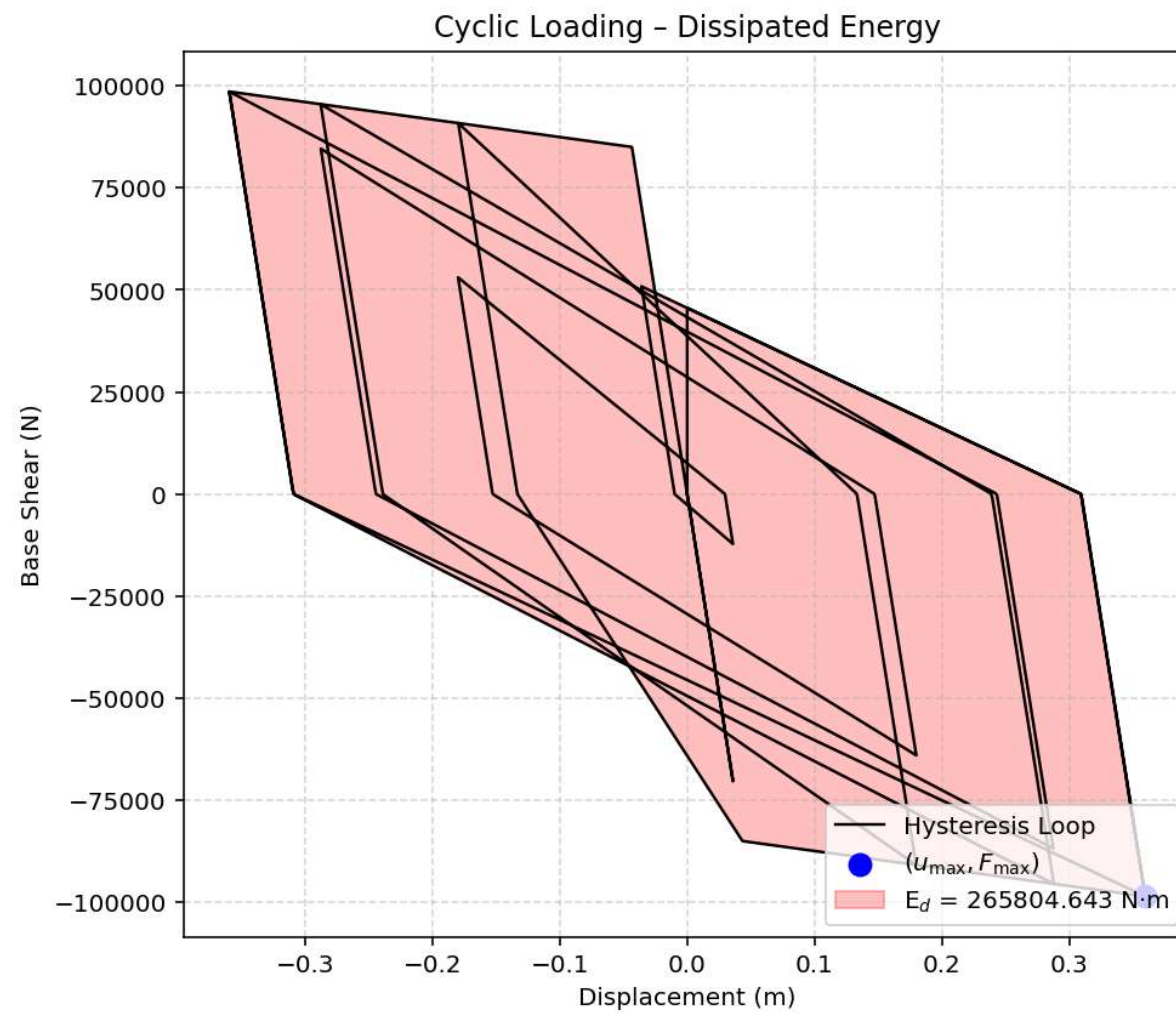


Equivalent viscous damping ratio - Seismic Hysteresis



Earthquake Response - Dissipated Energy

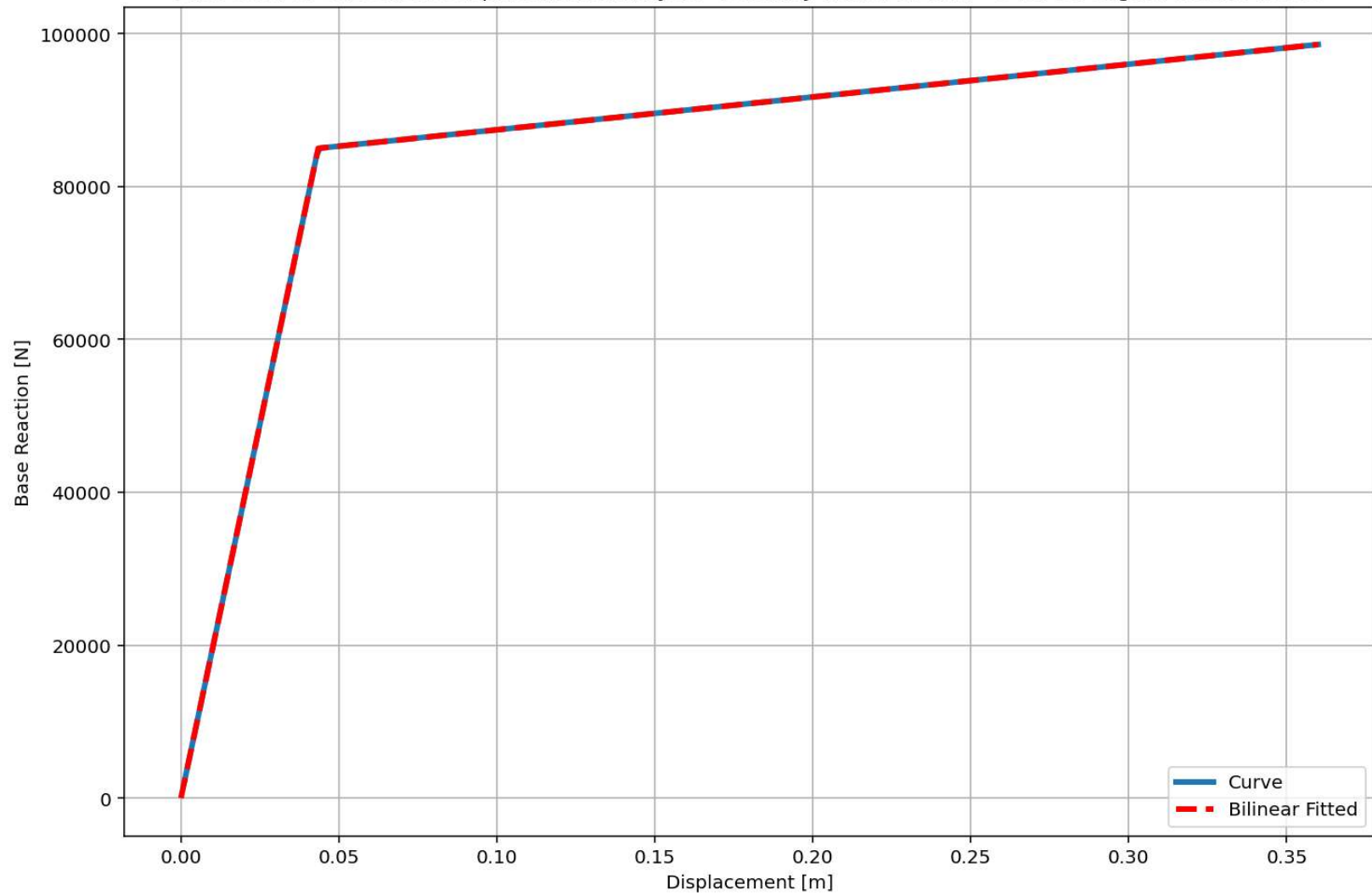




Dissipated Energy from Earthquake= 157253.54 N·m
Dissipated Energy from Cyclic Displacement= 265804.64 N·m

DISSIPATED ENERGY CAPACITY INDEX: 59.161 [%]
ZONE 6: SEVERE-LOW DAMAGE

Last Data of BaseShear-Displacement Analysis - Ductility Ratio: 8.2872 - Over Strength Factor: 1.1599



+=====+

= Analysis curve fitted =

Disp Base Shear

[[0.00000000e+00 0.00000000e+00]

[4.34404346e-02 8.49921547e+04]

[3.60000000e-01 9.85790074e+04]]

+=====+

+-----+

Structure Elastic Stiffness : 1956521.74

Structure Plastic Stiffness : 273830.58

Structure Tangent Stiffness : 42920.37

Structure Ductility Ratio : 8.29

Structure Over Strength Factor: 1.16

+-----+

Over Strength Coefficient (Ω_0): 1.1599

Displacement Ductility Ratio (μ): 8.2872

Ductility Coefficient (R_μ): 8.2872

Structural Behavior Coefficient (R): 9.6120

Structural Ductility Damage Index in X Direction: 104.1352 (%)

Seismic Fragility Curves by Damage State from DYN. TIME-HISTORY ANA. DUCTILITY DAMAGE INDEX [%]

